

**ECOLE POLYTECHNIQUE
ECOLES NORMALES SUPERIEURES**

CONCOURS D'ADMISSION 2025

**MARDI 15 AVRIL 2025
14h00 - 18h00
FILIERES MP-MPI - Epreuve n° 4
INFORMATIQUE A (XULSR)**

Durée : 4 heures

***L'utilisation des calculatrices n'est pas autorisée pour
cette épreuve***

*Cette composition ne concerne qu'une partie des candidats de la
filière MP, les autres candidats effectuant simultanément la composition
de Physique et Sciences de l'Ingénieur.*

*Pour la filière MP, il y a donc deux enveloppes de Sujets pour
cette séance.*

Arbres de classification d'arbres

Le sujet comporte 12 pages.

Vue d'ensemble du sujet.

Ce sujet porte sur la réalisation d'arbres d'identification des plantes basés sur des observations morphologiques. À partir d'une base de connaissance botaniste, il s'agit de réaliser un arbre dont les feuilles sont des diagnostics (typiquement des noms d'espèce) et les nœuds des *caractères morphologiques* (par exemple la couleur des fleurs). Un nœud a autant de fils que le caractère a de valeurs possibles (jaune, bleue, etc.).

Étant donnée une base de connaissance, il s'agit de construire un arbre dont la hauteur moyenne est la plus petite possible, ce qui revient à minimiser le nombre moyen d'observations nécessaires pour identifier une plante. La difficulté revient donc à choisir à chaque étape le caractère le plus *discriminant*, c'est-à-dire celui qui va le mieux séparer l'espace de recherche.

Pour tenir compte de la diversité au sein d'une espèce, nous adoptons une approche probabiliste basée sur un raisonnement bayésien : la valeur d'un caractère morphologique pour une espèce donnée sera probabiliste. Par exemple, la couleur des fleurs d'une certaine espèce pourra être bleue ou jaune, avec certaines probabilités.

La partie I introduit les éléments de raisonnement bayésien nécessaires à la construction des arbres. La partie II propose une approche par énumération pour générer un arbre qui minimise la hauteur moyenne. La partie III propose une heuristique basée sur l'entropie de Shannon pour éviter la construction de tous les arbres. La partie IV revient sur le calcul exact en proposant une optimisation permettant de réduire l'espace de recherche. Les parties III et IV sont indépendantes l'une de l'autre. Enfin, la partie V propose une approche logique pour étendre la notion de caractère. Cette dernière partie ne dépend que de la première.

Rappels de probabilités

Une **mesure** sur un ensemble fini X est une fonction $\varphi : X \rightarrow \mathbb{R}^+$. Sa **masse** est la quantité $\sum_{x \in X} \varphi(x)$. Une mesure de masse 1 est appelée une **distribution de probabilité**, ou simplement **distribution**. On note $D(X)$ l'ensemble des distributions sur l'ensemble fini X . Pour $x \in X$, la **dirac** en x , notée δ_x , est la distribution qui associe à x le poids 1, et zéro à tous les autres éléments. Étant donnés une famille de distributions $d_1, \dots, d_n \in D(X)$ et des poids $p_1, \dots, p_n \in [0; 1]$ tels que $\sum_{1 \leq i \leq n} p_i = 1$, on note $p_1 \cdot d_1 + \dots + p_n \cdot d_n$ la distribution $d \in D(X)$ telle que $d(x) = p_1 \times d_1(x) + \dots + p_n \times d_n(x)$ pour tout $x \in X$.

Si X est un ensemble, on notera $\mathcal{P}(X)$ l'ensemble de ses parties.

Rappels d'OCaml

Dans le reste du sujet, on pourra utiliser les fonctions suivantes de la bibliothèque standard OCaml pour les listes :

- `List.hd [e1;...;en]` renvoie `e1`, et `List.hd []` lève une exception.
- `List.tl [e1;...;en]` renvoie `[e2;...;en]`, et `List.tl []` lève une exception.
- `List.length l` renvoie la longueur (nombre d'éléments) de la liste `l`.
- `List.init n f` renvoie la liste `[f 0;...;f (n-1)]` pour `n` positif.
- `List.assoc k [(k1,v1);...;(kn,vn)]` renvoie le premier `vi` tel que `k = ki` s'il existe, et lève l'exception `Not_found` sinon.
- `List.find f [e1;...;en]` renvoie le premier `ei` tel que `f ei = true` s'il existe, et lève l'exception `Not_found` sinon.
- `List.map f [e1;...;en]` renvoie `[f e1;...;f en]`.
- `List.filter f l` renvoie la liste des éléments `e` de `l` tels que `f e = true`, dans l'ordre.
- `List.concat [l1;...;ln]` renvoie `l1@...@ln`, la concaténation des listes `li`.
- `List.concat_map f l` renvoie `List.concat (List.map f l)`.
- `List.fold_left f a0 [e1;...;en]` renvoie `an` où `ak+1 = f ak ek+1` pour $0 \leq k < n$.

Pour les tableaux, on pourra utiliser les fonctions suivantes :

- `Array.length a` renvoie la longueur (nombre d'éléments) du tableau `a`.
- `Array.init n f` renvoie le tableau `[|f 0;...;f (n-1)|]` pour `n` positif.
- `Array.of_list [e1;...;en]` renvoie le tableau `[|e1;...;en|]`.

On rappelle enfin que la fonction `log : float -> float` est disponible en OCaml pour calculer un logarithme naturel.

Partie I : Description probabiliste des plantes

On se donne un nombre de caractères $N \in \mathbb{N}^*$. Un **caractère** sera un entier $i \in \{0, 1, \dots, N-1\}$. Pour chaque caractère i , on se donne un ensemble fini V_i de **valeurs** possibles pour ce caractère.

Pour nos exemples, nous prendrons $N = 2$ et deux valeurs pour chaque caractère : $V_0 = \{4, 5\}$ représentera le nombre de pétales des fleurs, et $V_1 = \{\text{bleu}, \text{blanc}\}$ représentera leur couleur.

Nous utiliserons une représentation probabiliste des espèces pour modéliser la variabilité intra-espèce, ainsi que la variabilité des interprétations lors des observations : on peut trouver au sein d'une même espèce des individus à fleurs bleues et d'autres à fleurs blanches ; par ailleurs, une couleur bleue très claire pourra être interprétée comme bleue ou blanche en fonction de l'observateur. Ainsi, chaque espèce sera décrite par des valeurs possibles pour chaque caractère, organisées en distributions de probabilité : une **espèce** est un N -uplet $s = (s_0, \dots, s_{N-1})$ où $s_i \in D(V_i)$. Intuitivement, $s_i(v)$ représente la probabilité que la valeur v soit observée pour le caractère i chez un individu de l'espèce. On suppose donné un ensemble fini d'espèces $\mathcal{S}$.

Dans nos exemples, $\mathcal{S}$ sera composé des trois espèces suivantes, décrites en utilisant des diracs δ :

$$\begin{aligned} \text{myosotis} &= (\delta_5, 0, 8 \cdot \delta_{\text{bleu}} + 0, 2 \cdot \delta_{\text{blanc}}) \\ \text{lin} &= (\delta_5, \delta_{\text{bleu}}) \\ \text{gaillet} &= (\delta_4, \delta_{\text{blanc}}) \end{aligned}$$

On définit l'ensemble Ω suivant, dont les éléments modélisent des individus, décrits par leur espèce et les valeurs de leurs caractères :

$$\Omega \stackrel{\text{def}}{=} \{(s, v_0, \dots, v_{N-1}) \mid s \in \mathcal{S}, v_i \in V_i \text{ pour tout } i\}$$

On considérera l'espace probabilisable $(\Omega, \mathcal{P}(\Omega))$ obtenu en munissant Ω de la tribu discrète. On utilisera plusieurs lois de probabilité sur cet espace, définies en fonction d'un **état** $\sigma \in D(\mathcal{S})$: intuitivement, l'état indiquera la probabilité d'observer chaque espèce. La **loi de probabilité P^σ induite par un état σ** est définie comme l'unique loi satisfaisant l'équation suivante pour tout $(s, v_0, \dots, v_{N-1}) \in \Omega$:

$$P^\sigma(\{(s, v_0, \dots, v_{N-1})\}) = \sigma(s) \times \prod_{0 \leq i < N-1} s_i(v_i)$$

On notera S la variable aléatoire qui à $(s, v_0, \dots, v_{N-1}) \in \Omega$ associe s . Pour chaque caractère i , on notera C_i la variable aléatoire qui à $(s, v_0, \dots, v_{N-1}) \in \Omega$ associe v_i . On pourra ainsi écrire, par exemple, $P^\sigma(C_i = v \mid S = s)$, qui représente (dans l'état σ) la probabilité (conditionnelle) que le caractère i prenne la valeur v chez un individu de l'espèce s .

Préliminaires probabilistes. On établit tout d'abord quelques résultats élémentaires sur lesquels s'appuiera notre méthode de classification.

Question 1. On considère l'état $\sigma_0 = \{\text{gaillet} : 50\%, \text{myosotis} : 30\%, \text{lin} : 20\%\}$ sur les espèces de notre exemple. Quelle est la probabilité $P^{\sigma_0}(C_1 = \text{blanc})$ d'observer des fleurs blanches dans cet état ? Pour chaque espèce s , que vaut la probabilité $P^{\sigma_0}(S = s \mid C_1 = \text{blanc})$ d'observer un individu de l'espèce s sachant que celui-ci a des fleurs blanches ? Aucune justification n'est attendue.

Question 2. Étant donné un état $\sigma \in D(\mathcal{S})$, une espèce $s \in \mathcal{S}$, un caractère i et une valeur $v \in V_i$, exprimer simplement les probabilités $P^\sigma(S = s)$ et $P^\sigma(C_i = v)$, ainsi que $P^\sigma(C_i = v \mid S = s)$ quand elle est bien définie. Les réponses devront être justifiées.

Question 3. Étant donné un état $\sigma \in D(\mathcal{S})$, un caractère i , une valeur $v \in V_i$ et une espèce $s \in \mathcal{S}$, exprimer $P^\sigma(S = s \mid C_i = v)$. Sous quelle condition cette probabilité est-elle bien définie ?

Pour un état σ , un caractère i et une valeur $v \in V_i$, on définit **l'état mis à jour** $\sigma[i := v]$ comme la distribution qui à chaque espèce s associe $P^\sigma(S = s \mid C_i = v)$.

Question 4. Pour un état σ , deux caractères $i \neq j$ et des valeurs $v \in V_i$ et $v' \in V_j$, montrer que $P^\sigma(S = s \mid C_i = v, C_j = v') = P^{\sigma[i:=v]}(S = s \mid C_j = v')$ quand ces probabilités sont bien définies.

Le résultat précédent nous indique que, si l'on souhaite évaluer dans un état σ la probabilité d'observer une certaine espèce sachant que $C_i = v$ et $C_j = v'$, il nous suffit de calculer une probabilité conditionnée seulement par $C_j = v'$ mais dans l'état $\sigma[i := v]$. En allant plus loin, si l'on pose $\sigma' = \sigma[i := v]$ et $\sigma'' = \sigma'[j := v']$, on a $P^\sigma(S = s \mid C_i = v, C_j = v') = \sigma''(s)$. On admettra que ce résultat se généralise à un nombre arbitraire d'observations. Ainsi, il est possible de procéder à la reconnaissance de plantes en calculant des états mis à jours par des observations successives, jusqu'à atteindre un état de la forme δ_s ou ne plus pouvoir faire de nouvelle observation.

Choix des représentations pour le code. Un caractère sera naturellement représenté par un entier OCaml. On représentera aussi les valeurs par des entiers, en supposant que chaque V_i est de la forme $\{0, 1, \dots, k_i - 1\}$, ce qui revient à numéroté les valeurs possibles. On pose ainsi :

```
type caractere = int
type valeur = int
```

On suppose donné le nombre de valeurs pour chaque caractère sous la forme d'un tableau :

```
val num : int array    (* num.(i) = k_i, i.e., V_i = {0,1,...,num.(i)-1} *)
```

Enfin, on considèrera que la liste des caractères $[0;1;\dots;N-1]$ est définie en variable globale à partir du tableau `num` :

```
let caracteres : caractere list = List.init (Array.length num) (fun i -> i)
```

Pour notre exemple avec deux caractères, on aura ainsi :

```
(* Nombre de valeurs possibles pour le nombre de pétales et pour leur couleur *)
let num = [|2; 2|]
```

```
(* Nombre de pétales *)
let nb_petales : caractere = 0
let quatre, cinq = 0, 1
```

```
(* Couleur *)
let couleur_petales : caractere = 1
let bleu, blanc = 0, 1
```

Une mesure μ sur X sera représentée par une liste de couples $(x, \mu(x))$ avec $x \in X$ et $\mu(x) > 0$, sans doublons. **En particulier, les éléments de poids nuls ne sont pas inclus dans la liste, et l'ordre n'a pas d'importance.**

```
type 'x mesure = ('x * float) list
let dirac v = [(v, 1.)]
```

Enfin, on codera comme suit les espèces et états :

```
type espece = { name: string; mesures: valeur mesure array }
type etat = espece mesure
```

Pour notre exemple, on aura ainsi :

```
let myosotis : espece =
  { name = "myosotis";
    mesures = [| dirac cinq; [(bleu, 0.8);(blanc, 0.2)] |] }
let lin : espece =
  { name = "lin";
    mesures = [| dirac cinq; dirac bleu |] }
let gaillet : espece =
  { name = "gaillet";
    mesures = [| dirac quatre; dirac blanc |] }
let sigma0 : etat = [(gaillet, 0.5);(myosotis, 0.3);(lin, 0.2)]
```

Question 5. Écrire les fonctions suivantes :

```
val proba_de      : 'x -> 'x mesure -> float
val somme_ponderee : 'x mesure -> ('x -> float) -> float
val repondere     : 'x mesure -> ('x -> float) -> 'x mesure
```

La première renvoie le poids d'un élément selon une mesure donnée. La seconde renvoie la somme d'une mesure μ pondérée par une fonction $f : X \rightarrow \mathbb{R}$, définie par $\sum_{x \in X} \mu(x)f(x)$. La troisième repondère une mesure sur X par une fonction $f : X \rightarrow \mathbb{R}$: le poids de $x \in X$ passe de $\mu(x)$ à $\mu(x)f(x)$.

Question 6. Écrire deux fonctions

```
val proba_espece : espece -> caractere -> valeur -> float
val proba_etat   : etat -> caractere -> valeur -> float
```

où `proba_espece s i v` renvoie la probabilité $P^{\delta_s}(C_i = v)$ et `proba_etat  $\sigma$  i v` renvoie $P^\sigma(C_i = v)$.

Question 7. Écrire une fonction

```
val reponse : etat -> caractere -> valeur -> etat
```

qui, étant donnés un état σ , un caractère i et une valeur $v \in V_i$, renvoie l'état $\sigma[i := v]$ si celui-ci est bien défini, et lève une exception sinon.

Partie II : Arbre optimal

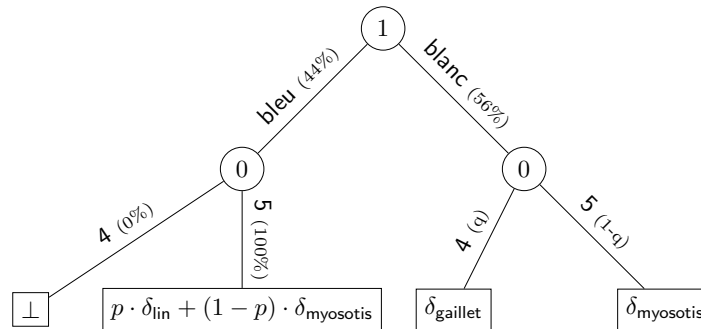
Dans cette seconde partie, on définit la notion d'arbre de décision, et on s'intéresse au calcul d'un arbre de décision optimal en un certain sens. On introduit les types de données suivants pour coder nos arbres :

```
type noeud = {caractere: caractere; enfants: (float * arbre) array}
and arbre =
  | Noeud of noeud      (* On pose une question sur un caractère. *)
  | Feuille of etat     (* On a atteint une feuille, on donne l'état. *)
  | Impossible         (* On a atteint une feuille impossible :
                        aucune espèce ne correspond aux réponses données. *)
```

Les valeurs de type `arbre` ne représentent pas toutes des arbres de décision. Une valeur t de type `arbre` est **un arbre de décision** pour un ensemble de caractères C et un état σ lorsque :

- L'arbre t n'est pas `Impossible`.
- Si t est de la forme `Feuille  $\sigma'$` , alors $\sigma' = \sigma$ et (au moins) l'une des deux conditions suivantes est satisfaite :
 - σ est une distribution dirac (il n'y a plus qu'une seule espèce possible) ;
 - $C = \emptyset$ (il n'y a plus de caractère disponible pour discriminer).
- Si t est de la forme `Noeud {caractere= $i$ ; enfants= $e$ }` alors i est un élément de C , e est un tableau à `num.(i)` éléments, et pour chaque valeur possible $0 \leq v < \text{num.}(i)$, $e.(v)$ est de la forme $(P^\sigma(C_i = v), t')$ avec :
 - si $P^\sigma(C_i = v) = 0$, alors t' est `Impossible` ;
 - sinon, t' est un arbre de décision pour l'ensemble de caractères $C \setminus \{i\}$ et l'état $\sigma[i := v]$.

Dans le contexte de l'exemple de la partie I, on représente graphiquement ci-dessous un arbre de décision pour l'ensemble complet de caractères $\{0, 1\}$ et l'état σ_0 de la question 1 (pour des valeurs de p et q non précisées) :



Les nœuds sont étiquetés par des caractères et les feuilles par des états ; la notation $\perp$ est utilisée pour représenter les cas impossibles où aucune espèce ne peut correspondre aux valeurs observées. Chaque arc est étiqueté par la valeur du caractère correspondant à l'enfant ainsi que la probabilité que cette valeur soit observée.

On définit la **hauteur moyenne** d'un arbre t , notée $h(t)$, comme suit :

- Si t est de la forme `Feuille  $\sigma$` ou `Impossible`, alors $h(t) = 0$.

— Si t est de la forme `Noeud {caractere=i;enfants=e}`, et si l'on note $(p_v, t_v) = \mathbf{e}.(v)$, alors :

$$h(t) = 1 + \sum_{0 \leq v < \text{num.}(i)} p_v \times h(t_v)$$

La hauteur moyenne correspond au nombre moyen de questions avant d'atteindre une feuille. Sur l'exemple précédent, la hauteur moyenne est de deux.

Question 8. Sur l'exemple de la question 1, énumérer tous les arbres de décision possibles pour les caractères $\{0, 1\}$ et l'état σ_0 , et donner leur hauteur moyenne — pour l'arbre déjà représenté ci-dessus, on pourra se contenter de préciser les valeurs de p et q . Quel est l'arbre de décision avec la hauteur moyenne minimale ?

Dans la suite de cette partie, on se propose de calculer naïvement un arbre de décision de hauteur moyenne minimale. On dira qu'un arbre t est **optimal pour C et σ** quand c'est un arbre de décision pour C et σ tel que $h(t)$ est minimale parmi les hauteurs moyennes de tous les arbres de décision pour C et σ . Dans un contexte où C et σ sont clairs, on dira simplement que t est optimal.

Question 9. Écrire une fonction

```
val choix_possibles : 'x list array -> 'x array list
```

qui renvoie la liste des tableaux obtenus en choisissant un élément dans chaque liste du tableau d'entrée. Par exemple :

```
choix_possibles [| [1; 2]; [3; 4; 5] |] = [
  [| 1; 3 |]; [| 1; 4 |]; [| 1; 5 |];
  [| 2; 3 |]; [| 2; 4 |]; [| 2; 5 |];
]
```

Question 10. Écrire une fonction qui, étant donné un état initial σ_0 , renvoie la liste de tous les arbres de décision possibles pour σ_0 et l'ensemble de tous les caractères :

```
val enumere : etat -> arbre list
```

On rappelle que l'ensemble $\{0, 1, \dots, N-1\}$ de tous les caractères est représenté par la liste `caracteres` déclarée en variable globale.

Question 11. Écrire une fonction

```
val argmin : 'x list -> ('x -> float) -> 'x
```

telle que `argmin l f` renvoie un élément de l qui minimise f , si la liste est non-vide. En déduire une fonction qui, étant donné un état initial σ_0 , renvoie un arbre de décision optimal pour σ_0 et l'ensemble de tous les caractères :

```
val arbre_optimal : etat -> arbre
```

Partie III : Algorithme glouton

Afin d'éviter l'énumération de tous les arbres de décision dans l'algorithme de la partie précédente, on cherche un algorithme glouton : l'idée générale est de décider localement du caractère à choisir en fonction de l'état courant, plutôt que d'envisager tous les choix possibles. Pour cela, on décide de prendre le caractère qui minimise l'**entropie moyenne**. Avant de définir cette quantité, on introduit l'entropie simple $H(\mu)$ d'une distribution $\mu \in D(X)$, qui est donnée par :

$$H(\mu) \stackrel{\text{def}}{=} - \left(\sum_{x \in X, \mu(x) \neq 0} \mu(x) \ln(\mu(x)) \right)$$

Question 12. Soit X un ensemble de cardinal $k \in \mathbb{N}$. Donner (en justifiant) les valeurs minimale et maximale de H sur $D(X)$ en fonction de k , ainsi que des distributions les atteignant.

Étant donné un caractère i et un état σ , on définit l'entropie moyenne de i dans l'état σ , notée $H_i(\sigma)$, comme l'espérance des entropies sur les distributions obtenues en faisant une observation sur le caractère i :

$$H_i(\sigma) \stackrel{\text{def}}{=} \sum_{v \in V_i} P^\sigma(C_i = v) H(\sigma[i := v])$$

Question 13. Écrire une fonction qui renvoie l'entropie moyenne d'un caractère dans un état donné :

```
val score_caractere : etat -> caractere -> float
```

Question 14. Écrire une fonction qui calcule un arbre de décision en suivant l'algorithme glouton qui construit l'arbre en choisissant à la racine le caractère minimisant l'entropie moyenne, et procède récursivement de la même façon pour les sous-arbres :

```
val glouton : etat -> arbre
```

On se propose maintenant de montrer que l'algorithme glouton n'est pas optimal. On considère une situation avec deux caractères, $V_0 = V_1 = \{\text{oui}, \text{non}\}$, et trois espèces a , b et c :

$$\begin{aligned} a &= (\delta_{\text{oui}}, 0, 5 \cdot \delta_{\text{oui}} + 0, 5 \cdot \delta_{\text{non}}) \\ b &= (\delta_{\text{non}}, \delta_{\text{oui}}) \\ c &= (\delta_{\text{non}}, \delta_{\text{non}}) \end{aligned}$$

On considère enfin l'état initial σ_0 suivant :

$$a : 20\%, \quad b : 40\%, \quad c : 40\%$$

L'espèce a est donc plus rare que les espèces b et c qui sont plus communes.

Question 15. Dessiner les arbres de décision possibles pour l'état σ_0 et les caractères $\{0, 1\}$, et déterminer l'arbre de décision optimal.

Question 16. Calculer les entropies moyennes $H_0(\sigma_0)$ et $H_1(\sigma_0)$. Que peut-on en conclure ? On admettra que $\ln(5) < \frac{12}{5} \ln(2)$.

Partie IV : Optimisation de l'algorithme naïf

On cherche à optimiser l'algorithme naïf de la partie II en évitant certains calculs inutiles. Pour permettre cela, on commence par changer la méthode de recherche d'un arbre optimal, en évitant de construire la liste de tous les arbres de décision possibles. La nouvelle méthode s'appuiera sur la fonction `arbre_optimal_avec_oracle` donnée en Figure 1. Le but de cette fonction est, étant donné un état σ et une liste de caractères C , de renvoyer $(t, h(t))$ où t est un arbre optimal pour C et σ . Il faut bien noter que cette fonction n'est pas récursive, mais qu'elle dispose d'un argument supplémentaire `oracle` dont les appels pourront correspondre à des appels récursifs : il s'agit de *réursion ouverte*.

Question 17. Donner des hypothèses (pré-conditions) aussi précises que possible sur les arguments C , σ et `oracle` de la fonction `arbre_optimal_avec_oracle`, sous lesquelles la fonction termine toujours et renvoie bien $(t, h(t))$ où t est un arbre optimal pour C et σ . On veillera à répondre à cette question après avoir réfléchi aux deux suivantes, où il faudra démontrer puis exploiter la correction de la fonction par rapport à la spécification énoncée ici.

Question 18. Démontrer que la fonction satisfait bien cette spécification, en veillant notamment à énoncer clairement l'invariant de boucle.

Question 19. Écrire une fonction, récursive cette fois-ci, et basée sur `arbre_optimal_avec_oracle`, qui renvoie un arbre de décision optimal pour un état et une liste de caractères donnés :

```
val arbre_optimal : etat -> caractere list -> arbre * float
```

Donner sa spécification et montrer qu'elle la satisfait.

Question 20. On cherche maintenant à optimiser notre algorithme en interrompant la génération d'un arbre dès que son score dépasse celui de l'`arbre_optimal` courant. Pour cela, écrire une nouvelle fonction

```
val arbre_optimal_opt : etat -> caractere list -> float -> (arbre * float) option
```

qui prend un paramètre supplémentaire représentant le score à ne pas dépasser, et renvoie `None` si tous les arbres de décision possibles sont de hauteur moyenne supérieure au score maximal, ou bien un arbre optimal de hauteur moyenne inférieure s'il en existe un. La preuve de correction n'est pas demandée.

```

1  let arbre_optimal_avec_oracle
2      (oracle : etat -> caractere list -> arbre * float)
3      (etat : etat)
4      (caracteres : caractere list) : arbre * float =
5  if caracteres = [] then
6      (Feuille etat, 0.)
7  else
8      let caracteres_a_tester = ref caracteres in
9      let score_optimal = ref infinity in
10     let arbre_optimal = ref None in
11     while !caracteres_a_tester <> [] do
12         let caractere = List.hd !caracteres_a_tester in
13         let score_total = ref 1. in
14         let enfants : (float * arbre) array =
15             Array.init num.(caractere) (fun valeur ->
16                 let p_valeur = proba_etat etat caractere valeur in
17                 let sous_arbre, score_sous_arbre =
18                     if p_valeur = 0. then Impossible, 0. else
19                         let etat' = reponse etat caractere valeur in
20                         let caracteres_restants =
21                             List.filter (fun c -> c <> caractere) caracteres
22                         in
23                             oracle etat' caracteres_restants
24                 in
25                 score_total := !score_total +. p_valeur *. score_sous_arbre;
26                 (p_valeur, sous_arbre))
27         in
28         if !score_total < !score_optimal then
29             (arbre_optimal := Some (Noeud {caractere=caractere;enfants=enfants});
30              score_optimal := !score_total);
31         caracteres_a_tester := List.tl !caracteres_a_tester
32     done;
33     match !arbre_optimal with
34     | Some a -> a, !score_optimal
35     | None -> assert false

```

FIGURE 1 – Fonction `arbre_optimal_avec_oracle`.

Partie V : Observations complexes et formules logiques

Pour aller plus loin, on se propose de représenter des observations complexes sous forme de formules logiques construites à partir des caractères, par exemple “la fleur est bleue et a cinq pétales”. La syntaxe de cette logique est la suivante :

$$\varphi ::= c_i \in V \mid \varphi_1 \wedge \dots \wedge \varphi_m \mid \varphi_1 \vee \dots \vee \varphi_m \quad (0 \leq i < N, V \subseteq V_i, m \geq 0)$$

Cela signifie que l'ensemble des formules est le plus petit ensemble contenant les formules **atomiques** de la forme $c_i \in V$ où $0 \leq i < N$ et $V \subseteq V_i$, et que si $\varphi_1, \dots, \varphi_m$ sont des formules pour $m \geq 0$, alors la **conjonction** $\varphi_1 \wedge \dots \wedge \varphi_m$ et la **disjonction** $\varphi_1 \vee \dots \vee \varphi_m$ sont aussi des formules. Une formule $c_i \in V$ signifie intuitivement que le caractère i prend une valeur dans l'ensemble $V \subseteq V_i$. On définit la formule $\perp$ (la formule fausse) comme la disjonction vide, et $\top$ (la formule vraie) comme la conjonction vide.

Par exemple, avec les ensembles V_0 et V_1 de l'exemple utilisé en partie I, $c_1 \in \{\text{bleu}\} \wedge c_0 \in \{5\}$ est une formule, correspondant à l'énoncé “la fleur est bleue et a cinq pétales”. Ou encore, si $V_i = \{0, 1, 2, 3, 4\}$, $c_i \in \{1\} \wedge c_i \in \{2, 3\}$ est une formule, correspondant à l'énoncé contradictoire “le caractère i vaut 1, et il vaut 2 ou 3”.

On définit la taille d'une formule φ , notée $|\varphi|$, comme suit :

$$\begin{aligned} |c_i \in V| &= 1 + |V| \\ |\varphi_1 \wedge \dots \wedge \varphi_m| &= 1 + m + \sum_{1 \leq i \leq m} |\varphi_i| \\ |\varphi_1 \vee \dots \vee \varphi_m| &= 1 + m + \sum_{1 \leq i \leq m} |\varphi_i| \end{aligned}$$

Dans un premier temps, nous définissons quand une situation certaine (dite déterministe) satisfait une certaine formule. On appelle **modèles déterministes** les éléments de $V_0 \times \dots \times V_{N-1}$. On notera $\vec{v} \in \vec{V}$ pour signifier que $\vec{v}$ est un tel modèle, et dans ce cas on notera v_i la i -ème composante de $\vec{v}$, ce qui revient à dire que $\vec{v} = (v_0, \dots, v_{N-1})$. On définit une relation de **satisfaction** entre les modèles déterministes et les formules, notée $\vec{v} \models \varphi$, comme suit :

$$\begin{aligned} \vec{v} \models c_i \in V & \quad \text{si et seulement si} \quad v_i \text{ appartient à l'ensemble } V \\ \vec{v} \models \varphi_1 \wedge \dots \wedge \varphi_m & \quad \text{si et seulement si} \quad \vec{v} \models \varphi_k \text{ pour tout } k \in \{1, \dots, m\} \\ \vec{v} \models \varphi_1 \vee \dots \vee \varphi_m & \quad \text{si et seulement si} \quad \vec{v} \models \varphi_k \text{ pour un } k \in \{1, \dots, m\} \end{aligned}$$

On étend ensuite cette relation de satisfaction au cadre probabiliste de la manière suivante. Un **modèle probabiliste** est un élément de $D(V_0) \times \dots \times D(V_{N-1})$. Un tel modèle associe une distribution de probabilité plutôt qu'une valeur déterminée à chaque caractère. On remarquera que cette modélisation suppose que les valeurs de deux caractères distincts sont probabilistes mais indépendantes l'une de l'autre. Étant donné une formule φ et un modèle probabiliste $(d_0, \dots, d_{N-1})$, on définit la probabilité que $\vec{d} = (d_0, \dots, d_{N-1})$ satisfasse φ , notée $P(\vec{d} \models \varphi)$, ainsi :

$$P(\vec{d} \models \varphi) \stackrel{\text{def}}{=} \sum_{\vec{v} \in \vec{V}, \vec{v} \models \varphi} \prod_{0 \leq i < N} d_i(v_i)$$

Question 21. Pour deux formules φ et ψ , montrer l'équivalence entre ces deux propositions :

- Pour tout modèle déterministe $\vec{v}$, $\vec{v} \models \varphi$ si et seulement si $\vec{v} \models \psi$.
- Pour tout modèle probabiliste $\vec{d}$, $P(\vec{d} \models \varphi) = P(\vec{d} \models \psi)$.

On dira que φ et ψ sont **équivalentes**, noté $\varphi \equiv \psi$, quand l'une ou l'autre des propositions précédentes est vraie.

Dans les questions qui suivent, on reprend le codage OCaml des distributions, et on représente directement les formules via le type suivant :

```
type formule =
  | Feuille of caractere * valeur list
  | Conjonction of formule list
  | Disjonction of formule list
```

On supposera de plus que les listes codant les ensembles de valeurs dans les formules atomiques sont sans doublons. On codera les modèles déterministes (resp. probabilistes) par des tableaux de N valeurs (resp. distributions). On notera enfin K le cardinal maximal des V_i :

$$K \stackrel{\text{def}}{=} \max_{0 \leq i < N} |V_i|$$

Question 22. On considère l'algorithme qui, étant donné un modèle probabiliste $\vec{d}$ et une formule φ , calcule $P(\vec{d} \models \varphi)$ en suivant naïvement sa définition. Quelle est la complexité temporelle de cet algorithme, en fonction de N , K et $|\varphi|$? Un programme OCaml détaillé n'est pas demandé, mais on veillera à en décrire les aspects nécessaires pour justifier l'analyse de complexité.

Dans la suite, nous allons élaborer une méthode de calcul de $P(\vec{d} \models \varphi)$ dont la complexité ne dépend pas de N . Cette méthode s'appuie sur des formes particulières de formules. Une **clause** est une conjonction de formules atomiques concernant des caractères différents. En reprenant les exemples précédents, $c_1 \in \{\text{bleu}\} \wedge c_0 \in \{5\}$ est une clause mais pas $c_i \in \{1\} \wedge c_i \in \{2, 3\}$.

Question 23. Décrire un algorithme qui, étant donné un modèle probabiliste $\vec{d}$ et une clause φ , calcule $P(\vec{d} \models \varphi)$ avec une complexité qui ne dépend que de K et $|\varphi|$, mais pas de N . Justifier la correction de l'algorithme et son analyse de complexité. Il n'est pas nécessaire de donner un programme OCaml détaillé.

Question 24. Étant données deux clauses φ et ψ , montrer que $\varphi \wedge \psi$ est équivalente à une clause que l'on notera $\varphi \bar{\wedge} \psi$.

Question 25. Montrer que pour toute formule φ il existe $m \geq 0$ et des clauses $\varphi_1, \dots, \varphi_m$ telles que $\varphi \equiv \varphi_1 \vee \dots \vee \varphi_m$.

Question 26. Décrire un algorithme qui, étant donné un modèle probabiliste $\vec{d}$ et une formule φ , calcule $P(\vec{d} \models \varphi)$ avec une complexité temporelle (potentiellement non-polynomiale) qui ne dépend que de K et $|\varphi|$ mais pas de N .

En pratique, un tel algorithme est utile car on va avoir des modèles avec de nombreux caractères mais un nombre de valeurs possibles restreint pour chaque caractère, et l'on s'intéressera à des formules de taille limitée.

Fin du sujet.

**ECOLE POLYTECHNIQUE
ECOLES NORMALES SUPERIEURES**

CONCOURS D'ADMISSION 2025

JEUDI 17 AVRIL 2025

08h00 - 12h00

FILIERE MPI - Epreuve n° 7

INFORMATIQUE C (XULSR)

Durée : 4 heures

***L'utilisation des calculatrices n'est pas autorisée pour
cette épreuve***

Couplages maximaux et parfaits

Le sujet comporte 18 pages, numérotées de 1 à 18.

Début de l'épreuve.

Complexité. Par la complexité d'un algorithme, on entend le nombre d'opérations élémentaires (comparaison, addition, soustraction, multiplication, division, affectation, test, etc.) nécessaires à l'exécution de celui-ci dans le pire cas. Lorsque la complexité dépend d'un ou plusieurs paramètres $k_0, \dots, k_r$, on dit que l'algorithme a une complexité en $O(f(k_0, \dots, k_r))$ s'il existe une constante $C > 0$ telle que, pour toutes les valeurs de $k_0, \dots, k_r$ suffisamment grandes (c'est-à-dire plus grandes qu'un certain seuil), pour toute instance du problème de paramètres $k_0, \dots, k_r$ la complexité est au plus $C \times f(k_0, \dots, k_r)$.

Fonctions utiles en OCaml. Dans les questions de programmation en OCaml, il est interdit d'utiliser des fonctions de la bibliothèque qui ne sont pas incluses dans la liste suivante.

- **Option.get**: `'a option -> 'a` renvoie `v` si l'argument de type `'a option` est égal à `Some v`, et lève une exception sinon. La complexité est en $O(1)$.
- **List.length**: `'a list -> int` renvoie la taille de la liste donnée en argument. La complexité est en $O(n)$, où n est la longueur de la liste donnée en argument.
- **List.hd**: `'a list -> 'a` renvoie le premier élément de la liste donnée en argument si celle-ci n'est pas vide, et lève une exception sinon. La complexité est en $O(1)$.
- **List.tl**: `'a list -> 'a list` renvoie la liste donnée en argument privée de son premier élément si la liste donnée en argument n'est pas vide, et lève une exception sinon. La complexité est en $O(1)$.
- **List.rev**: `'a list -> 'a list` renvoie la liste donnée en argument dans l'ordre inverse. La complexité est en $O(n)$, où n est la longueur de la liste donnée en argument.
- **List.map**: `('a -> 'b) -> 'a list -> 'b list` renvoie la liste de type `'b list` obtenue en appliquant la fonction de type `'a -> 'b` donnée en premier argument à chaque élément de la liste de type `'a list` donnée en second argument. Si la complexité de `f` est en $O(1)$, alors la complexité de `List.map f lst` est en $O(n)$, où n est la longueur de `lst`.
- **List.mem**: `'a -> 'a list -> bool` renvoie `true` si la valeur donnée en premier argument est un élément de la liste donnée en second argument, et renvoie `false` sinon. La complexité est en $O(n)$, où n est la longueur de la liste donnée en second argument.
- **List.assoc**: `'a -> ('a * 'b) list -> 'b` : cette fonction spécifique aux listes d'association est décrite en page 4.

Fonctions utiles en C. Dans les questions de programmation en C, on pourra supposer que les en-têtes `<stdbool.h>`, `<stdio.h>`, `<stdlib.h>`, et `<math.h>` ont été inclus. En particulier, on pourra utiliser la fonction `double log2(double x)` définie par l'entête `<math.h>`, qui calcule le logarithme en base 2.

Présentation du sujet. Ce sujet concerne les couplages dans les graphes non orientés. La partie I, à composer dans le langage OCaml, concerne la recherche d'un couplage maximal dans un graphe. On étudiera et implémentera un algorithme dû à Edmonds. Les parties II et III, à composer dans le langage C, concernent le problème de l'existence d'un couplage parfait dans un graphe. On y implémente des méthodes algébriques et probabilistes basées sur des calculs de déterminants. La partie II concerne l'implémentation d'un algorithme pour calculer le déterminant d'une matrice carrée dû à Mahajan et Vinay. La partie III exploite cet algorithme pour implémenter un algorithme probabiliste testant l'existence d'un couplage parfait dans un graphe non orienté.

Les parties I et II sont indépendantes. La partie III dépend de la partie II et de notions introduites en partie I.

Graphes non orientés. On travaillera dans les parties I et III sur des graphes non orientés et sans boucles. Un tel graphe G est défini par une paire d'ensembles finis (V, E) , où V est l'ensemble des sommets du graphe et E l'ensemble de ses arêtes. Une arête est une paire non-ordonnée $\{u, v\}$ d'exactly deux sommets $u, v \in V$. Ainsi, les arêtes sont non orientées ($\{u, v\} = \{v, u\}$) et il n'y a pas de boucles ($\{v\} = \{v, v\}$ n'est jamais une arête de G). Une arête $\{u, v\}$ est dite **incidente** aux sommets u et v . On supposera toujours que V est non vide.

Étant donné un graphe $G = (V, E)$:

- Un **chemin** est une suite finie de sommets $v_0, v_1, \dots, v_n$ avec $n \geq 0$ et telle que pour tout $i = 0, \dots, n-1$, on a $\{v_i, v_{i+1}\} \in E$. La **longueur** du chemin $v_0, v_1, \dots, v_n$ est n .
- Un chemin est **élémentaire** si ses sommets sont deux-à-deux distincts.
- Un **cycle** est un chemin $v_0, v_1, \dots, v_n$ de longueur ≥ 3 , avec $v_0 = v_n$ et tel que le chemin $v_1, \dots, v_n$ est élémentaire. Autrement dit, le seul sommet autorisé à apparaître plusieurs fois dans un cycle $v_0, v_1, \dots, v_n$ est le sommet $v_0 = v_n$, qui apparaît exactement deux fois (une fois au début et une fois à la fin).

Partie I : Algorithme d'Edmonds

Cette partie concerne l'algorithme d'Edmonds pour la recherche de couplage maximal.

Question 1. *Écrire une fonction `del: 'a list -> 'a list -> 'a list` qui prend en entrée deux listes `lst1` et `lst2` et renvoie la liste des éléments de `lst1` n'apparaissant pas dans `lst2`. On attend une complexité en $O(m_1 m_2)$ où m_1 est la longueur de `lst1` et m_2 est la longueur de `lst2`, sans la justifier.*

On suppose définie une fonction `join: 'a list -> 'a list -> 'a list` telle que `join lst1 lst2` est une liste sans répétitions contenant l'ensemble de tous les éléments apparaissant dans `lst1` et de tous les éléments apparaissant dans `lst2`.

Nous utilisons des listes d'association pour représenter les graphes. Les listes d'association implémentent une structure de dictionnaire ; il s'agit de listes de paires (a, b) , dont le premier élément a représente la clef et le second élément b représente la valeur associée à cette clef.

Il existe des fonctions OCaml permettant de travailler avec les listes d'association. Dans la suite, nous utiliserons seulement la fonction `List.assoc: 'a -> ('a * 'b) list -> 'b` qui prend en arguments une clef x et une liste d'association `lst`, et renvoie la valeur b associée à cette clef, c'est-à-dire l'élément b du premier couple (x, b) appartenant à `lst`, s'il en existe. La fonction lève une exception si x n'est pas une clef de la liste `lst`. La complexité est en $O(n)$, où n est la longueur de `lst`.

Question 2. *Écrire une fonction `keys: ('a * 'b) list -> 'a list` qui prend en entrée une liste d'association `lst` et qui renvoie la liste de ses clefs. On attend une complexité en $O(m)$ où m est la longueur de `lst`, sans la justifier.*

Nous représentons les graphes en utilisant le type OCaml suivant :

```
type graphe = (int * (int list)) list
```

Un graphe est représenté par une liste de paires (a, lst) où a est un sommet du graphe et `lst` est la liste d'adjacence de ce sommet. Dans la suite, on suppose toujours que :

- les sommets du graphe sont exactement les clefs de la liste d'association ;
- chaque clé de la liste d'association est unique.

Par exemple, le graphe

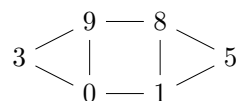
$$2 \text{ --- } 0 \text{ --- } 4$$

peut être représenté par la liste d'association `[(0, [2;4]); (4, [0]); (2, [0])]`.

Couplages

Un **couplage** C dans un graphe G est un ensemble (possiblement vide) d'arêtes de G sans intersections : un sommet de G ne peut appartenir à plus d'une arête dans C .

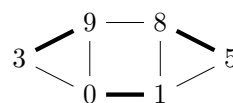
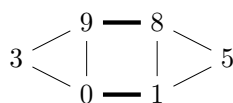
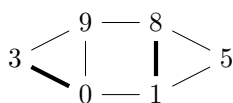
Considérons par exemple le graphe :



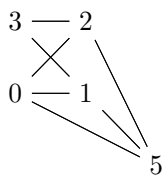
Les ensembles d'arêtes suivants sont des couplages dans ce graphe :

$$\{\{0, 3\}, \{1, 8\}\} \quad \{\{0, 1\}, \{8, 9\}\} \quad \{\{0, 1\}, \{5, 8\}, \{3, 9\}\}$$

Les arêtes d'un couplage sont représentées par des traits plus épais. Par exemple, les trois couplages ci-dessus sont représentés comme suit :



Question 3. On considère le graphe suivant :



Indiquer lesquels des ensembles suivants sont des couplages dans le graphe ci-dessus :

- (1) $\{\{1, 2\}, \{0, 5\}\}$,
- (2) $\{\{1, 3\}, \{0, 5\}\}$,
- (3) $\{\{0, 1\}, \{2, 3\}, \{0, 5\}\}$.

Un couplage est représenté par une liste de paires de sommets :

```
type couplage = (int * int) list
```

Dans toute la suite, une arête $\{a, b\}$ sera représentée par la paire $(\min a \ b, \max a \ b)$.

Étant donné un couplage C dans un graphe G , on dit qu'un sommet v de G est **couvert** par C s'il existe une arête dans le couplage C qui est incidente à v .

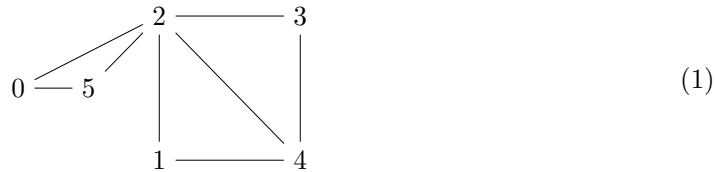
Question 4. Écrire une fonction `couverts: couplage -> int list` qui prend en entrée un couplage `cpl` et qui renvoie la liste des sommets couverts par `cpl`.

On attend une complexité en $O(m)$ où m est la longueur de la liste `cpl`, sans la justifier.

Couplages maximaux

Nous allons maintenant nous intéresser aux couplages maximaux, c'est-à-dire aux couplages ayant un nombre maximal d'arêtes. On remarque qu'un couplage dans un graphe à n sommets ne peut contenir plus de $\lfloor n/2 \rfloor$ arêtes.

Considérons le graphe suivant :



Le couplage $\{\{0, 5\}, \{2, 3\}, \{1, 4\}\}$ est maximal. Mais le couplage $\{\{0, 5\}, \{2, 4\}\}$ n'est pas maximal bien qu'il ne soit pas possible de lui adjoindre d'arête supplémentaire.

Question 5. Donner un autre couplage maximal dans le graphe ci-dessus.

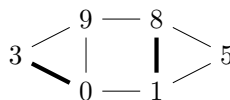
Chemins d'augmentation

L'algorithme d'Edmonds est basé sur la recherche de chemins d'augmentation. Étant donné un couplage C dans un graphe G , un **chemin d'augmentation** pour C (dans G) est un chemin élémentaire $v_0, \dots, v_n$ dans le graphe G tel que $n > 0$ et :

- v_0 et v_n ne sont pas couverts par C ;
- c'est un chemin **alternant** : pour tout $i = 0, \dots, n-2$, on a $\{v_i, v_{i+1}\} \in C$ si et seulement si $\{v_{i+1}, v_{i+2}\} \notin C$.

Un chemin d'augmentation P pour C permet de construire un nouveau couplage $C(P)$ constitué des arêtes de C qui ne sont pas dans P et des arêtes de P qui ne sont pas dans C .

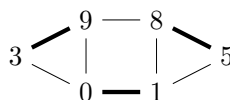
Par exemple, dans le graphe



le chemin suivant est un chemin d'augmentation :

$$9 \text{ --- } 3 \text{ --- } 0 \text{ --- } 1 \text{ --- } 8 \text{ --- } 5$$

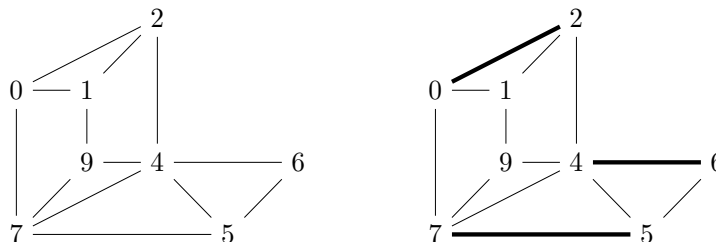
Le nouveau couplage obtenu à partir de ce chemin d'augmentation est le suivant :



Question 6. Considérons le graphe en (1) ci-dessus. Donner un chemin d'augmentation P

pour le couplage $C = \{\{0, 5\}, \{2, 4\}\}$ de manière à ce que le couplage $C(P)$ soit maximal.

Question 7. On considère le graphe et le couplage suivants :



Donner un chemin d'augmentation et le couplage augmenté correspondant.

Question 8. Soit P un chemin d'augmentation pour un couplage C dans un graphe G . Montrer que $C(P)$ est bien un couplage, et qu'il contient strictement plus d'arêtes que C .

En OCaml, un chemin est représenté par une liste de sommets, c'est-à-dire par une valeur de type `int list`.

Question 9. Écrire une fonction

```
separer : int list -> ((int * int) list) * ((int * int) list)
```

qui prend en entrée un chemin d'augmentation `chm` et renvoie deux listes `lstin` et `lstout` où `lstin` contient les arêtes composant le chemin `chm` qui appartiennent au couplage, et où `lstout` contient les arêtes composant le chemin `chm` qui n'appartiennent pas au couplage.

On attend une complexité en $O(m)$, où m est la longueur de la liste `chm`, sans la justifier.

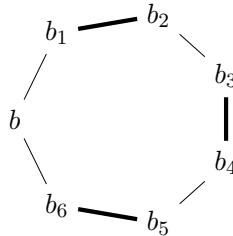
Question 10. Écrire une fonction `augmente: couplage -> int list -> couplage` qui prend en entrée un couplage `cpl` et un chemin d'augmentation `chm`, et qui renvoie le couplage `cpl(chm)`.

L'algorithme d'Edmonds, que nous allons voir ci-dessous, repose essentiellement sur le lemme de Berge. Ce résultat, que nous admettrons, énonce qu'un couplage C dans un graphe G est maximal si et seulement s'il n'existe pas de chemin d'augmentation pour C dans G .

Bourgeons

L'algorithme d'Edmonds va de plus effectuer des opérations de **contraction de bourgeons**. Soit C un couplage dans un graphe G . Un bourgeon est un cycle dans G de longueur impaire $2m + 1$ dont exactement m arêtes sont dans C . La **base** b d'un bourgeon B est le seul sommet

de B dont les deux arêtes adjacentes dans B ne sont pas dans C :



La contraction d'un bourgeon consiste à identifier tous les sommets de celui-ci. On représente un bourgeon B par une valeur de type `int list` qui contient exactement les sommets de B , dans un ordre quelconque, mais sans répétition. Par exemple, le bourgeon illustré ci-dessus peut être représenté par la liste $[b_1; b_6; b_2; b_5; b_3; b; b_4]$.

Étant donné un graphe $G = (V, E)$ et une liste de sommets $L = \ell_1, \dots, \ell_k$, le **graphe contracté** G/L est défini comme suit :

- L'ensemble des sommets du graphe G/L est $(V \setminus L) \cup \{w\}$ où w , le **nouveau sommet de G/L** , est un entier qui n'est pas un sommet de G .
- Les arêtes de G/L sont les arêtes de G entre sommets n'étant pas dans L , ainsi que les arêtes de la forme $\{u, w\}$ où u n'appartient pas à L mais est adjacent dans G à un sommet de L . L'ensemble des arêtes de G/L est donc défini comme $\{\{u, v\} \in E \mid u \notin L \text{ et } v \notin L\} \cup \{\{u, w\} \mid u \notin L \text{ et } u \text{ adjacent dans } G \text{ à un sommet de } L\}$

On suppose donnée une fonction `renomme : int list -> int list -> int -> int list` telle que `renomme lst rnm w` renvoie une liste obtenue en remplaçant dans `lst` tous les éléments de `rnm` par `w`, et en supprimant tous les doublons.

Question 11. Écrire une fonction `contracteG : graphe -> int list -> int -> graphe` qui prend en entrée un graphe `grph`, une liste de sommets `lst`, et le nom `w` du nouveau sommet, et qui renvoie le graphe contracté `grph/lst`.

Soit C un couplage dans un graphe G , et soit B un bourgeon. On définit un couplage C/B dans le graphe contracté G/B . En notant w le nouveau sommet de G/B , l'ensemble C/B a pour éléments :

- les arêtes $\{a, b\} \in C$ telles que $a \notin B$ et $b \notin B$;
- les arêtes $\{a, w\}$ telles que $a \notin B$ et telles qu'il existe un $b \in B$ avec $\{a, b\} \in C$.

On remarquera que C/B ne dépend que de C , B , et du nom w du nouveau sommet.

Question 12. Montrer que C/B est un couplage dans G/B .

Question 13. Écrire une fonction

`contracteC : couplage -> int list -> int -> couplage`

qui prend en entrée un couplage `cpl`, un bourgeon `brg`, et le nom `w` du nouveau sommet, et qui renvoie le couplage `cpl/brg`.

On admettra dans la suite qu'il existe un chemin d'augmentation pour le couplage C/B dans G/B si et seulement s'il existe un chemin d'augmentation pour le couplage C dans G .

Recherche de chemins d'augmentation

La recherche de chemins d'augmentation va utiliser des forêts, c'est-à-dire des listes d'arbres. Les types ci-dessous représentent des arbres et des forêts dont les nœuds sont étiquetés par des entiers. Ces entiers seront par la suite des sommets d'un graphe.

```
type arbre = N of int * foret
and foret = arbre list
```

Question 14. Écrire une fonction `find: foret -> int -> (int list)` option telle que `find foret v` renvoie `None` si l'entier `v` n'étiquette aucun des nœuds de `foret`, et telle que `find foret v` renvoie `Some c` sinon, où `c` est la liste des étiquettes le long d'un chemin allant d'une racine à un nœud d'étiquette `v` dans `foret`.

On attend une complexité en $O(n)$, où n est le nombre de nœuds de `foret`, sans la justifier.

Question 15. Écrire une fonction `extend: foret -> int -> int -> foret`, qui prend en arguments une forêt `foret` et deux entiers `u` et `v`, et qui renvoie une copie de `foret` dans laquelle pour chaque nœud étiqueté par `u`, on a créé un nouvel enfant étiqueté par `v`.

On attend une complexité en $O(n)$, où n est le nombre de nœuds de `foret`, sans la justifier.

La **profondeur** d'un nœud dans un arbre est définie inductivement comme suit : la racine est à profondeur 0, et pour $h \geq 0$, les nœuds à profondeur $h + 1$ sont les enfants des nœuds à profondeur h .

L'algorithme de recherche de chemins d'augmentation prend en entrée un graphe G et un couplage C dans G . Il manipule un ensemble de sommets T et peut être décrit comme suit.

- (1) On construit une forêt F dont les nœuds consistent exactement en une racine u pour chaque sommet u de G non couvert par C .
- (2) On fixe $T = \emptyset$.
- (3) Tant qu'il existe un sommet $u \notin T$ tel que u apparait à profondeur paire dans F :

On ajoute u à l'ensemble T .

Pour chaque voisin v de u dans G tel que $v \notin T$:

- (a) Si v n'est pas dans la forêt F , il est couvert par une arête $\{v, z\}$ de C , et on ajoute les arêtes $\{u, v\}$ et $\{v, z\}$ à F .
- (b) Si v est dans la forêt F :
 - (i) Si u, v apparaissent à profondeurs paires dans des arbres distincts de racines respectives r_u, r_v , alors on renvoie la suite des sommets de G vus dans F en allant de r_u à u (r_u et u inclus), puis en allant de v à r_v (v et r_v inclus).
 - (ii) Si u et v apparaissent dans le même arbre à profondeurs paires, alors on construit le bourgeon B correspondant, et on cherche un chemin d'augmentation pour le couplage C/B dans le graphe G/B . Si on en trouve un, alors on renvoie un chemin d'augmentation pour C dans G .

Question 16. Montrer que la suite de sommets renvoyée par l'algorithme dans le cas (3)(b)(i) est un chemin d'augmentation pour C dans G .

On admet dans toute la suite que l'algorithme ci-dessus renvoie un chemin d'augmentation pour C si et seulement s'il en existe un.

Nous allons étudier une implémentation OCaml de cet algorithme. On suppose données les fonctions suivantes.

- La fonction `frais: int list -> int` renvoie un entier n'appartenant pas à la liste donnée en argument.
- La fonction `prochain: foret -> int list -> int option` prend en arguments une forêt `f` et une liste de sommets `lst`. Un appel `prochain f lst` renvoie `Some d` où `d` est un entier apparaissant dans `f` à profondeur paire et n'appartenant pas à `lst` si un tel entier `d` existe. Sinon, `prochain f lst` renvoie `None`.
- La fonction `apparie: couplage -> int -> int` prend en arguments un couplage `cpl` et un sommet `u` couvert par le couplage, et renvoie le sommet apparié, c'est-à-dire l'unique `v` tel que $(\min u v, \max u v)$ appartient à `cpl`.
- La fonction `gonfle: graphe -> int list -> int list -> int -> int list` prend en arguments un graphe `grph`, un bourgeon `brg`, un chemin d'augmentation dans le graphe contracté `grph/brg`, et le nom du nouveau sommet `nv`, et renvoie le chemin d'augmentation correspondant dans `grph`.

La figure 1 page 11 représente une implémentation *incomplète* d'une fonction

`recherche : graphe -> couplage -> (int list) option`

Les questions ci-dessous demandent de donner le code de la ligne 15, des lignes 17–18 et des lignes 24–26, ainsi que d'implémenter la fonction `extraire` (ligne 20).

La fonction `recherche` prend en arguments un graphe et un couplage. Elle doit renvoyer `Some p` s'il existe un chemin d'augmentation `p`, et renvoyer `None` sinon. Certaines fonctions appelées dans le code de `recherche` peuvent lever des exceptions, mais on admettra que ces exceptions ne sont jamais levées si les données d'entrée sont bien formées.

Question 17. Donner le code devant apparaître à la ligne 15 de la figure 1.

Question 18. Donner le code devant apparaître dans le bloc aux lignes 17–18 de la figure 1.

Question 19. Écrire une fonction `extraire: int list -> int list -> int list` telle que l'appel `extraire cu cv` ligne 20 renvoie le bourgeon associé aux chemins `cu` et `cv`.

Question 20. Donner le code devant apparaître dans le bloc aux lignes 24–26 de la figure 1.

Question 21. Écrire une fonction `edmonds: graphe -> couplage` qui prend en entrée un graphe et qui renvoie un couplage maximal.

```

1  let rec recherche graphe couplage =
2      let nc = del (keys graphe) (couverts couplage) in
3      let foret = ref (List.map (fun x -> N(x,[])) nc) in
4      let rec aux_voisins u liste_voisins =
5          let cu = Option.get (find !foret u) in
6          match liste_voisins with
7          | [] -> None
8          | v :: tl ->
9              match find !foret v with
10             | None -> let z = apparie couplage v in
11                 foret := extend (extend !foret u v) v z;
12                 aux_voisins u tl
13             | Some cv when (List.length cv) mod 2 = 0
14                 (* nombre de sommets pair = profondeur impaire *)
15                 -> ...
16             | Some cv when (List.hd cu) <> (List.hd cv)
17                 -> ...
18                 ...
19             | Some cv
20                 -> let bourgeon = extrait cu cv in
21                     let nv = frais (keys graphe) in
22                     let ng = contracteG graphe bourgeon nv in
23                     let nc = contracteC couplage bourgeon nv in
24                     ...
25                     ...
26                     ...
27      in
28      let rec aux_sommets traitees =
29          match prochain !foret traitees with
30          | None -> None
31          | Some u -> let liste_voisins = del (List.assoc u graphe) traitees in
32              match aux_voisins u liste_voisins with
33              | None -> aux_sommets (u::traitees)
34              | Some ch -> Some ch
35      in
36      aux_sommets []

```

FIGURE 1 – Fonction recherche.

Partie II : Calculs de déterminants

Cette partie concerne l'algorithme de Mahajan-Vinay pour le calcul de déterminants de matrices carrées. Elle est à composer dans le langage C.

Préliminaires : matrices linéarisées

On travaille avec des matrices carrées de taille arbitraire indexées à partir de 0. On utilise une représentation linéarisée de ces matrices. Une matrice A de dimension $n \times n$ (c'est-à-dire avec n lignes et n colonnes) est représentée par un tableau A avec n^2 éléments de manière à ce que le coefficient $A_{i,j}$ de la matrice (indice de ligne i et indice de colonne j) corresponde à l'élément $A[i*n+j]$ du tableau.

On suppose dans la suite que les opérations arithmétiques ont une complexité en $O(1)$.

Question 22. Écrire des fonctions

```
int read_sqmatrix (int n, int *A, int i, int j)
void write_sqmatrix (int n, int *A, int i, int j, int val)
```

telles que

- `read_sqmatrix(n,A,i,j)` renvoie la valeur du coefficient $A_{i,j}$ où A est la matrice $n \times n$ représentée par A ;
- `write_sqmatrix(n,A,i,j,val)` modifie la valeur du coefficient $A_{i,j}$ pour qu'il soit égal à `val`.

On supposera que A est un tableau avec n^2 éléments et que $i, j \in \{0, \dots, n-1\}$.

On attend une complexité en $O(1)$ pour ces fonctions, sans la justifier.

Algorithme de Mahajan-Vinay

L'algorithme de Mahajan-Vinay est basé sur une formulation du déterminant d'une matrice A en termes de **suites de marches fermées** dans un graphe associé à A .

Soit A une matrice carrée de dimension $n \times n$. On associe à A son **graphe d'adjacence** $G(A)$. Le graphe $G(A)$ est **orienté et pondéré**. Il est défini comme suit :

- les sommets de $G(A)$ sont les entiers de 0 à $n-1$,
- les arcs sont les paires ordonnées $e = (i, j)$ telles que $A_{i,j} \neq 0$,
- le poids $\omega(e)$ d'un arc $e = (i, j)$ est égal à $A_{i,j}$.

Les sommets de $G(A)$ sont donc des entiers. L'ordre usuel sur ces entiers est utilisé dans la suite de cette partie.

Question 23. Dessiner le graphe d'adjacence de la matrice suivante :

$$\begin{pmatrix} 2 & 3 \\ -2 & 1 \end{pmatrix}$$

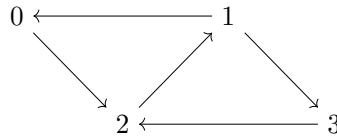
Une **marche fermée** dans le graphe $G(A)$ est une suite de sommets $C = v_0 \dots v_m$ avec $m > 0$ et qui satisfait les conditions suivantes :

- pour tout $i = 0, \dots, m-1$, il existe un arc (v_i, v_{i+1}) dans $G(A)$;
- le sommet $v_0 = v_m$ est le plus petit entier de la suite, appelé la **tête** de C , notée $t(C)$;
- le sommet v_0 n'apparaît qu'au début et à la fin de la suite : pour tout $i = 1, \dots, m-1$, on a $v_i \neq v_0$.

La **longueur** $|C|$ de la marche fermée C est le nombre d'arcs qui la composent. La longueur de $C = v_0 \dots v_m$ est donc égale à m . Le **poids** $\omega(C)$ d'une marche fermée C est le produit des poids des arcs (avec multiplicités) qui la composent :

$$\omega(C) = \prod_{i=0}^{m-1} \omega(v_i, v_{i+1}).$$

Exemple. Considérons le graphe suivant, dont les arcs ont tous poids 1 :



Dans ce graphe, les suites de sommets suivantes sont des marches fermées :

$$0 \rightarrow 2 \rightarrow 1 \rightarrow 0 \quad \text{et} \quad 0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0.$$

On remarque qu'il est possible de passer plusieurs fois par le même sommet. Cependant la suite

$$0 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow 2 \rightarrow 1 \rightarrow 0$$

n'est pas une marche fermée car la tête ne doit apparaître qu'au début et à la fin. La suite

$$2 \rightarrow 1 \rightarrow 0 \rightarrow 2$$

n'est pas non plus une marche fermée car la tête n'est pas le plus petit sommet.

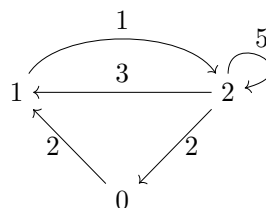
Question 24. Donner une marche fermée de longueur 3 et de tête 1 dans le graphe ci-dessus.

Une **suite de marches fermées** est une suite S de marches fermées $C_1, \dots, C_k$ telle que :

- les têtes sont croissantes : $t(C_1) < t(C_2) < \dots < t(C_k)$;
- le nombre total d'arcs (en comptant les multiplicités) est égal au nombre de sommets n de $G(A)$: $\sum_{i=1}^k |C_i| = n$.

Le **poids** d'une suite de marches fermées est le produit des poids des marches fermées qui la composent : $\omega(S) = \prod_{i=1}^k \omega(C_i)$. La **tête** de S , notée $t(S)$, est la tête de la première marche fermée de S : $t(S) = t(C_1)$.

Question 25. On considère le graphe suivant :



- (1) Donner les marches fermées de longueur au plus 3 et leurs poids.
- (2) Donner les suites de marches fermées et leurs poids.

On définit aussi le **signe** d'une suite de marches fermées. Si $S = C_1 \dots C_k$, alors on pose :

$$\text{sgn}(S) = (-1)^{n+k}.$$

On dit que S est une suite de marches fermées **positive** lorsque $\text{sgn}(S) = 1$, et que S est une suite de marches fermées **négative** lorsque $\text{sgn}(S) = -1$.

Le résultat fondamental de Mahajan et Vinay est que le déterminant d'une matrice carrée A peut être exprimé comme suit :

$$\det(A) = \sum_{\substack{S \text{ suite de marches fermées} \\ \text{dans } G(A)}} \text{sgn}(S) \omega(S).$$

Soit A une matrice avec n lignes et n colonnes. Pour implémenter le calcul du déterminant de A selon la formule ci-dessus, Mahajan et Vinay introduisent un nouveau graphe orienté et pondéré H_A , que nous allons maintenant décrire.

Le graphe H_A possède trois sommets distingués : s , t_0 , et t_1 . La construction de H_A permet d'assurer que les chemins de s à t_0 (respectivement t_1) dans H_A correspondent aux suites de marches fermées positives (respectivement négatives) dans $G(A)$. Les autres sommets sont des quadruplets $\langle p, h, u, i \rangle$ avec $p \in \{0, 1\}$, $0 \leq h \leq u \leq n-1$, et $i \in \{0, \dots, n-1\}$. Ceux-ci représentent des étapes de tentatives de construction de suites de marches fermées. Le sommet $\langle p, h, u, i \rangle$ représente le cas où des marches fermées $C_1, \dots, C_k$ ont été construites, et où on cherche à construire une marche fermée commençant par $C_{k+1} = v_0, v_1, \dots, v_m$, où

- le signe de la suite de marches fermées $C_1, \dots, C_k$ est $(-1)^p$;
- h est la tête de la marche fermée en cours de construction, c'est-à-dire $h = v_0$;
- u est le sommet courant, c'est-à-dire $u = v_m$;
- i est le nombre d'arcs parcourus jusque là, c'est-à-dire $i = m + \sum_{j=1}^k |C_j|$.

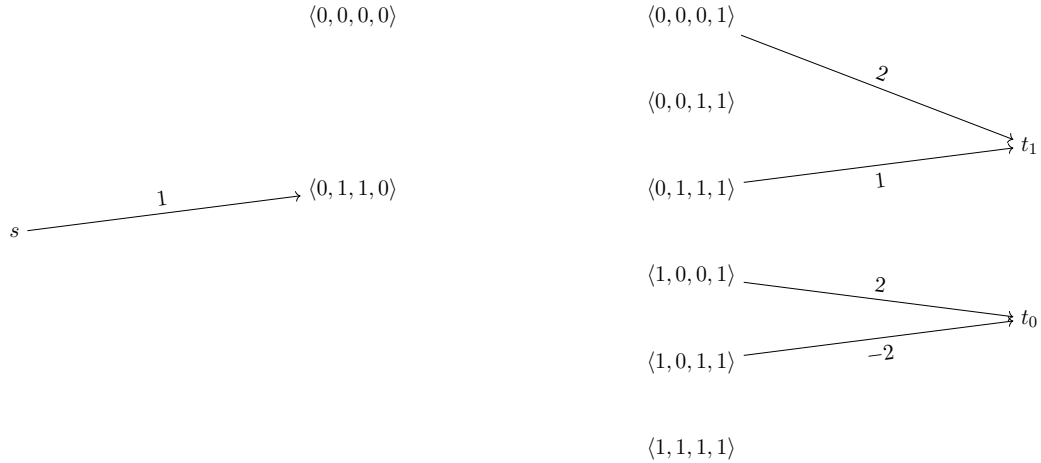
Le graphe H_A a les arcs suivants.

- (1) Un arc de poids 1 de s vers chaque sommet de la forme $\langle n \bmod 2, h, h, 0 \rangle$. Ces arcs permettent d'initialiser les suites de marches fermées.
- (2) Pour chaque coefficient $A_{u,v} \neq 0$, chaque $h < v$ et chaque $i < n-1$, un arc de poids $A_{u,v}$ de $\langle p, h, u, i \rangle$ vers $\langle p, h, v, i+1 \rangle$. Ces arcs correspondent à l'extension de la marche fermée en cours de construction (la marche C_{k+1} dans l'explication ci-dessus).
- (3) Pour chaque coefficient $A_{u,h} \neq 0$, chaque $h' \in \{h+1, \dots, n-1\}$, et chaque $i < n-1$, un arc de poids $A_{u,h}$ de $\langle p, h, u, i \rangle$ vers $\langle 1-p, h', h', i+1 \rangle$. Ces arcs correspondent à la fermeture de la marche fermée en cours de construction, et à l'initialisation d'une nouvelle marche fermée de tête h' .
- (4) Pour chaque coefficient $A_{u,h} \neq 0$ et chaque $p \in \{0, 1\}$, un arc de poids $A_{u,h}$ de $\langle p, h, u, n-1 \rangle$ vers t_{1-p} . Ces arcs correspondent à la fermeture de la dernière marche fermée de la suite.

Question 26. On considère la matrice A suivante :

$$\begin{pmatrix} 2 & 3 \\ -2 & 1 \end{pmatrix}$$

La figure ci-dessous représente certains sommets du graphe H_A correspondant, ainsi que certains arcs. Indiquer les arcs manquants entre les sommets représentés, ainsi que leurs poids.



Question 27. Soit A une matrice carrée avec n lignes et n colonnes. Montrer que si $C = h, u_1, \dots, u_m, h$ est une marche fermée dans $G(A)$, alors

$$\langle p, h, h, i \rangle \rightarrow \langle p, h, u_1, i+1 \rangle \rightarrow \dots \rightarrow \langle p, h, u_m, i+m \rangle$$

est un chemin dans H_A pour tout $p \in \{0, 1\}$ et tout $i \in \{0, \dots, n-1-m\}$.

En déduire qu'à chaque suite de marches fermées positive dans $G(A)$ correspond un chemin de s à t_0 dans H_A .

On peut démontrer que les chemins de s à t_0 dans H_A sont en fait en bijection avec les suites de marches fermées positives dans $G(A)$. Dans toute la suite, on admet ce résultat ainsi que le résultat analogue pour les chemins de s à t_1 dans H_A et les suites de marches fermées négatives dans $G(A)$.

Nous ne construirons pas le graphe H_A mais travaillerons de manière implicite avec celui-ci. Afin de calculer le déterminant de la matrice A en utilisant la formule de Mahajan-Vinay, nous allons calculer la somme des poids des chemins de s vers t_0 et t_1 dans H_A . Soit F_A la fonction qui à chaque sommet x de H_A associe la somme des poids des chemins de s à x dans H_A .

Nous allons commencer par calculer la valeur de $F_A(x)$ pour l'ensemble des sommets de la forme $x = \langle p, h, v, i \rangle$. On remarque que les sommets du graphe H_A peuvent être organisés en couches selon la valeur du dernier entier. Formellement, la **couche** i de H_A est l'ensemble des sommets de H_A de la forme $\langle p, h, v, i \rangle$.

Question 28. Exprimer la valeur de $F_A(\langle p, h, v, i \rangle)$ en fonction des valeurs de la couche $i-1$, c'est-à-dire des $F_A(\langle p', h', v', i-1 \rangle)$.

Implémentation

On veut représenter $F_A(x)$ pour chaque x de la forme $\langle p, h, v, i \rangle$. Pour cela, on suppose donnés les types et fonctions C suivants.

- Un type `Fourtable`.
- Une fonction `Fourtable *create (int n)`. Un appel `create(n)` alloue dans le tas une valeur de type `Fourtable` et renvoie un pointeur `t` vers celle-ci. La valeur `*t` ainsi créée a la taille nécessaire pour enregistrer les valeurs de F_A lorsque A est une matrice $n \times n$. Les valeurs contenues dans `*t` sont toutes initialisées à 0. On supposera que `free(t)` a une complexité en $O(1)$.
- Une fonction `void write(Fourtable *t, int val, int p, int h, int v, int i)` qui enregistre dans `*t` l'entier `val` comme valeur correspondant au sommet $\langle p, h, v, i \rangle$.
- Une fonction `int read(Fourtable *t, int p, int h, int v, int i)` qui renvoie la valeur correspondant au sommet $\langle p, h, v, i \rangle$ dans `*t`.

On suppose de plus que les fonctions `read` et `write` ont une complexité en $O(1)$, et que la fonction `create` a une complexité en $O(n^3)$, où n est la valeur donnée en argument.

Notons que $F_A(\langle p, h, u, 0 \rangle)$ ne dépend que de p, h, u et de la dimension de la matrice A .

Question 29. *Écrire une fonction `Fourtable *initialise (int n)` telle que `initialise(n)` renvoie un pointeur vers un `Fourtable` contenant les valeurs de F_A pour tous les sommets de la couche 0 de H_A , où A est une matrice $n \times n$.*

On attend une complexité en $O(n^3)$ pour cette fonction, sans la justifier.

Question 30. *Écrire une fonction `Fourtable *remplit (int n, int *A)` qui prend en entrée la représentation linéarisée A d'une matrice A de dimension $n \times n$, et renvoie un pointeur vers un `Fourtable` contenant les valeurs de F_A pour tous les sommets de la forme $\langle p, h, v, i \rangle$ avec $p \in \{0, 1\}$, $0 \leq h \leq v \leq n - 1$, et $0 \leq i \leq n - 1$.*

On attend une complexité $O(n^4)$ pour cette fonction, qu'il faudra justifier.

Question 31. *Écrire une fonction `int determinant (int n, int *A)` qui prend en entrée la représentation linéarisée A d'une matrice A de dimension $n \times n$ et renvoie son déterminant. On veillera à libérer correctement la mémoire allouée dans le tas.*

On attend une complexité en $O(n^4)$ pour cette fonction, qu'il faudra justifier.

Partie III : Méthode algébrique et probabiliste pour les couplages parfaits

Cette partie concerne un algorithme qui teste si un graphe possède un couplage parfait, c'est-à-dire un couplage qui couvre tous ses sommets. Elle est à composer dans le langage C. On pourra réutiliser le matériel de la partie précédente.

On ne considère que des graphes **non orientés** (voir page 3). Un tel graphe G d'ensemble de sommets $\{0, \dots, n-1\}$ est représenté par sa **matrice d'adjacence** A , de dimension $n \times n$, et dont les coefficients sont donnés par

$$A_{i,j} = \begin{cases} 1 & \text{si } \{i, j\} \text{ est une arête de } G, \\ 0 & \text{si } \{i, j\} \text{ n'est pas une arête de } G. \end{cases}$$

Nous allons utiliser des méthodes algébriques, basées sur le déterminant de la matrice de Tutte. La matrice de Tutte d'un graphe est une matrice symbolique, et son déterminant est un polynôme réel à plusieurs variables. Un polynôme réel à m variables $X_1, \dots, X_m$ est une somme finie de monômes, c'est-à-dire de termes de la forme $aX_1^{d_1} \dots X_m^{d_m}$ avec $a \in \mathbb{R}$ et $d_1, \dots, d_m \in \mathbb{N}$. Lorsque $d_i = 0$, la variable X_i est omise dans $aX_1^{d_1} \dots X_m^{d_m}$. Par exemple, le monôme $aX_1^0 \dots X_m^0$ est noté a .

Considérons un graphe non orienté G dont l'ensemble de sommets est $\{0, \dots, n-1\}$. La **matrice de Tutte** $T(G)$ de G est une matrice $n \times n$ dont les éléments sont des monômes sur l'ensemble de variables $\{X_{i,j} \mid 0 \leq i < j \leq n-1\}$. Les coefficients de $T(G)$ sont donnés par

$$T(G)_{i,j} = \begin{cases} 0 & \text{si } i = j \text{ ou si } \{i, j\} \text{ n'est pas une arête de } G, \\ X_{i,j} & \text{si } i < j \text{ et } \{i, j\} \text{ est une arête de } G, \\ -X_{j,i} & \text{si } i > j \text{ et } \{i, j\} \text{ est une arête de } G. \end{cases}$$

Un théorème de Tutte énonce que G possède un couplage parfait si, et seulement si, le déterminant de $T(G)$ est différent du polynôme nul (c'est-à-dire du polynôme dont toutes les valeurs sont 0).

Une partie du problème consiste donc à décider si un polynôme est nul ou non. Nous allons implémenter un algorithme **probabiliste** qui teste si un polynôme s'annule en un certain nombre de points choisis aléatoirement.

Dans la suite, on suppose donnée une fonction `int rand_range(int n)` qui prend en entrée un entier `n` et renvoie un entier tiré aléatoirement entre 0 et `n` (inclus) selon la distribution uniforme. On suppose que cette fonction s'exécute en temps constant.

On suppose que l'instruction suivante, qui crée un tableau d'entiers `t` de longueur `m`, s'exécute en temps $O(m)$:

```
int t [m];
```

Question 32. Écrire une fonction `int tirage_tutte(int n, int *A)` qui prend en entrée la représentation linéarisée `A` d'une matrice d'adjacence A de dimension $n \times n$, et renvoie le

déterminant d'une instantiation de la matrice de Tutte du graphe représenté par A obtenue en tirant aléatoirement les valeurs des variables $X_{i,j}$ dans l'ensemble $\{0, 1, 2, \dots, \mathbf{n}*\mathbf{n} - 1, \mathbf{n}*\mathbf{n}\}$. On attend une complexité en $O(\mathbf{n}^4)$ pour cette fonction, sans la justifier.

On va utiliser la fonction définie ci-dessus pour décider si un polynôme réel à plusieurs variables est nul ou non. On se propose d'appliquer le lemme de Schwartz-Zippel. Ce résultat permet, pour un polynôme non nul P , de borner le nombre de zéros de P (dans un ensemble fini donné) en fonction du degré total de P (et de la taille de cet ensemble). Le degré total d'un monôme $aX_1^{d_1} \dots X_n^{d_n}$ est la somme $d_1 + \dots + d_n$. Le **degré total** d'un polynôme est le degré total maximal des monômes qui le composent.

Soit P un polynôme réel non nul à m variables et de degré total $d \geq 0$, et soit S un sous-ensemble fini non vide de $\mathbb{R}$. On note $\Pr[P(r_1, r_2, \dots, r_m) = 0]$ la probabilité pour que P s'annule en $r_1, r_2, \dots, r_m$, où $r_1, r_2, \dots, r_m$ sont des réels tirés aléatoirement dans S , de manière uniforme et indépendante. Le lemme de Schwartz-Zippel énonce que

$$\Pr[P(r_1, r_2, \dots, r_m) = 0] \leq \frac{d}{|S|}.$$

On souhaite obtenir un algorithme probabiliste décidant si un graphe possède un couplage parfait. En s'aidant du lemme de Schwartz-Zippel, on pourra assurer la correction de cet algorithme et en borner l'erreur. On pourra utiliser la fonction `log2` définie lorsque l'entête `<math.h>` est incluse.

Question 33. *Écrire une fonction*

`bool parfait (int n, int *A, int p)`

*qui prend en entrée la matrice d'adjacence linéarisée A d'un graphe G ayant $\mathbf{n}$ sommets, et décide si G possède un couplage parfait avec une probabilité d'erreur inférieure à 2^{-p} , **qu'il faudra justifier**.*

*On attend une complexité en $O\left(p \frac{\mathbf{n}^4}{\log_2(\mathbf{n})}\right)$, **qu'il faudra justifier**.*

Fin du sujet.

A2025 – INFO I MPI

**ÉCOLE NATIONALE DES PONTS et CHAUSSÉES,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

**Concours Mines-Télécom,
Concours Centrale-Supélec (Cycle International).**

CONCOURS 2025**PREMIÈRE ÉPREUVE D'INFORMATIQUE**

Durée de l'épreuve : 3 heures

L'usage de la calculatrice ou de tout dispositif électronique est interdit.

*Les candidats sont priés de mentionner de façon apparente
sur la première page de la copie :*

INFORMATIQUE I - MPI

Cette épreuve concerne uniquement les candidats de la filière MPI.

L'énoncé de cette épreuve comporte 13 pages de texte.

Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence
Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.

Tout autre usage est soumis à une autorisation préalable du Concours commun Mines-Ponts.



Étude des tableaux associatifs

Préliminaires

L'épreuve est composée d'un problème unique, comportant 34 questions. Le problème consiste en l'étude et l'implémentation d'un tableau associatif en C, suivant différentes méthodes.

Le problème est divisé en 4 sections distinctes :

- l'implémentation du tableau associatif en utilisant une structure d'arbre (section 1),
- l'ajout d'un « ramasse miettes » pour gérer la mémoire (section 2),
- l'implémentation du tableau associatif en utilisant un algorithme probabiliste (section 3),
- la conception d'un analyseur syntaxique pour créer un tableau associatif à partir d'une chaîne de caractères (section 4).

Les quatre sections sont indépendantes.

Travail attendu

Les résultats attendus pour certaines questions pourront être admis afin de faciliter la résolution des questions suivantes.

Le langage C constitue l'unique langage autorisé pour le traitement des questions de programmation.

Une attention particulière sera portée quant à la correction et la lisibilité du code proposé. Si une tolérance raisonnable peut être envisagée sur certains aspects syntaxiques dans le cadre d'une épreuve réalisée sur support papier (par exemple, l'oubli d'un « ; »), un code difficilement lisible ou l'emploi de fonctions ou structures de contrôle non conformes aux standards du langage C, notamment celles empruntées à d'autres langages, sera sanctionné.

Un soin particulier sera également apporté à la gestion de la mémoire, à la bonne initialisation des structures ainsi qu'au contrôle systématique des allocations mémoires.

La gestion rigoureuse des erreurs sera valorisée (l'utilisation d'assert est recommandée), de même que la qualité, la simplicité et la lisibilité du code.

En conformité avec le programme officiel, il sera admis que certains en-têtes standards soient présumés comme inclus dans le code : `<assert.h>`, `<stdbool.h>`, `<stddef.h>`, `<stdint.h>`, `<stdio.h>`, `<stdlib.h>`, et `<string.h>`.

Les tableaux associatifs

Les tableaux associatifs sont des structures de données permettant un accès rapide à une *valeur* à partir d'une *clef*.

Afin de simplifier l'écriture du code, nous considérerons que les clefs sont toujours de type `char*` et les valeurs de type `int`.

1. Implémentation par un arbre binaire de recherche

Le but de cette section est d'étudier une implémentation d'un tableau associatif reposant sur un arbre binaire de recherche.

Nous adopterons une approche fonctionnelle pour l'implémentation des structures de données, en les considérant comme immuables. Ainsi, chaque fonction de manipulation renverra systématiquement une nouvelle version modifiée de la structure, sans altérer l'originale.

Cette méthode présente l'avantage d'éliminer les effets de bord lors des appels de fonction. Toutefois, elle implique une augmentation du nombre d'allocations mémoire, puisque chaque modification nécessite la création d'une copie de la structure de données.

Dans cette section nous ne nous occuperons pas de la récupération de la mémoire, car nous introduirons en section 2 un « ramasse miettes » pour gérer la mémoire automatiquement.

Arbre binaire de recherche

Définitions : Soit A un arbre binaire de recherche. On notera avec une lettre minuscule (par exemple x ou y) un nœud de A , et avec une lettre majuscule (par exemple T_1 , T_2 ou T_3) un sous-arbre de A . Soit x un nœud, on notera x_{gauche} et x_{droit} , respectivement le sous-arbre gauche et le sous-arbre droit de x .

On rappelle que la taille d'un arbre est égale au nombre de nœuds qu'il contient.

Pour un arbre de racine x , on notera $h(x)$ sa hauteur, et on définit récursivement la hauteur d'un arbre par :

- la hauteur d'un arbre vide est -1 ,
- $h(x) = \max(h(x_{\text{gauche}}), h(x_{\text{droit}})) + 1$.

Pour un arbre de racine x , on définit la valeur d'équilibre par :

$$e(x) = h(x_{\text{gauche}}) - h(x_{\text{droit}})$$

On qualifiera d'*équilibré* un arbre binaire de recherche A si pour tout nœud x de A ,

$$-1 \leq e(x) \leq 1$$

Définition : On définit un nœud d'un arbre binaire de recherche par la structure suivante :

```

1. struct abrNoeud_s {
2.     char* clef;
3.     int valeur;
4.
5.     struct abrNoeud_s* fils_gauche;
6.     struct abrNoeud_s* fils_droit;
7. };
8. typedef struct abrNoeud_s abrNoeud;
```

On utilisera la valeur du champ `clef` pour ordonner les nœuds de l'arbre, en utilisant l'ordre alphabétique.

□ 1 – Nous nous intéressons aux valeurs du champ `clef` de la structure ci-dessus. Expliquer comment une chaîne de caractères est représentée en mémoire en C ? Combien d'octets sont nécessaires pour représenter la chaîne "arbre" par exemple ?

Première épreuve d'informatique MPI 2025

□ 2 – Sachant que le code ASCII du caractère ‘a’ est 97, quelle est sa représentation en binaire? Quelle est sa représentation en hexadécimal?

□ 3 – Écrire une fonction C `copie_chaine` qui renvoie une copie de la chaîne de caractères passée en paramètre. La signature de la fonction est :

```
char* copie_chaine(char* chaine)
```

Indication C : On considèrera que la chaîne de caractères passée en paramètre (`chaine`) n’est jamais NULL.

□ 4 – Écrire une fonction C `abr_creer_noeud` qui crée un nœud à partir d’une clef, d’une valeur et de 2 sous-arbres donnés, et qui renvoie ce nouveau nœud créé. La signature de la fonction est :

```
abrNoeud* abr_creer_noeud(char* clef, int valeur,  
                           abrNoeud* fils_gauche,  
                           abrNoeud* fils_droit)
```

Indication C : le champ `clef` du nœud créé doit contenir une copie de la chaîne de caractères passée en paramètre. Par contre, il n’est pas nécessaire de dupliquer les sous-arbres passés en paramètre.

□ 5 – Écrire une fonction C `abr_equilibre` qui renvoie la valeur d’équilibre de l’arbre dont le nœud racine est passé en paramètre. La signature de la fonction est :

```
int abr_equilibre(abrNoeud* noeud)
```

□ 6 – Pour un arbre de taille N , quelle est la complexité de la fonction `abr_equilibre`? Sans proposer de code, expliquer quelles seraient les modifications à apporter à la structure de donnée et/ou à l’algorithme pour améliorer cette complexité?

□ 7 – Soit A un arbre binaire de recherche équilibré et de hauteur h , quelle approximation peut-on faire du nombre minimal N de nœuds que peut contenir cet arbre?

□ 8 – En déduire une borne supérieure de la hauteur h d’un arbre binaire de recherche équilibré de taille N .

□ 9 – Écrire une fonction C récursive `abr_rechercher` qui, étant donné un arbre dont le nœud racine est passé en paramètre ainsi qu’une clef, renvoie la valeur associée à cette dernière. La signature de la fonction est :

```
int abr_rechercher(abrNoeud* noeud, char* clef)
```

Indication : On considèrera que la clef donnée est toujours dans l’arbre binaire de recherche.

Indication C : l’opérateur `==` permet de comparer des entiers ou des pointeurs mais pas de comparer le contenu de deux chaînes de caractères.

Première épreuve d'informatique MPI 2025

- ☐ 10 – En application de la question 8, quelle est la complexité de la fonction `abr_rechercher` pour un arbre équilibré de taille N ?
- ☐ 11 – Quel est l'intérêt d'avoir un arbre binaire de recherche qui soit tout le temps équilibré ?

2. Ajout d'un ramasse miettes

Un « ramasse miettes » est un ensemble de fonctions qui permet de gérer la mémoire de manière automatique. L'idée est de libérer la mémoire qui avait été allouée dynamiquement pour des structures ou des tableaux lorsque ces derniers ne sont plus utilisés.

Dans un contexte fonctionnel où les données sont immuables, il est parfois compliqué de déterminer le moment opportun pour libérer la mémoire allouée aux nœuds. En effet, l'immuabilité des structures permet de partager le même sous-arbre à plusieurs endroits d'un arbre, sans indication explicite de ce partage, rendant ainsi difficile de savoir si un nœud doit être libéré ou non. L'utilisation d'un ramasse miettes dans un tel cas est donc tout indiquée. C'est ce que nous allons étudier dans cette section.

Pour simplifier le problème, nous considérons qu'une structure ou un tableau devient inutilisé dès lors qu'il n'est plus atteignable à partir des variables globales du programme. Autrement dit, en partant des variables globales, il est impossible d'y accéder en suivant des pointeurs, même de manière transitive.

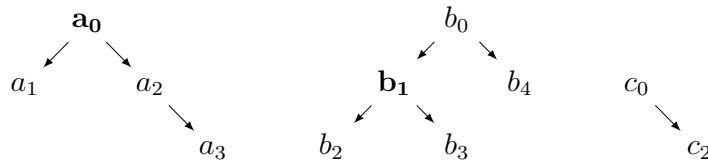
Pour pouvoir décrire l'algorithme de ramasse miettes qui nous intéresse, nous avons besoin de considérer qu'un nœud peut être « marqué ». Cela consiste à mémoriser une information dans la structure elle-même. Dans notre situation, nous utiliserons la structure suivante :

```
1. struct abrNoeud_s {  
2.     char* clef;  
3.     int valeur;  
4.  
5.     struct abrNoeud_s* fils_gauche;  
6.     struct abrNoeud_s* fils_droit;  
7.  
8.     // partie propre au ramasse-miettes  
9.     bool marque;  
10. };  
11. typedef struct abrNoeud_s abrNoeud;
```

Un nœud est alors dit « marqué » si et seulement si le champ `marque` a pour valeur `true`. Nous considérons qu'initialement aucun nœud n'est marqué. L'algorithme de ramasse miettes fonctionne en deux temps :

1. Marquer tous les nœuds atteignables par clôture transitive depuis les variables globales. C'est-à-dire tous les nœuds atteignables directement ou bien atteignables à partir d'un nœud lui-même atteignable.
2. Libérer tous les nœuds alloués mais non marqués et effacer la marque des nœuds marqués.

Première épreuve d'informatique MPI 2025

FIGURE 1 – Exemples d’arbres binaires de recherche, où les a_i , b_i , c_i sont des nœuds.

□ 12 – On considère que *seuls* a_0 et b_1 sont directement référencés par des variables globales (Figure 1). Quels sont les nœuds marqués par l’étape 1 de l’algorithme ? Quels nœuds seront libérés par le ramasse miettes ?

En déduire (sans démonstration) la complexité en temps de l’algorithme de marquage et de l’algorithme de libération.

Pour pouvoir libérer tous les nœuds non marqués il est nécessaire de maintenir une liste des nœuds de l’arbre alloués en mémoire. Nous proposons d’utiliser la structure de données suivante :

```

1. struct rmListeMemoire_s {
2.     abrNoeud* element;
3.     struct rmListeMemoire_s* suivant;
4. };
5. typedef struct rmListeMemoire_s rmListeMemoire;
  
```

□ 13 – Écrire une fonction C permettant d’insérer, en temps constant, un nœud d’arbre binaire de recherche dans une liste. La signature de la fonction est :

```

rmListeMemoire* rm_inserer_tete(abrNoeud* element,
                                rmListeMemoire* liste)
  
```

Indication : la donnée référencée par *liste* n’a pas besoin d’être modifiée.

Nous considérons maintenant la structure de donnée **RM** permettant de mémoriser dans le champ *arbres* les nœuds racines des différents arbres binaires de recherche accessibles (dans notre cas il peut y avoir jusqu’à 16 nœuds racines) ainsi que la liste des nœuds alloués :

```

1. #define MAX_NB_ARBRES 16
2. struct RM_s {
3.     abrNoeud* arbres[MAX_NB_ARBRES]; /* arbres accessibles */
4.     int nb_elements; /* nombre total de noeuds alloués */
5.     rmListeMemoire* premierElement;
6. };
7. typedef struct RM_s RM;
8.
9. RM* rm; /* variable globale pour l'ensemble du programme */
  
```

Le champ *premierElement* est une référence vers le premier élément d’une liste. Cette dernière est gérée de la façon suivante :

— lorsqu’un nœud d’un arbre binaire de recherche est alloué, il est ajouté à la liste,

Première épreuve d'informatique MPI 2025

— lorsqu'un nœud d'un arbre binaire de recherche est libéré, il est retiré de la liste.

□ 14 – La fonction `main` commence par l'instruction `rm = rm_creer_rm()` ; Écrire le code de la fonction `rm_creer_rm` permettant d'allouer et d'initialiser la structure de données qui est affectée à la variable `rm`. La signature de la fonction est :

RM* `rm_creer_rm(void)`

Le code suivant correspond aux deux étapes de l'algorithme de ramasse miettes :

```
1. void rm_ramasse_miettes(RM* rm) {  
2.     assert(rm != NULL);  
3.     int nb_noeuds = rm->nb_elements;  
4.     for (int i = 0; i < MAX_NB_ARBRES; i++) {  
5.         if (rm->arbres[i] != NULL) {  
6.             rm_marquer_elements_accessibles(rm->arbres[i]);  
7.         }  
8.     }  
9.     rm_recuperer_elements_inaccessibles(rm);  
10.    printf("noeuds collectes : %d", nb_noeuds - rm->nb_elements);  
11.    printf("noeuds encore accessibles : %d\n", rm->nb_elements);  
12. }
```

□ 15 – Écrire une fonction C correspondant à la première étape de l'algorithme : marquer un nœud donné ainsi que tous les nœuds atteignables à partir de ce dernier. La signature de la fonction est :

void `rm_marquer_elements_accessibles(abrNoeud* element)`

 Première épreuve d'informatique MPI 2025

Pour libérer tous les nœuds non marqués on considère la fonction suivante :

```

1.  void rm_recuperer_elements_inaccessibles(RM* rm) {
2.      assert(rm != NULL);
3.      rmListeMemoire* objet = rm->premierElement;
4.      rmListeMemoire* objets_conservees = NULL;
5.
6.      while (objet != NULL) {
7.          if (!objet->element->marque) {
8.              /* Ce nœud n'a pas été marqué,
9.                 on le retire de la liste et on le libère */
10.             rmListeMemoire* a_detruire = objet;
11.             objet = a_detruire->suivant;
12.             // PARTIE A
13.             // ici il faut libérer la mémoire
14.             // FIN PARTIE A
15.             rm->nb_elements--;
16.         } else {
17.             // PARTIE B
18.             /* Ce nœud a été marqué,
19.                on enlève la marque et on passe au nœud suivant */
20.             objet->element->marque = false;
21.             rmListeMemoire* a_garder = objet;
22.             objet = a_garder->suivant;
23.             // FIN PARTIE B
24.         }
25.     }
26.     rm->premierElement = objets_conservees;
27. }
```

L'algorithme est le suivant :

- on parcourt tous les éléments de la liste premierElement,
- si le nœud n'est pas marqué, il faut libérer la mémoire et enlever le nœud de la liste,
- si le nœud est marqué il faut enlever la marque et le laisser dans la liste.

☐ 16 – Compléter le code de la partie A de la fonction ci-dessus pour libérer la mémoire lorsque le nœud est marqué.

Indication : Seul le code entre *// PARTIE A* et *// FIN PARTIE A* est à compléter. Il n'est pas nécessaire de recopier le code autour.

☐ 17 – Quelle spécificité ont les nœuds de la liste référencée par le champ premierElement à la fin de l'exécution de la fonction ?

☐ 18 – Expliquer pourquoi le code proposé dans la partie B de la fonction ci-dessus n'est pas correct, et proposez une correction.

Indication : Seul le code entre *// PARTIE B* et *// FIN PARTIE B* est à compléter. Il n'est pas nécessaire de recopier le code autour.

3. Implémentation par l'utilisation d'un algorithme probabiliste

Nous allons nous intéresser dans cette section à une autre manière d'implémenter un tableau associatif, en utilisant un algorithme probabiliste : les « *skip lists* ».

3.1. Principe des skip lists

Une skip list est une liste chaînée optimisée pour accélérer la recherche d'un élément. Pour ce faire, certains nœuds stockent dans un tableau des pointeurs vers d'autres nœuds situés plus loin dans la liste. Chaque case du tableau correspond à un « niveau » et un pointeur de niveau i mène vers un nœud qui possède lui aussi un pointeur de niveau i .

Le niveau 0 contient l'ensemble des éléments, classés par ordre alphabétique en fonction de leur clef. Les niveaux supérieurs regroupent un sous-ensemble des éléments, de façon à espacer progressivement les nœuds. Par exemple, idéalement, le niveau 1 contiendrait un nœud sur deux, le niveau 2 un nœud sur quatre, et ainsi de suite (voir Figure 2).

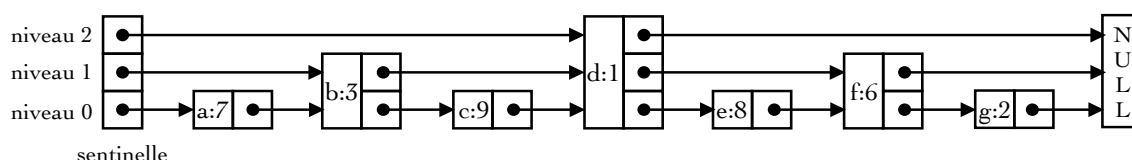


FIGURE 2 – Exemple d'une skip list idéale.

Le premier nœud de la skip list est une sentinelle : un élément spécial qui ne contient pas de donnée utile mais qui possède des pointeurs pour tous les niveaux de la liste.

Pour chercher un élément, on démarre à partir de la sentinelle au niveau le plus élevé. À ce niveau, on suit les pointeurs vers les nœuds suivants tant que la clef du nœud actuel est inférieure à celle recherchée. Dès qu'on rencontre un nœud dont la clef est supérieure, on descend d'un niveau et on répète la même opération. Lorsque la recherche atteint le niveau 0, elle est terminée et, si l'élément recherché y est présent, il est renvoyé (voir Figure 3).

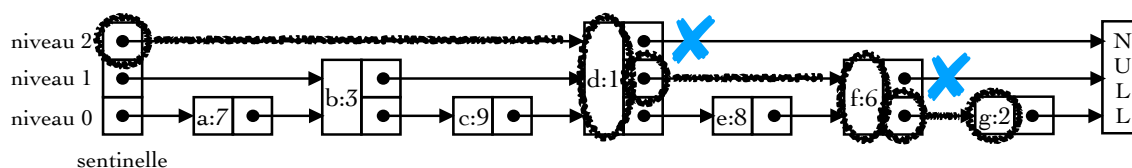


FIGURE 3 – Parcours lors de la recherche de la clef "g".

En pratique, chaque nœud a une probabilité de $\frac{1}{2}$ d'être présent dans le niveau supérieur.

3.2. Implémentation

Définition : On définit une skip list et un nœud d'une skip list par les structures suivantes :

```

1.  struct slNoeud_s {
2.      char* clef;
3.      int valeur;
4.
5.      struct slNoeud_s* suivant_par_niveau[];
6.  };
7.  typedef struct slNoeud_s slNoeud;
8.
9.  struct slListe_s {
10.     int niveau_actuel;
11.     slNoeud* sentinelle;
12.  };
13. typedef struct slListe_s slListe;

```

On définit une constante, `MAX_NIVEAU`, qui correspond au nombre maximal de niveaux qu'un nœud peut avoir. La sentinelle de la skip list est créée comme un nœud particulier, avec une clef vide, une valeur nulle, et un tableau de pointeurs de taille `MAX_NIVEAU+1` (de 0 à `MAX_NIVEAU` inclus) tous initialisés à `NULL`.

□ 19 – Écrire, en suivant l'algorithme décrit section 3.1, une fonction C `sl_rechercher` qui recherche, dans la skip list passée en paramètre, la clef donnée et qui renvoie :

- soit la valeur associée à la clef si elle est présente dans la skip list,
- soit `-1` si la clef n'est pas présente dans la skip list.

La signature de la fonction est :

```
int sl_rechercher(slListe* liste, char* clef)
```

□ 20 – On rappelle que la fonction `rand()` renvoie un entier aléatoire (de type `int`) compris entre 0 et un certain entier supérieur à 32767.

Quelles sont les propriétés de la valeur renvoyée par la fonction suivante ?

```

1.  int sl_mystere() {
2.      int niveau = 0;
3.      while ((rand() % 2) == 1 && niveau < MAX_NIVEAU) {
4.          niveau++;
5.      }
6.      return niveau;
7.  }

```

On suppose que la fonction suivante est disponible :

```
slNoeud* sl_creer_noeud(int niveau, char* clef, int valeur)
```

Cette fonction crée un nouveau nœud de niveau `niveau` avec la clef et la valeur passées en paramètre. Les pointeurs du nouveau nœud sont tous initialisés à `NULL`.

 Première épreuve d'informatique MPI 2025

La fonction `sl_ajoute_valeur`, qui ajoute une clef et une valeur dans la skip list passée en paramètre, a la structure suivante :

```

1.  void sl_ajoute_valeur(slListe* liste, char* clef, int valeur) {
2.      int niveau = sl_mystere();
3.      slNoeud* nouveau_noeud = sl_creer_noeud(niveau, clef, valeur);
4.
5.      slNoeud* noeuds_a_mettre_a_jour[MAX_NIVEAU + 1];
6.      slNoeud* courant = liste->sentinelle;
7.
8.      // PARTIE A
9.      /* parcourt l'ensemble des niveaux de la liste pour trouver
10.         dans chaque niveau le noeud après lequel insérer
11.         le nouveau noeud */
12.      // FIN PARTIE A
13.
14.      if (niveau > liste->niveau_actuel) {
15.          // PARTIE B
16.          /* si le niveau du nouveau noeud est plus grand
17.             que le niveau actuel de la liste
18.             préparer la sentinelle */
19.          // FIN PARTIE B
20.      }
21.
22.      // PARTIE C
23.      /* insérer le nouveau noeud */
24.      // FIN PARTIE C
25.  }
```

☐ 21 – En vous basant sur les instructions du commentaire, écrire le code correspondant à la partie A de la fonction `sl_ajoute_valeur`.

Indication C : Les nœuds devant être mis à jour sont stockés dans le tableau `noeuds_a_mettre_a_jour`.

Indication : Seul le code entre `// PARTIE A` et `// FIN PARTIE A` est à compléter. Il n'est pas nécessaire de recopier le code autour.

☐ 22 – En vous basant sur les instructions du commentaire, écrire le code correspondant à la partie B de la fonction `sl_ajoute_valeur`.

Indication C : On réutilisera la variable `noeuds_a_mettre_a_jour`.

Indication : Seul le code entre `// PARTIE B` et `// FIN PARTIE B` est à compléter. Il n'est pas nécessaire de recopier le code autour.

☐ 23 – En vous basant sur les instructions du commentaire, écrire le code correspondant à la partie C de la fonction `sl_ajoute_valeur`.

Indication : Seul le code entre `// PARTIE C` et `// FIN PARTIE C` est à compléter. Il n'est pas nécessaire de recopier le code autour.

Première épreuve d'informatique MPI 2025

□ 24 – L'algorithme des skip lists est un algorithme probabiliste. Est-ce un algorithme de type « Monte-Carlo » ou de type « Las Vegas » ? Justifier.

□ 25 – En supposant une skip list idéale comme dans la Figure 2, où chaque niveau possède exactement la moitié des éléments du niveau inférieur idéalement répartis, quelle est la complexité en temps de la recherche d'un élément dans une skip list de taille n ?

On considère désormais une skip list non idéale, où les éléments sont répartis de manière aléatoire avec une probabilité $p = \frac{1}{2}$ d'être présent dans le niveau supérieur.

□ 26 – Quelle est l'espérance du nombre d'éléments au niveau k ?

□ 27 – Quelle est l'espérance du niveau le plus haut d'une skip list de taille n ?

On acceptera dans la suite le résultat suivant : cette espérance est proportionnelle à $\log_2(n)$.

□ 28 – Quel est en moyenne le nombre de nœuds parcourus horizontalement dans un niveau k lors d'une recherche dans une skip list de taille n ?

□ 29 – Des questions 27 et 28, déduisez la complexité en moyenne en temps de la recherche d'un élément dans une skip list de taille n .

4. Analyse syntaxique

On souhaite construire une fonction analyse qui prend en paramètre une chaîne de caractères décrivant un tableau associatif sous la forme de couples `clef:valeur`.

Un exemple d'utilisation pourrait être : `analyse("{id:5,valeur:12,ok:0}")`

Le format de la chaîne de caractères passée en paramètre est spécifié par les règles suivantes :

- la chaîne doit commencer par une accolade ouvrante, être suivie d'une liste de couples `clef:valeur`, et terminée par une accolade fermante,
- chaque couple est composé d'une clef et d'une valeur séparées par le caractère `:`,
- ni la clef ni la valeur ne peuvent être absentes,
- la clef est une suite finie non vide de lettres minuscules (de `'a'` à `'z'`),
- la valeur est un entier non signé, sans 0 non significatif,
- les couples sont séparés par le caractère `,`.

Le but de cette partie est d'étudier la grammaire et l'automate de ce langage de description.

 Première épreuve d'informatique MPI 2025

Définition : On utilise la notation $x \mid \dots \mid y$ pour indiquer la disjonction de symboles allant de x à y avec x et y des chiffres ou x et y des lettres.

$$N \rightarrow 0 \mid \dots \mid 9$$

On considère connues les règles de production suivantes :

$$M \rightarrow 1 \mid \dots \mid 9$$

$$L \rightarrow a \mid \dots \mid z$$

☐ 30 – Écrire la règle de production C correspondant à la description d'une clef.

☐ 31 – Écrire la règle de production V correspondant à la description d'une valeur.

On considère la grammaire suivante :

$$T \rightarrow \{S\}$$

$$S \rightarrow K \mid K, S$$

$$K \rightarrow C : V$$

☐ 32 – Le mot $\{\text{id:5,valeur:12,ok:0}\}$ appartient-il au langage engendré par cette grammaire? Le cas échéant, dessinez un arbre de dérivation.

Même question pour le mot $\{\text{james:007}\}$.

☐ 33 – Décrire un automate (avec un schéma ou une table de transitions) déterministe et sans transition vide, correspondant au langage reconnu par la règle T . On s'autorisera à étiqueter les transitions avec des disjonctions de lettres ou de chiffres.

☐ 34 – Vérifiez en précisant les étapes de l'automate que les séquences $\{\text{code:102}\}$, $\{\text{v:0}\}$, et $\{\text{a:1,b:2}\}$, sont bien reconnues par l'automate proposé. Dans quel état est l'automate lorsqu'on lui demande de reconnaître la séquence $\{\text{james:007}\}$?

A2025 – INFO II MPI

**ÉCOLE NATIONALE DES PONTS et CHAUSSÉES,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

**Concours Mines-Télécom,
Concours Centrale-Supélec (Cycle International).**

CONCOURS 2025**DEUXIÈME ÉPREUVE D'INFORMATIQUE**

Durée de l'épreuve : 4 heures

L'usage de la calculatrice ou de tout dispositif électronique est interdit.

*Les candidats sont priés de mentionner de façon apparente
sur la première page de la copie :*

INFORMATIQUE II - MPI

Cette épreuve concerne uniquement les candidats de la filière MPI.

L'énoncé de cette épreuve comporte 9 pages de texte.

Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence
Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.

Tout autre usage est soumis à une autorisation préalable du Concours commun Mines-Ponts.



Épreuve d'Informatique 2 MPI 2025

Le jeu de Shannon

Préliminaires

L'épreuve est composée d'un problème unique, comportant 33 questions.

Dans ce problème, nous nous intéressons au *jeu de Shannon*, qui est un jeu de stratégie dans lequel l'un des joueurs doit parvenir à établir une certaine connexion en posant des pièces de jeu, tandis que son opposant doit l'en empêcher en essayant de le bloquer. Le problème est divisé en trois sections. Dans la première section (page 1), nous introduisons les règles du jeu et une étude de tournois. Dans la deuxième section (page 5), nous analysons les stratégies de jeu conduisant l'un des joueurs à la victoire. Dans la troisième section (page 7), nous étudions la construction de deux arbres couvrants disjoints dans un graphe, ce qui est un prérequis de l'une des stratégies d'un joueur, vue dans la deuxième section.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractères différentes désigne la même entité, mais du point de vue mathématique pour la police en italique (par exemple n) et du point de vue informatique pour celle en romain avec espacement fixe (par exemple `n`).

Travail attendu

Pour répondre à une question, il est permis de réutiliser le résultat d'une question précédente, même sans avoir réussi à établir ce résultat.

Selon les consignes, il faudra coder des fonctions à l'aide du langage de programmation C exclusivement, en reprenant le prototype de fonction fourni par le sujet, ou en pseudo-code (c.-à-d. dans une syntaxe souple mais conforme aux possibilités offertes par le langage C). Inclure les entêtes tels que `<assert.h>`, `<stdbool.h>`, etc., n'est pas demandé.

Quand l'énoncé demande de coder une fonction, sauf indication explicite, il n'est pas nécessaire de justifier que celle-ci est correcte ou de tester que des préconditions sont satisfaites.

Le barème tient compte de la clarté et de la concision des programmes : nous recommandons de choisir des noms de variables intelligibles ou encore de structurer de longs codes par des blocs ou par des fonctions auxiliaires dont on décrira le rôle.

1 Le jeu de Shannon

1.1 Implémentation du jeu en langage C

Un graphe non orienté et connexe $G = (V, E)$ est fixé, ainsi que deux sommets particuliers : le sommet s et le sommet t . La lettre n désigne le nombre de sommets.

Définition : Le *jeu de Shannon* sur le graphe G est un jeu qui se joue entre deux joueurs, *Positus* et *Minus*, qui agissent à tour de rôle. Chaque coup consiste à marquer une arête qui n'a pas été marquée précédemment. Dans ce jeu, *Positus* remporte la victoire s'il parvient à marquer un ensemble d'arêtes dans lequel les sommets s et t sont reliés par un chemin (appelé aussi une *chaîne* d'arêtes). *Minus* gagne le jeu si toutes les arêtes ont été marquées sans que *Positus* ne soit parvenu à relier les sommets s et t .

Épreuve d'Informatique 2 MPI 2025

En pratique, *Positus* « verrouille » des arêtes tandis que *Minus* « supprime » des arêtes encore jouables.

Indication C : Dans l'ensemble du sujet, les sommets du graphe $G = (V, E)$ sont numérotés entre 0 et $n - 1$, $S = \{0, 1, 2, \dots, n - 1\}$. Par convention, les sommets s et t correspondent aux sommets de numéros 0 et 1. Les arêtes des graphes sont de la forme $\{u, v\}$ et donc E est un sous-ensemble de $P_2(S) = \{X \subset S \mid |X| = 2\}$. Nous notons $M \subseteq E$ l'ensemble des arêtes marquées. Nous introduisons les déclarations suivantes.

```

1.  #define s 0
2.  #define t 1
3.
4.  struct link {
5.      int node;
6.      struct link * next;
7.  };
8.  typedef struct link link_t;
9.
10. struct game {
11.     int n;           // Nombre de sommets
12.     int nm;          // Nombre d'arêtes non marquées
13.     link_t ** adj_arr; // Voisins non marques
14.     int * cmps;       // Composantes connexes creees par Positus
15. };
16. typedef struct game game_t;
17.
18. game_t gm;

```

Le type `link_t` permet de représenter des maillons de listes simplement chaînées. Le graphe $(V, E \setminus M)$ est représenté par un tableau de listes d'adjacences. Le type `game_t` permet de représenter un jeu de Shannon en cours de déroulement. Dans les enregistrements de ce type,

- le champ `n` désigne le nombre de sommets,
- le champ `nm` désigne le nombre d'arêtes non marquées,
- le champ `adj_arr` renvoie à un tableau de pointeurs où, pour tout entier u compris entre 0 et $n - 1$, le pointeur `adj_array[u]` désigne le premier maillon d'une liste simplement chaînée contenant tous les voisins du sommet u dans le graphe $(V, E \setminus M)$,
- le champ `cmps` renvoie à un tableau permettant de représenter les sommets reliés entre eux par des arêtes marquées par *Positus* selon une convention détaillée aux questions 6 et 7.

Une variable globale `gm` est utilisée dans tout le sujet pour désigner le jeu en cours. On notera que les arêtes marquées au cours du jeu sont perdues et ne figurent plus dans la variable `gm`.

□ 1 – Écrire une fonction C `void gm_init(int n)` qui initialise la variable globale `gm` afin qu'elle contienne un jeu sur le graphe à n sommets et zéro arêtes. Il est attendu que le tableau `gm.cmps` contienne initialement les entiers $0, 1, \dots, n - 1$, dans cet ordre.

Épreuve d'Informatique 2 MPI 2025

□ 2 – Écrire une fonction C `void gm_addedge(int u, int v)` dont l'effet est d'ajouter une arête entre les sommets u et v au graphe stocké dans la variable globale `gm`. On veillera à représenter une arête par deux arcs orientés (u, v) et (v, u) .

□ 3 – Écrire une fonction C `bool gm_remove(int u, int v)` dont l'effet est de retirer une arête entre les sommets u et v au graphe `gm` et dont la valeur de retour est `true` si la suppression s'est bien déroulée, et `false` sinon.

Nous rappelons que la structure « unir & trouver » (ou, en anglais, *union find*) permet de représenter les classes d'équivalence d'une relation d'équivalence pour un ensemble E . On choisit d'implémenter cette structure à l'aide d'un tableau associatif A , de taille n pour un ensemble E de cardinal n dont les éléments sont numérotés de 0 à $n - 1$. Le tableau rend compte d'une forêt où chaque arbre est orienté des feuilles vers la racine et représente une classe d'équivalence. Pour un élément de numéro u , $A[u]$ vaut u s'il s'agit de la racine, que l'on appellera **identifiant** de la classe, ou $A[u]$ vaut le numéro de son parent dans l'arbre dans tout autre cas.

□ 4 – Écrire une fonction C `int uf_find(int n, int * A, int u)` dont la valeur de retour est l'identifiant de la classe à laquelle appartient l'élément u dans la relation représentée par A . Le tableau A ne doit pas être modifié.

□ 5 – Écrire une fonction C `bool uf_unite(int n, int * A, int u, int v)` dont l'effet est de réunir les classes des éléments u et v dans la relation d'équivalence représentée par A . Il n'y a pas d'effet si les éléments u et v appartiennent déjà à la même classe. La valeur de retour est `true` si A a été modifié et `false` sinon.

Nous appelons $M^+ \subseteq E$ l'ensemble des arêtes marquées par le joueur *Positus*. Nous notons $\leftrightarrow_P$ la relation d'équivalence dans laquelle deux sommets u et v sont en relation si et seulement s'il existe une chaîne entre u et v formée d'arêtes de M^+ . Les classes d'équivalence sont ainsi les composantes connexes du graphe (V, M^+) . Nous utilisons le champ `cmps` de la variable `gm` afin de stocker cette relation d'équivalence selon une structure « unir & trouver ».

□ 6 – Écrire une fonction C `int gm_find(int u)` dont la valeur de retour est l'identifiant de la composante connexe à laquelle appartient le sommet u dans le graphe (V, M^+) .

□ 7 – Écrire une fonction C `void gm_unite(int u, int v)` dont l'effet est de réunir les composantes connexes des sommets u et v .

□ 8 – Écrire une fonction C `bool has_P_won(void)` dont la valeur de retour indique si le jeu est dans un état gagnant pour le joueur *Positus*.

□ 9 – Écrire une fonction C `bool has_P_lost(void)` dont la valeur de retour indique si le jeu est dans un état perdant pour le joueur *Positus*.

Épreuve d'Informatique 2 MPI 2025

□ 10 – Écrire une fonction C `void play_P(int u, int v)` dont l'effet est de transformer la variable `gm` en jouant l'arête entre les sommets u et v par le joueur *Positus*. On se défendra, à l'aide d'une assertion, du cas où le coup joué par *Positus* est illicite.

□ 11 – Écrire une fonction C `void play_M(int u, int v)` dont l'effet est de transformer la variable `gm` en jouant l'arête entre les sommets u et v par le joueur *Minus*. On se défendra, à l'aide d'une assertion, du cas où le coup joué par *Minus* est illicite.

1.2 Étude de tournois en langage SQL

Un *tournoi* de jeux de Shannon est une compétition dans laquelle s'affrontent exactement deux joueurs (A et B) au cours de plusieurs matches. Le vainqueur d'un tournoi est le joueur qui a gagné le plus grand nombre de matches. Le rôle de chaque joueur (*Positus* ou *Minus*) peut changer entre chaque match.

On dispose d'une base de donnée SQL composée de trois tables (les clés primaires sont soulignées)

- Joueurs(JoueurID, NomJ)
- Tournois(TournoiID, NomT, JoueurAID, JoueurBID)
- Matches(MatchID, TournoiID, RoleJoueurA, VainqueurID)

dont voici certains extraits :

JoueurID	NomJ	TournoiID	NomT	JoueurAID	JoueurBID
2	Alice	82	TournoiDu12Dec	2	8
8	Bob	59	TournoiDu17Fev	7	8
7	Charlie	61	TournoiDu29Mai	8	2

MatchID	TournoiID	RoleJoueurA	VainqueurID
119	82	Positus	2
934	82	Minus	8
925	82	Minus	2
384	61	Positus	2

Par exemple, le tournoi du 12 décembre a opposé Alice et Bob, il s'est joué en 3 matches. Alice, qui a gagné deux des trois matches, a été victorieuse.

□ 12 – Écrire une requête SQL renvoyant le nombre de matches joués dans chaque tournoi.

□ 13 – Écrire une requête SQL renvoyant le nombre de tournois gagnés par chaque joueur.

□ 14 – Écrire une requête SQL renvoyant le nombre de fois où chaque joueur a joué le rôle de *Positus*.

Épreuve d'Informatique 2 MPI 2025

2 Stratégies de jeu

2.1 Stratégies gagnantes

☐ 15 – Démontrer, à l'aide d'un variant, que le jeu se termine toujours et justifier qu'il ne peut pas se terminer par un match nul.

☐ 16 – Rappeler la définition du terme *stratégie gagnante*.

☐ 17 – Nous supposons que *Positus* dispose d'une stratégie gagnante s'il joue en second. Démontrer qu'il dispose aussi d'une stratégie gagnante s'il joue en premier.

☐ 18 – Démontrer qu'il existe trois types d'instances du jeu de Shannon :

- un type d'instance dans laquelle *Positus* dispose d'une stratégie gagnante qu'il soit premier ou second joueur,
- un type dans laquelle *Minus* dispose d'une stratégie gagnante qu'il soit premier ou second joueur,
- et enfin un type dans laquelle le joueur qui commence la partie dispose d'une stratégie gagnante.

On donnera un exemple de graphe G illustrant chacun des trois types.

2.2 Une stratégie pour *Positus*

Définition : Selon la terminologie usuelle, nous appelons *arbre couvrant* tout sous-ensemble de $n - 1$ arêtes $T \subseteq E$ ne contenant pas de cycle. Il est rappelé les faits suivants :

- le graphe (V, T) est connexe ;
- tout ajout d'une arête $e \in E \setminus T$, à un arbre couvrant T , crée un unique cycle noté $C(T, e)$;
- si T est un arbre couvrant, alors, pour tout couple de sommets u et v de V , il existe un unique chemin simple reliant u et v avec des arêtes de T .

☐ 19 – Soient deux arbres couvrants $T \subseteq E$ et $T' \subseteq E$ et une arête α appartenant à T . Montrer qu'il existe une arête β appartenant à T' telle que l'ensemble $T'' = T \cup \{\beta\} \setminus \{\alpha\}$ est encore un arbre couvrant.

Indication C : Nous représentons tout arbre couvrant par un tableau T de taille n tel que $T[0]$ vaut 0 et, pour tout autre sommet u , $T[u]$ vaut le parent du sommet u dans l'arbre enraciné en 0.

Pour représenter un arc, nous déclarons :

Épreuve d'Informatique 2 MPI 2025

```

19. struct pair {
20.     int first;
21.     int second;
22. };
23. typedef struct pair pair_t;

```

□ 20 – Écrire une fonction C `pair_t edge_find(int n, int * T, int * Tprime, pair_t alpha` ainsi spécifiée :

Précondition : Les variables T et $Tprime$ contiennent des arbres couvrants T et T' sur les mêmes n sommets. La variable α est un arc de l'arbre T orienté des feuilles vers la racine.

Valeur de retour : Une arête β , appartenant à l'arbre T' , telle que $T'' = T \cup \{\beta\} \setminus \{\alpha\}$ est encore un arbre couvrant.

□ 21 – Écrire une fonction C `void edge_swap(int n, int * T, pair_t alpha, pair_t beta)` ainsi spécifiée :

Précondition : La variable T correspond à un arbre couvrant T à n sommets. La variable α est un arc de l'arbre T orienté des feuilles vers la racine. La variable β est une arête qui n'appartient pas à l'arbre T telle que $T'' = T \cup \{\beta\} \setminus \{\alpha\}$ est un arbre couvrant.

Effet : Le tableau T est transformé afin de contenir l'arbre T'' .

Dans ce qui suit, nous supposons que *Minus* joue en premier.

Définition : Pour tout ensemble de sommets $U \subseteq V$ qui contient les sommets s et t , nous notons G_U le *graphe induit par* U , c'est-à-dire le sous-graphe (U, E_U) où,

$$E_U = \{\{u, v\} \in E \mid u, v \in U\}.$$

Pour tout entier naturel k , nous notons $M_k^+ \subseteq E$ l'ensemble des k arêtes marquées par *Positus*, après que k coups ont été joués par *Positus*. De même, nous notons $M_k^- \subseteq E$ l'ensemble des k arêtes marquées par *Minus* après que k coups ont été joués par *Minus*. Enfin, nous appelons $(\boxtimes_k)$ l'assertion suivante :

Après que $2k$ coups ont été joués par *Minus* et *Positus*, il existe un ensemble de sommets U contenant les sommets s et t ainsi que deux arbres couvrant le graphe induit G_U , que l'on note $T_k \subseteq E_U$ et $T'_k \subseteq E_U$, tels que l'on a $T_k \cap T'_k = M_k^+$ et $(T_k \cup T'_k) \cap M_k^- = \emptyset$. ($\boxtimes_k$)

□ 22 – Soit k un entier naturel. Dans cette question, nous supposons que $2k$ coups ont d'ores et déjà été joués et que l'assertion $(\boxtimes_k)$ est vérifiée ; pour son $(k+1)^e$ coup, le joueur *Minus* choisit de marquer une arête appartenant à l'arbre T_k . Montrer que le joueur *Positus* peut alors trouver une arête à marquer telle que l'assertion $(\boxtimes_{k+1})$ est vérifiée.

□ 23 – Soit k un entier naturel. Dans cette question, nous supposons que $2k$ coups ont été joués et que l'assertion $(\boxtimes_k)$ est vérifiée ; le joueur *Minus* marque une arête n'appartenant ni à l'arbre T_k , ni à l'arbre T'_k . Montrer que le joueur *Positus* peut marquer une arête telle que l'assertion $(\boxtimes_{k+1})$ est vérifiée.

Épreuve d'Informatique 2 MPI 2025

□ 24 – Démontrer que, si au cours de la partie, il existe un entier naturel k_0 tel que l'assertion $(\mathfrak{X}_{k_0})$ est satisfaite, alors *Positus* possède une stratégie gagnante.

□ 25 – Dans cette question, nous supposons que la paire $\{s, t\}$ n'appartient pas à E et nous notons $\hat{G}$ le graphe obtenu à partir de G en ajoutant une arête supplémentaire entre les sommets s et t . Nous supposons que l'assertion $(\mathfrak{X}_0)$ est vraie pour le nouveau graphe $\hat{G}$. Démontrer que le joueur *Positus* dispose d'une stratégie gagnante lorsqu'il joue en premier.

3 Arbres couvrants disjoints

Dans toute cette section, nous supposons que la paire $\{s, t\}$ n'appartient pas à l'ensemble des arêtes E . Nous notons $\hat{G}$ le graphe obtenu à partir de G en ajoutant une arête supplémentaire entre les sommets s et t . Nous appelons $\hat{E}$ l'ensemble d'arêtes $E \cup \{s, t\}$.

3.1 Une condition suffisante pour $(\mathfrak{X}_0)$

Soient $T \subseteq \hat{E}$ et $T' \subseteq \hat{E}$ deux arbres couvrants de $\hat{G}$ et α_0 une arête n'appartenant pas à $T \cup T'$. Notons $L_0 = \{\alpha_0\}$. Notons $L_1 \subseteq \hat{E}$ l'unique cycle $C(T, \alpha_0)$ présent dans l'ensemble $T \cup \{\alpha_0\}$.

Définition : Pour tout couple d'arbres couvrants, $T \subseteq \hat{E}$ et $T' \subseteq \hat{E}$, notons $\delta(T, T')$ le cardinal de l'intersection $T \cap T'$. Deux arbres couvrants T_0 et T'_0 sont *éloignés* si le couple (T_0, T'_0) atteint le minimum global de la fonction δ parmi tous les couples d'arbres couvrants de $\hat{G}$.

Définition : Une arête $\alpha \in \hat{E}$ est *principale* s'il existe un couple d'arbres couvrants T et T' éloignés, tel que l'arête α n'appartient pas à $T \cup T'$.

□ 26 – En supposant qu'il existe une arête $\alpha_1 \in L_1$ appartenant à $T \cap T'$, construire deux arbres couvrants $\tilde{T}$ et $\tilde{T}'$ du graphe $\hat{G}$ tels que

$$\delta(\tilde{T}, \tilde{T}') < \delta(T, T').$$

Si L_1 ne contient aucune arête de $T \cap T'$, nous appelons $L_2 \subseteq \hat{E}$ la réunion des cycles $\bigcup_{\alpha \in L_1} C(T', \alpha)$ qui se créent lorsque l'on ajoute à T' l'une des arêtes de L_1 .

□ 27 – En supposant qu'il existe une arête $\alpha_2 \in L_2$ appartenant à $T \cap T'$, construire deux arbres couvrants $\tilde{T}$ et $\tilde{T}'$ du graphe $\hat{G}$ tels que

$$\delta(\tilde{T}, \tilde{T}') < \delta(T, T').$$

Plus généralement, nous construisons par récurrence la suite croissante $(L_k)_{k \in \mathbb{N}} \subseteq \hat{E}$ d'ensembles d'arêtes où

Épreuve d'Informatique 2 MPI 2025

- pour tout entier $k \geq 1$, si l'ensemble L_{2k} ne contient aucune arête de $T \cap T'$, alors on pose

$$L_{2k+1} = \bigcup_{\alpha \in L_{2k}} C(T, \alpha),$$

- pour tout entier $k \geq 0$, si l'ensemble L_{2k+1} ne contient aucune arête de $T \cap T'$, alors on pose

$$L_{2k+2} = \bigcup_{\alpha \in L_{2k+1}} C(T', \alpha).$$

□ 28 – Démontrer que l'un ou l'autre des cas suivants se produit :

- la suite $(L_k)_{k \in \mathbb{N}} \subseteq E^{\mathbb{N}}$ n'est définie que pour un nombre fini de termes et dans ce cas montrer qu'il existe deux arbres couvrants $\tilde{T}$ et $\tilde{T}'$ du graphe $\hat{G}$ tels que

$$\delta(\tilde{T}, \tilde{T}') < \delta(T, T') ;$$

- ou bien la suite $(L_k)_{k \in \mathbb{N}} \subseteq E$ est définie pour tout rang k et donc qu'il existe un rang à partir duquel la suite est égale à une constante $\Lambda(\alpha_0) \subseteq \hat{E}$.

□ 29 – Démontrer que, si l'arête $\{s, t\}$ est une arête principale du graphe $\hat{G}$, alors l'assertion $(\mathfrak{X}_0)$ est vérifiée.

3.2 Recherche des arêtes principales

□ 30 – Nommer un algorithme qui construit un arbre couvrant pour le graphe G et qui soit cohérent avec les choix opérés par le type `game_t` de représenter un graphe par des listes d'adjacence.

Soit T et T' un couple d'arbres couvrants éloignés. On pose

$$P = \bigcup_{\alpha_0 \in E \setminus (T \cup T')} \Lambda(\alpha_0).$$

On appelle U l'ensemble des arêtes incidentes à P . On appelle n_U le cardinal de U , m_P le cardinal de P , et $c = |E \setminus (T \cup T')|$ le nombre d'arêtes hors de T et T' . On note κ le nombre de composantes connexes de (U, P) .

Soit $\tilde{T}$ et $\tilde{T}'$ un autre couple d'arbres couvrants éloignés.

Notons $\tilde{c}_1$ le cardinal de $(E \setminus (\tilde{T} \cup \tilde{T}')) \cap P$.

□ 31 – Montrer que

$$c \geq \tilde{c}_1 \geq m_P - 2(n_U - \kappa).$$

En déduire que le cas 2 de la question 28 se produit pour toute arête α_0 uniquement quand T et T' sont éloignés.

□ 32 – D'après la question 28, pour toute arête α_0 n'appartenant pas à $T \cup T'$, l'ensemble $\Lambda(\alpha_0) \subseteq \hat{E}$ est bien défini. Démontrer que P est l'ensemble des arêtes principales.

Épreuve d'Informatique 2 MPI 2025

□ 33 – Expliquer comment on peut construire l'ensemble des arêtes principales et en déduire une stratégie gagnante pour un joueur. Une réponse en pseudo-code est permise, mais seule la logique de programmation sera évaluée.

FIN DE L'ÉPREUVE



Informatique

MPI

2025

4 heures

Calculatrice autorisée

Rush Hour

L'épreuve est composée de deux problèmes indépendants. L'objectif du premier problème est d'écrire un programme permettant de résoudre le casse-tête de déplacements nommé *Rush Hour* dont le but est de faire sortir une voiture d'un parking. Dans le second problème, on souhaite connaître le maximum de véhicules qu'il est possible de placer sur le terrain d'une *casse automobile*.

Les candidats devront répondre aux questions en utilisant le langage C dans le premier problème et le langage OCaml dans le second problème. Ils sont invités, lorsque c'est pertinent, à décrire succinctement le fonctionnement de leurs programmes, un dessin étant souvent plus parlant qu'un long discours. Le barème tient compte de la clarté et de la concision des programmes. Nous recommandons de choisir des noms de variables intelligibles et de structurer, si cela s'avère nécessaire, de longs codes par des fonctions auxiliaires dont on décrira le rôle.

On identifiera une même grandeur écrite dans deux polices différentes, en italique lorsque nous la considérerons d'un point de vue mathématique (par exemple n), et en police à largeur fixe lorsque nous la considérerons d'un point de vue informatique (par exemple `n`). Si $a, b \in \mathbb{Z}$, on note $\llbracket a, b \rrbracket$ l'ensemble $\{k \in \mathbb{Z} \mid a \leq k < b\} = \{a, a+1, \dots, b-1\}$. Un graphe non orienté est un couple $G = (S, A)$ où S est un ensemble fini non vide d'éléments appelés sommets et A un ensemble de parties de S à deux éléments, appelées arêtes. Si $x, y \in S$ sont deux sommets distincts de S , l'arête $\{x, y\}$ sera notée $x - y$. Un graphe orienté est un couple $G = (S, A)$ où S est un ensemble fini non vide d'éléments appelés sommets et A un ensemble d'éléments (x, y) de $S \times S$ tels que $x \neq y$, appelés arcs. Si $x, y \in S$ sont deux sommets distincts de S , l'arc (x, y) sera noté $x \rightarrow y$.

Par complexité en temps d'un algorithme, on entend nombre d'opérations élémentaires (comparaison, addition, soustraction, multiplication, division, affectation, test, etc.) nécessaires à l'exécution de l'algorithme dans le pire des cas.

Partie A – Rush Hour

Le jeu de plateau *Rush Hour* simule un embouteillage dans un parking à l'heure de pointe. Dans ce jeu à un joueur, on doit extraire le véhicule gris d'une grille dans laquelle plusieurs autres véhicules noirs bloquent la sortie. Il est possible de déplacer chaque véhicule, d'une ou plusieurs cases, vers l'avant ou l'arrière. Les véhicules sont soit des voitures, soit des camions. Les voitures occupent 2 cases tandis que les camions occupent 3 cases. La sortie est repérée par une flèche qui se situe sur le côté droit de la grille. La figure 1 illustre une position de départ du jeu.

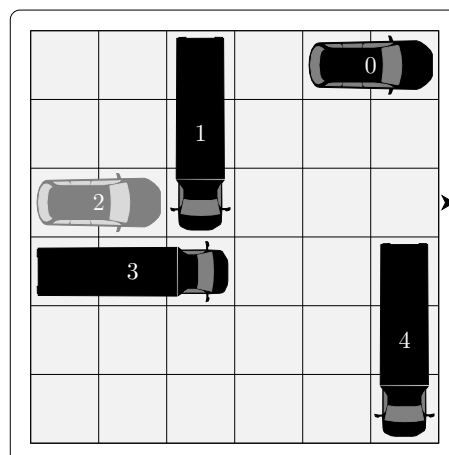


Figure 1.

Pour résoudre ce casse-tête, on commence par déplacer la voiture 0 d'une case en arrière. On se retrouve alors dans la configuration de la figure 2.a. Puis on déplace le camion 4 de 3 cases en arrière, le camion 3 de 3 cases en avant et le camion 1 de 3 cases en avant. On se retrouve alors dans la configuration de la figure 2.b qui nous permet de déplacer la voiture grise numérotée 2 de 3 cases vers l'avant et nous retrouver sans la configuration de la figure 2.c.

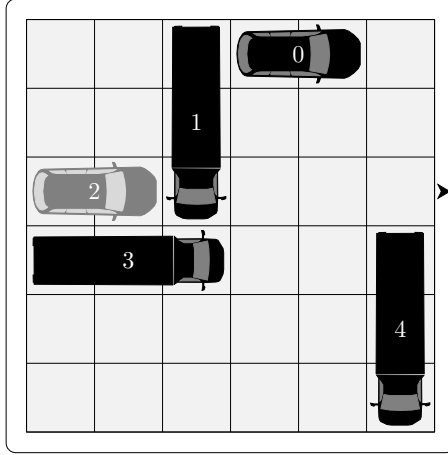


Figure 2.a

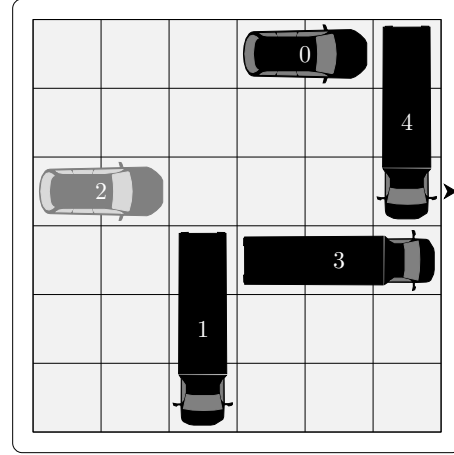


Figure 2.b

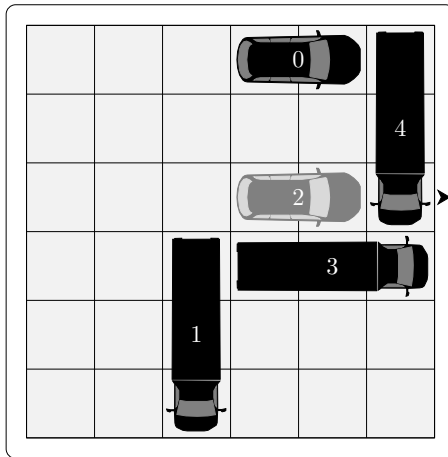


Figure 2.c

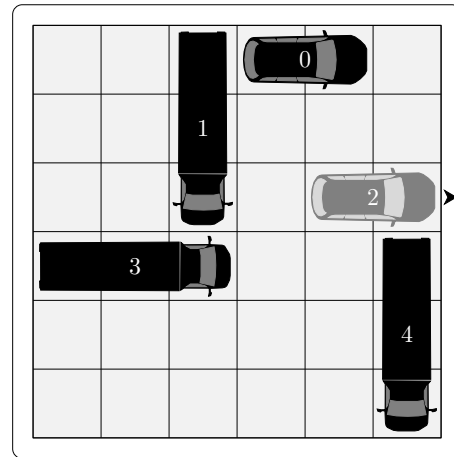


Figure 2.d

On déplace alors le camion 1 de 3 cases vers l'arrière, puis le camion 3 de 3 cases vers l'arrière et le camion 4 de 3 cases vers l'avant. On avance enfin la voiture grise numérotée 2 d'une case. Elle se retrouve devant la sortie indiquée par la flèche sur la droite de la grille. Cette configuration gagnante est indiquée à la figure 2.d.

Dans ce problème, nous supposons que la grille du jeu est carrée et comporte $M \times M$ cases. Comme tous les véhicules occupent au moins 2 cases, le nombre de véhicules présents sur la grille est majoré par $(M \times M)/2$. Dans notre programme, cet entier est noté MAX_V . Les cases sont repérées par leurs coordonnées $(i, j) \in \llbracket 0, M \rrbracket^2$, l'indice i indiquant le numéro de la ligne et l'indice j indiquant le numéro de la colonne. La ligne sur laquelle se trouve la sortie est indiquée par S . Dans l'exemple de la figure 3 disponible page suivante, on a $M = 6$ et $S = 2$. Dans la suite du sujet, nous supposons que la voiture grise est l'unique voiture placée horizontalement sur la ligne de la sortie. Comme c'est la seule voiture pouvant sortir de la grille, contrairement aux schémas présents dans ce sujet sur lesquels elle est en gris, elle ne fera l'objet d'aucun marquage particulier dans notre algorithme.

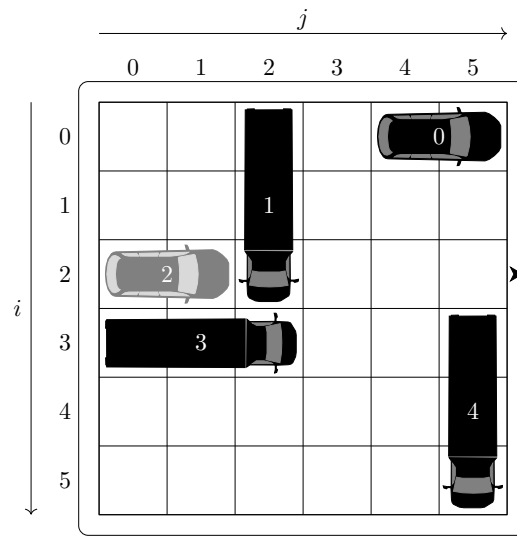


Figure 3.

Nous supposons que les entêtes suivants ont été inclus et que les constantes ci-dessous sont définies au début du programme.

```

1 #include <assert.h>
2 #include <stdbool.h>
3 #include <stdint.h>
4 #include <stdlib.h>
5
6 #define M 6
7 #define MAX_V (M * M) / 2
8 #define S 2

```

Les directives **#define** présentes ici ont pour but de définir des synonymes pour les constantes entières utilisées dans le sujet. Chaque fois que l'identifiant **M** sera présent dans la suite, il sera directement substitué par la constante 6 par le compilateur.

I – Introduction

La position de chaque véhicule sur la grille est représentée par la structure de donnée suivante :

```

1 struct vehicule_s {
2     int i;
3     int j;
4     int longueur;
5     char direction;
6 };
7 typedef struct vehicule_s vehicule;

```

Le couple (i, j) représente les coordonnées de la case occupée par le véhicule située la plus en haut à gauche sur la grille. Par exemple, sur la figure 3, les coordonnées du véhicule 0 sont $(0, 4)$ et celles du véhicule 1 sont $(0, 2)$. Le champ **longueur** indique la longueur, en nombre de cases, du véhicule : 2 pour une voiture et 3 pour un camion. Enfin, le champ **direction** indique la direction du véhicule : 'H' si le véhicule est placé de manière horizontale, comme le véhicule 0 de la figure 3, et 'V' si le véhicule est placé de manière verticale, comme le véhicule 1 de la figure 3.

La configuration des véhicules sur le plateau de jeu est représentée par la structure suivante :

```

1 struct etat_s {
2     vehicule v[MAX_V];
3     int n;
4     int distance;
5 };
6 typedef struct etat_s etat;

```

Le nombre de véhicules présents sur la grille est donné par le champ `n` et vérifie `n ≤ MAX_V`. Pour tout $k \in \llbracket 0, n \rrbracket$, le champ `v[k]` donne la position et les caractéristiques du véhicule d'indice k . Enfin, le champ `distance` n'est pas utilisé pour le moment et sera initialisé à 0.

- Q1.** Écrire une fonction `etat *e_nouveau(void)` renvoyant un état ne possédant aucune voiture sur son plateau, ainsi qu'une fonction `void e_libere(etat *e)` libérant la mémoire associée à cet état.
- Q2.** Écrire une fonction `bool e_egal(etat *e1, etat *e2)` déterminant si deux états sont égaux, c'est-à-dire possédant le même nombre de véhicules n et tels que pour tout $k \in \llbracket 0, n \rrbracket$ les véhicules d'indice k ont mêmes positions et mêmes caractéristiques. Deux états peuvent être égaux en ayant des valeurs différentes associées au champ `distance`.
- Q3.** Écrire une fonction `void e_ajoute_vehicule(int i, int j, int longueur, char direction, etat *e)` ajoutant à l'état e un véhicule à la position (i, j) dont la longueur et la direction sont données par les variables homonymes. On ne cherchera pas à vérifier que les cases sont libres, mais on utilisera une assertion pour vérifier que le nombre n de véhicules présents sur la grille reste inférieur ou égal à `MAX_V`.

Afin de faciliter la manipulation des états, nous allons définir une structure `plateau` représentant le plateau de jeu. Cette structure est définie comme suit :

```
1 const int vide = -1;
2
3 struct plateau_s {
4     int tab[M][M];
5 };
6 typedef struct plateau_s plateau;
```

- Q4.** Écrire une fonction `plateau *p_etat(etat *e)` renvoyant un plateau dont les cases sont initialisées à la valeur `vide`, excepté les cases sur lesquelles se trouvent les véhicules de l'état e qui sont chacune initialisée avec l'indice du véhicule la recouvrant.

Pour chaque état, seul un nombre fini de déplacements est valide. Pour manipuler cette collection, nous allons utiliser une liste chaînée, dont le maillon `deplacement` est défini comme ceci :

```
1 struct deplacement_s {
2     int k;
3     int pas;
4     struct deplacement_s *suivant;
5 };
6 typedef struct deplacement_s deplacement;
```

Un tel maillon permet de représenter un déplacement de la voiture d'indice k de `pas` cases dans le sens des i (ou des j) croissants. Par exemple, dans la solution du casse-tête de la figure 1, le premier mouvement de la voiture 0 d'une case vers la gauche aura une valeur de `k = 0` et de `pas = -1`. Le second mouvement du camion 4 de 3 cases vers le haut aura une valeur de `k = 4` et de `pas = -3`. Le champ `suivant` pointe vers le prochain maillon de la liste chaînée. La liste vide est représentée par le pointeur `NULL`.

- Q5.** Écrire une fonction `deplacement *d_cons(int k, int pas, deplacement *lst)` ajoutant en tête de la liste chaînée `lst`, un déplacement de la voiture d'indice k de `pas` cases.

Pour la suite du sujet, on suppose disposer d'une fonction `deplacement *e_deplacements(etat *e)` renvoyant la liste des déplacements valides dans l'état e .

- Q6.** Implémenter une fonction `void d_libere(deplacement *lst)` libérant la mémoire occupée par les différents éléments de la liste `lst`.

II – Implémentation d'une file

Dans cette partie, nous allons réaliser une file à l'aide d'un tableau circulaire. Sur l'exemple ci-dessous, nous avons une file d'entiers représentée par un tableau possédant 8 cases. On dit que la file a une capacité de 8 éléments.

8					17	25	9
0	1	2	3	4	5	6	7

prochain

Ce tableau représente la file $17 \leftarrow 25 \leftarrow 9 \leftarrow 8$. Le prochain élément à sortir de la file est 17 et le dernier élément à avoir été ajouté est 8. L'indice **prochain**, qui est égal à 5 sur cet exemple, indique l'indice du prochain élément qui sortira de la file. Cette file possède 4 éléments. On dit que sa taille est égale à 4.

La file que nous allons implémenter est une file d'états. Elle est représentée par la structure suivante :

```

1 struct file_s {
2     etat **tab;
3     int prochain;
4     int taille;
5     int capacite;
6 };
7 typedef struct file_s file;
```

- Q7.** Écrire une fonction `file *f_nouveau(void)` qui renvoie une file de taille et de capacité nulle. Écrire d'autre part une fonction `void f_libere(file *f)` libérant la mémoire associée à la file f . Cette fonction ne devra pas libérer la mémoire associée aux états présents dans la file.
- Q8.** Écrire une fonction `void f_change_capacite(file *f, int c)` prenant en entrée une file de taille m ainsi qu'un entier $c \geq m$. Cette fonction devra modifier la capacité de la file f afin qu'elle devienne égale à c .
- Q9.** Écrire les fonctions `void f_enfile(file *f, etat *e)` et `etat *f_defile(file *f)` permettant respectivement d'ajouter un nouvel élément e à l'entrée de la file f et d'enlever et récupérer un élément à la sortie de la file. Lorsqu'on souhaite ajouter un nouvel élément dans une file dont la capacité est égale à la taille, on multiplie cette capacité par 2 (sauf pour le cas particulier où la capacité est nulle où on la change en 1) avant d'ajouter notre nouvel élément.

On souhaite montrer que la complexité amortie des fonctions `f_enfile` et `f_defile` est en $\mathcal{O}(1)$, c'est-à-dire que si l'on part d'une file vide, la complexité d'une succession de n appels, chaque appel se faisant à l'une de ces fonctions, a une complexité en $\mathcal{O}(n)$. Pour les deux questions suivantes, et seulement pour ces deux questions, la complexité d'une opération sera calculée comme le nombre d'accès aux éléments du tableau `tab`, que ces accès soient en lecture ou en écriture. Nous allons utiliser pour cela la méthode du potentiel. On définit le potentiel Φ d'une file f par

$$\Phi(f) = |4 \cdot \text{taille}(f) - 2 \cdot \text{capacite}(f)|.$$

On note $f_0, f_1, \dots, f_n$ les états successifs de la file. L'état f_0 est son état initial qui est une file de taille et de capacité nulle. Pour tout $k \in \llbracket 1, n \rrbracket$, f_k est l'état de la file après la k -ième opération.

$$f_0 \xrightarrow{\text{op}_1} f_1 \xrightarrow{\text{op}_2} f_2 \cdots f_{n-1} \xrightarrow{\text{op}_n} f_n.$$

Pour tout $k \in \llbracket 1, n \rrbracket$, on note C_k la complexité de l'opération op_k qui fait passer la file de l'état f_{k-1} à l'état f_k .

- Q10.** Montrer qu'il existe un entier $K \geq 0$ que l'on déterminera, tel que

$$\forall k \in \llbracket 1, n \rrbracket, \quad C_k + \Phi(f_k) - \Phi(f_{k-1}) \leq K.$$

- Q11.** En déduire que

$$\sum_{k=1}^n C_k \leq Kn$$

puis conclure.

III – Table de hachage

Dans cette partie, nous allons implémenter une structure d'ensemble à l'aide d'une table de hachage en adressage ouvert. Les éléments de cet ensemble sont de type `etat *`. Nous allons commencer par définir une fonction de hachage. Pour tout état e , on définit

$$\text{hash}(e) = \sum_{\substack{0 \leq i < M \\ 0 \leq j < M}} d_{i,j} 3^{iM+j}$$

où pour tout $i, j \in \llbracket 0, M \rrbracket$

$$d_{i,j} = \begin{cases} 0 & \text{si la case } (i, j) \text{ est inoccupée,} \\ 1 & \text{si la case } (i, j) \text{ est occupée par un véhicule horizontal,} \\ 2 & \text{si la case } (i, j) \text{ est occupée par un véhicule vertical.} \end{cases}$$

On suppose disposer d'une fonction `plateau *p_hash(etat *e)` qui étant donné un état e , renvoie un plateau p dont les éléments `tab[i][j]` sont égaux à $d_{i,j}$.

Q12. Écrire une fonction `int64_t e_hash(etat *e)` prenant en entrée un état e et renvoyant $\text{hash}(e)$.

Q13. Montrer que si e_1 et e_2 sont deux états accessibles à partir d'un même état initial e et que $\text{hash}(e_1) = \text{hash}(e_2)$, alors $e_1 = e_2$.

Pour stocker les états dans notre table de hachage, nous allons utiliser un tableau possédant un nombre de cases appelé capacité de la table de hachage. Cette capacité sera de la forme 2^p . La taille de la table de hachage est le nombre d'états qu'elle contient. Lorsqu'on souhaite ajouter un état e dans notre table de hachage, on commencera par calculer $h = \text{hash}(e) \bmod 2^p$. Si la case d'indice h est vide, on placera e dans cette case. Sinon, on cherchera la première case vide en partant de l'indice h et en avançant de un en un de manière circulaire dans le tableau, jusqu'à trouver une case vide. Par exemple, si l'on part d'une table de hachage vide de capacité 8, et qu'on ajoute successivement les états e_0, e_1, e_2, e_3 et e_4 ayant des hash respectifs de $h_0 = 1, h_1 = 7, h_2 = 3, h_3 = 3$ et $h_4 = 7$, on obtiendra la table de hachage suivante.

e_4	e_0		e_2	e_3			e_1
0	1	2	3	4	5	6	7

Lorsque la table de hachage est trop peuplée, on double sa capacité et on réinsère les éléments de l'ancienne table de hachage dans la nouvelle. Nous utiliserons la structure suivante pour représenter notre table de hachage.

```
1 struct tableh_s {
2     etat **tab;
3     int taille;
4     int capacite;
5 };
6 typedef struct tableh_s tableh;
```

Le tableau `tab` contient les différents états présents dans la table de hachage. Nous utiliserons le pointeur `NULL` pour représenter les cases vides de la table. On suppose que l'on dispose d'une fonction `tableh *t_nouveau(void)` créant une table de hachage vide ayant une capacité de 1 et d'une fonction `void t_libere(tableh *t)` libérant la mémoire associée à la table de hachage t , ainsi que la mémoire associée à tous les états qu'elle contient.

Q14. Écrire une fonction `bool t_contient(tableh *t, etat *e)` déterminant si l'état e est présent dans la table de hachage t .

On suppose disposer d'une fonction `void t_augmente_capacite(tableh *t)` qui prend en entrée une table de hachage t de capacité $c \geq 1$ et qui augmente sa capacité à $2c$.

Q15. Écrire une fonction `void t ajoute(tableh *t, etat *e)` qui ajoute un état e à la table de hachage t . Si m est la taille de la table et c sa capacité, on fera attention à ce que l'on ait toujours $m \leq 2c/3$.

Q16. Expliquer en quoi ce serait une mauvaise idée de remplacer le 3^{iM+j} par 4^{iM+j} dans la définition de $\text{hash}(e)$.

IV – Recherche d’une solution optimale

On supposera disposer dans cette partie d’une fonction `bool e_gagne(etat *e)` déterminant si la voiture grise est devant la sortie, ainsi que d’une fonction `etat *e_copie(etat *e)` effectuant une copie d’un état e .

Q17. Écrire une fonction `int resout(etat *e)` prenant en entrée un état e et renvoyant le nombre minimal de mouvements pour résoudre le jeu. La fonction devra renvoyer -1 dans le cas où il n’y a pas de solution. On souhaite de plus que la mémoire associée à l’état e soit libérée à la fin de l’appel à `resout`. On pourra utiliser le champ `distance` d’un état pour indiquer le nombre de coups minimal nécessaires pour atteindre cet état à partir de l’état initial.

Q18. Montrer que la fonction `resout` n’engendre aucun problème mémoire.

Partie B – La casse automobile

Dans cette partie, nous disposons d’un terrain sur lequel nous souhaitons entreposer le plus grand nombre de véhicules hors d’usage. Ce terrain est modélisé par un ensemble de cases du plan, extraites d’une grille régulière. La taille d’un terrain désigne le nombre de cases le constituant. Deux types de véhicules sont disponibles : les voitures, qui occupent 2 cases, et les camions, qui occupent 3 cases.

Le premier objectif de cette partie est d’implémenter un algorithme efficace, ayant une complexité polynomiale en la taille du terrain, permettant de déterminer le maximum de voitures qu’il est possible de disposer sur ce dernier. Cet algorithme nous permet notamment de décider s’il est possible de couvrir tout le terrain à l’aide de voitures. La figure 4 donne un exemple de couverture complète d’un terrain possédant 14 cases à l’aide de voitures.

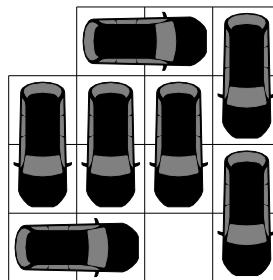


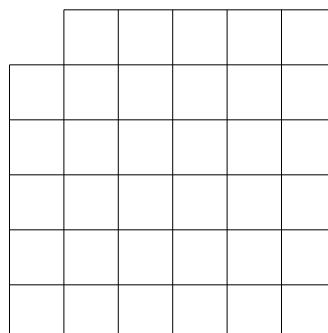
Figure 4.

Le second objectif de cette partie est de démontrer que, si l’on remplace les voitures, qui occupent 2 cases, par des camions, qui occupent 3 cases, le problème devient NP-complet.

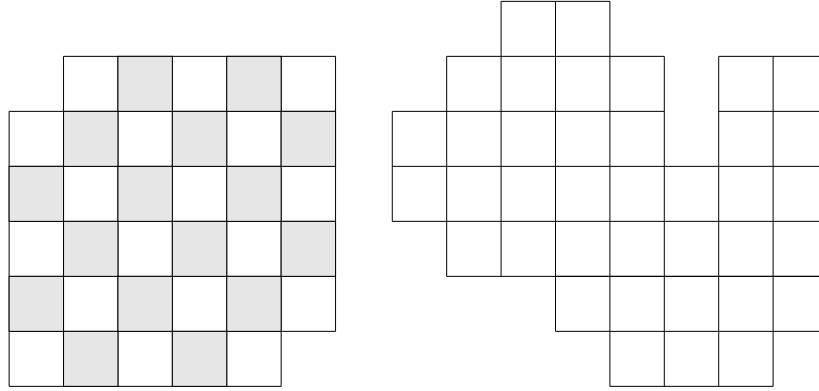
I – Remplir une casse avec le maximum de voitures

Dans cette partie B.I, tous les véhicules considérés sont des voitures et occupent 2 cases.

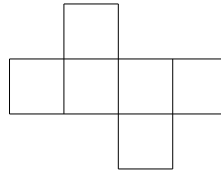
Q19. Prouver qu’il est impossible de couvrir tout le terrain suivant à l’aide de voitures. En déduire une condition nécessaire sur le terrain pour qu’il soit possible de le couvrir entièrement à l’aide de voitures.



- Q20.** Prouver qu'il est impossible de couvrir entièrement le terrain de gauche ci-dessous à l'aide de voitures. Ce terrain a une taille de 34 cases et la coloration de ses cases en blanc et en noir à la manière d'un échiquier n'a aucune incidence sur la disposition des voitures. Est-il possible de couvrir entièrement le terrain de droite ci-dessous ? On attend soit un exemple de couverture, soit une preuve de son impossibilité. En déduire une nouvelle condition nécessaire sur le terrain pour qu'il soit possible de le couvrir entièrement à l'aide de voitures.



- Q21.** En considérant le terrain ci-dessous, prouver que la condition nécessaire trouvée dans la question précédente n'est pas suffisante pour assurer l'existence d'une couverture complète à l'aide de voitures.



Dans la suite de cette partie, nous allons écrire un algorithme permettant de déterminer le plus grand nombre de voitures qu'il est possible de disposer sur un terrain, ainsi qu'une telle disposition. Pour cela, on définit le graphe non orienté $G = (S, A)$ dont l'ensemble des sommets S est constitué des cases du terrain, une arête $x - y$ appartenant à A si et seulement si x et y sont deux cases adjacentes, c'est-à-dire lorsque y est l'une des 4 cases au-dessus, en dessous, à gauche ou à droite de x . Nous choisissons arbitrairement de colorier une case du terrain en noir, puis de colorier le reste du terrain à la manière d'un échiquier en alternant les couleurs blanches et noires. On note N l'ensemble des sommets noirs et B l'ensemble des sommets blancs. Puisqu'une arête relie toujours deux sommets de couleurs différentes, $G = (N \sqcup B, A)$ est un graphe biparti. On numérote les r cases du terrain en commençant par les p cases noires, puis les q cases blanches. On a donc $r = p + q$, $N = \llbracket 0, p \rrbracket$ et $B = \llbracket p, r \rrbracket$. En OCaml, ce graphe sera représenté par un tableau de listes d'adjacence ($g : \text{int list array}$) ainsi que l'entier ($p : \text{int}$). Pour tout $x \in \llbracket 0, r \rrbracket$, $g.(x)$ est la liste des voisins du sommet x .

On rappelle qu'un couplage de $G = (S, A)$ est une partie C de A telle que chaque sommet n'est l'extrémité que d'au plus une arête de C . Un sommet $x \in S$ est dit libre lorsqu'il n'est l'extrémité d'aucune arête de C . Deux sommets $x, y \in S$ sont dits conjoints lorsqu'ils sont les extrémités d'une même arête de C . En OCaml, un couplage sera représenté par le tableau ($\text{conjoint} : \text{int option array}$) défini par

- $\text{conjoint}.(x) = \text{None}$ lorsque le sommet x est libre.
- $\text{conjoint}.(x) = \text{Some } y$ et $\text{conjoint}.(y) = \text{Some } x$ lorsque x et y sont conjoints.

Un chemin $x_0 - x_1 - \dots - x_k$ de longueur k sera représenté en OCaml par la liste $[x_0; \dots; x_k]$ et sera représenté par la variable ($w : \text{int list}$). Un chemin est dit élémentaire lorsque ses sommets sont deux à deux distincts. Un chemin est dit alternant pour C lorsqu'il est élémentaire et ses arêtes $x_i - x_{i+1}$ alternent entre des éléments de C et $A \setminus C$. On dit qu'il est augmentant lorsqu'il est alternant et que ses extrémités sont libres.

Si X et Y sont deux parties de A , on définit la différence symétrique $X \Delta Y = (X \cup Y) \setminus (X \cap Y)$. On admet que cette opération est associative et commutative. Si X est une partie de A et w est un chemin, $X \Delta w$ est défini en considérant le chemin w comme un ensemble d'arêtes.

Q22. Montrer que si C est un couplage et que w est un chemin augmentant pour C , alors $C \Delta w$ est un couplage.

Q23. Écrire une fonction `delta` (`conjoint : int option array`) (`w : int list`) : `unit` prenant en entrée un couplage C représentée par le tableau `conjoint` et un chemin w que l'on supposera augmentant et modifiant `conjoint` de manière à ce qu'il représente le couplage $C \Delta w$.

On définit le graphe orienté G_C possédant les mêmes sommets que G et possédant un arc orienté pour chaque arête $n - b$ de G où $n \in N$ et $b \in B$: si $n - b \in C$, cet arc est $b \rightarrow n$, sinon cet arc est $n \rightarrow b$. D'autre part, dans la suite de cette partie, on utilisera l'enregistrement

```
1 type couplage = {
2   graphe : int list array;
3   p : int;
4   conjoint : int option array;
5 }
```

pour représenter un couplage ainsi que le graphe biparti auquel il est associé. Si `c` est une variable de type `couplage`, `c.graphe` permet d'accéder au tableau de listes d'adjacence du graphe, `c.p` permet d'accéder à l'entier p et `c.conjoint` permet d'accéder au tableau `conjoint`. On construit un couplage à l'aide de la syntaxe

```
{ graphe = g; p = p; conjoint = c }
```

Enfin, on note respectivement N_ℓ et B_ℓ l'ensemble des sommets libres de N et de B .

Q24. Montrer qu'un chemin $x_0 - x_1 - \dots - x_k$ de G dont le premier élément x_0 est dans N_ℓ et le dernier élément x_k est dans B_ℓ est augmentant pour C si et seulement si $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_k$ est un chemin de G_C .

Q25. Écrire une fonction `orienter` (`c : couplage`) : `int list array` prenant en entrée un couplage C et renvoyant le tableau de listes d'adjacence représentant le graphe orienté G_C .

Pour tout $x \in S$, on note $d(x)$ la plus petite longueur parmi les longueurs des chemins alternants reliant un élément $n \in N_\ell$ à x . Si un tel chemin n'existe pas, on pose par convention $d(x) = +\infty$. Afin de manipuler efficacement cette distance, nous utiliserons par la suite l'enregistrement

```
1 type distance = {
2   d : int array;
3   dmin : int;
4 }
```

où `d` est un tableau tel que `d.(x) = d(x)` pour tout $x \in \llbracket 0, r \rrbracket$, la valeur $+\infty$ étant représenté par l'entier `max_int`, et `dmin` est la longueur $d_{\min}$ du plus court chemin augmentant pour C , avec par convention `dmin = max_int` lorsqu'un tel chemin n'existe pas.

Q26. Écrire une fonction `distance` (`c : couplage`) : `distance` prenant en entrée un couplage C et renvoyant l'enregistrement (`dist : distance`) associé à la distance d . On pourra utiliser le module `Queue` de la bibliothèque standard OCaml offrant une implémentation d'une file impérative et dont la signature est rappelée ci-dessous.

- `Queue.create () : 'a Queue.t`, création d'une file vide.
- `Queue.push (x : 'a) (f : 'a Queue.t) : unit`, enfiler un élément x dans la file f .
- `Queue.pop (f : 'a Queue.t) : 'a`, défiler et renvoyer le prochain élément en attente de la file f .
- `Queue.is_empty (f : 'a Queue.t) : bool`, renvoie `true` lorsque la file f est vide, `false` sinon.

Nous souhaitons construire le plus grand ensemble de chemins augmentants pour C tel que ces chemins n'aient aucun sommet en commun et soient tous de longueur $d_{\min}$. Un tel ensemble est appelé ensemble de chemins de Hopcroft-Karp. Pour construire cet ensemble, on commence par définir le graphe G_{HK} possédant les mêmes sommets que G_C , en ne conservant que les arcs $x \rightarrow y$ tels que $d(y) = d(x) + 1$ et $d(y) \leq d_{\min}$. Ensuite, on initialise notre ensemble de chemins à l'ensemble vide. Puis, pour chaque sommet $x \in N_\ell$, on cherche un chemin reliant x à un sommet $y \in B_\ell$ dans G_{HK} à l'aide d'un parcours en profondeur. Tant qu'un tel chemin existe, on retire de G_{HK} les sommets empruntés par notre chemin, on ajoute ce chemin à notre ensemble de chemins et on réitère l'opération. Pour notre implémentation, on ne

cherchera pas à générer le graphe G_{HK} , mais on travaillera uniquement à partir du graphe G_C .

On souhaite donc écrire une fonction `ensemble_chemin` (`c : couplage`) (`dist : distance`) : `int list list` prenant en entrée un couplage C , ainsi que l'enregistrement `dist` calculé par la fonction `distance` et renvoyant un ensemble de chemins de Hopcroft-Karp sous forme de liste. Pour cela, on introduit l'exception

```
1 exception Chemin of int list
```

et on définit la fonction

```
1 let ensemble_hc (c : couplage) (dist : distance) : int list list =
2   let r = Array.length c.graphe in
3   let visite = Array.make r false in
4   let go = oriente c in
5   let ensemble = ref [] in
6   let rec chemin (x : int) (w : int list) : unit =
7     <code1>
8   in
9   <code2>
10  !ensemble
```

Q27. Compléter la partie `<code1>` de la fonction `ensemble_hc` afin de définir la fonction

`chemin (x : int) (w : int list) : unit.`

Cette fonction doit effectuer un parcours en profondeur dans G_{HK} à partir du sommet x en ayant déjà parcouru le chemin w codé à l'envers (le dernier sommet visité est en tête de liste), et renvoyer une exception `Chemin` w lorsqu'un chemin augmentant w a été trouvé.

Q28. Compléter la partie `<code2>` de la fonction `ensemble_hc`.

Dans la question suivante, on admet le lemme de Berge : un couplage dans un graphe est maximum pour l'inclusion si et seulement si il n'existe pas de chemin augmentant.

Q29. Définir la fonction `couplage_maximum` (`g : int list array`) (`p : int`) : `int option array` prenant en entrée un graphe biparti donné par son tableau de listes d'adjacence ainsi que par son nombre de sommets noirs et renvoyant un couplage maximum pour ce graphe.

II – Remplir une casse avec le maximum de camions

Dans cette partie, on souhaite montrer que le problème qui consiste, étant donné un terrain t ainsi qu'un entier n , de déterminer s'il est possible de placer au moins n camions sur le terrain t , est NP-complet. Dans la suite, on nomme ce problème 3CASSE.

Q30. Définir ce qu'est un problème de décision appartenant à la classe NP. Définir ce qu'est un problème NP-difficile, puis un problème NP-complet.

On définit les types

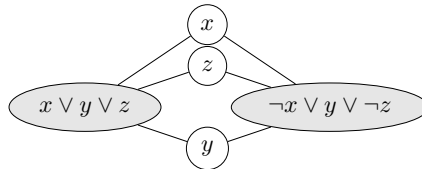
```
1 type terrain = bool array array
2 type direction = Horizontal | Vertical
3 type camion = {i : int; j : int; d : direction}
```

le type `terrain` représentant un terrain à l'aide d'un tableau bidimensionnel de booléens. Pour chaque case de cette grille, le booléen associé est égal à `true` si et seulement si elle fait partie du terrain. Le type `camion` représente quant à lui la position d'un camion sur le terrain, de manière similaire à ce qui a été décrit dans la partie A.I.

Q31. Écrire une fonction `est_valide` (`t : terrain`) (`n : int`) (`lst : camion list`) : `bool` prenant en entrée un terrain t , un entier n et une liste de positions de camions `lst` et renvoyant `true` si la liste possède n camions et qu'il est possible de les placer sur le terrain t comme spécifié. Si ce n'est pas le cas, la fonction renverra `false`.

Q32. En déduire que le problème 3CASSE est dans la classe NP.

On souhaite montrer que 3CASSE est NP-difficile. Pour cela, nous allons partir d'une variante du problème 3SAT, nommée Planar 3SAT et qui est également NP-complet. Tout comme pour 3SAT, les instances de ce problème de décision sont des formules logiques en forme normale conjonctive. Pour une formule φ on considère le graphe non orienté dont les sommets sont formés des variables et des clauses et dont les arêtes relient une variable à une clause si et seulement si la variable est présente dans la clause. Le problème Planar 3SAT est semblable à 3SAT mais restreint les formules à celles dont le graphe est planaire. Par exemple, la formule $\varphi = (x \vee y \vee z) \wedge (\neg x \vee y \vee \neg z)$ est planaire puisqu'on peut la représenter par le graphe suivant :



Étant donnée une formule φ de Planar 3SAT, nous allons construire une instance de 3CASSE. Pour cela, nous allons construire un terrain à l'aide de *gadgets*. L'essentiel de ces gadgets peut accueillir un nombre fixe de camions. Les gadgets représentant des clauses sont des exceptions et peuvent accueillir un camion en plus lorsque cette clause est vraie. Les gadgets sont reliés entre eux afin de créer des chemins, transportant des *signaux*. Chaque signal représente la valeur d'un littéral x ou de sa négation $\neg x$.

Concrètement, les gadgets sont reliés entre eux en superposant une de leur case, par laquelle sera transmis ce signal. Les gadgets sont agencés de manière à ce que tout camion placé sur le terrain se trouve entièrement sur un unique gadget. Par exemple, le terrain de la figure 5 est composé de 4 gadgets $\alpha, \beta, \gamma, \delta$ reliés par les cases a, b, c et d . Les gadgets α et γ sont reliés par la case a qui appartient aux deux gadgets. Sur ce terrain, nous avons placé deux camions. Le camion 1 appartient au gadget γ . Bien qu'il empiète sur le gadget α , on ne considère pas qu'il appartienne à ce gadget. Le camion 2 appartient quant à lui au gadget δ .

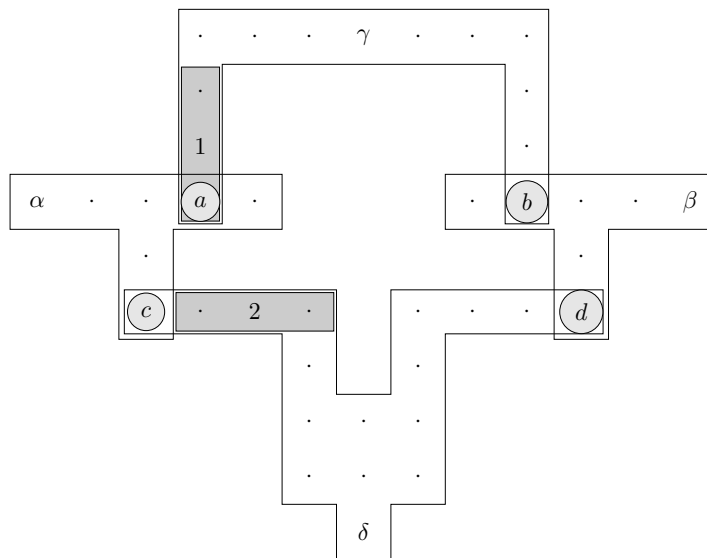
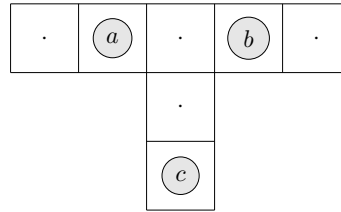


Figure 5.

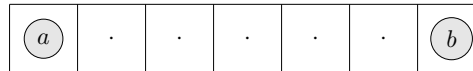
La disposition des camions sur le terrain va permettre de représenter les signaux. Si ε est un gadget et qu'un camion n'appartenant pas à ce gadget empiète sur la case e du gadget ε , on dit que le gadget ε possède une protrusion en e . On dit dans ce cas que, pour le gadget ε , le signal e est faux. Dans le cas contraire, on dit que ce signal est vrai. Par exemple, avec la disposition de camions ci-dessus, pour le gadget α , on dit que le signal a est faux et que le signal c est vrai.

Q33. Afin de représenter une clause $a \vee b \vee c$, nous allons utiliser le gadget suivant sur lequel il peut y avoir des protrusions sur les cases a, b, c .



Montrer que ce gadget permet seulement d'accueillir 0 ou 1 camion et que l'un des signaux a , b , c est vrai si et seulement si il est possible d'y accueillir un camion.

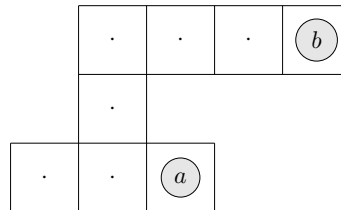
Q34. Afin de transmettre une valeur, nous allons utiliser le gadget suivant, sur lequel on souhaite placer deux camions.



Montrer que, si le signal a est faux, pour placer deux camions sur ce gadget, il est nécessaire de rendre le signal b faux pour le gadget suivant. Montrer que, si le signal a est vrai, il est possible de placer ces deux camions de manière à ce que le signal b soit vrai pour le gadget suivant.

L'idée du gadget précédent peut être généralisée afin d'obtenir un gadget permettant de transmettre un signal tout en se décalant d'un nombre de cases qui est un multiple de 3. Il est cependant nécessaire de transmettre des signaux tout en se décalant d'un nombre de cases quelconque.

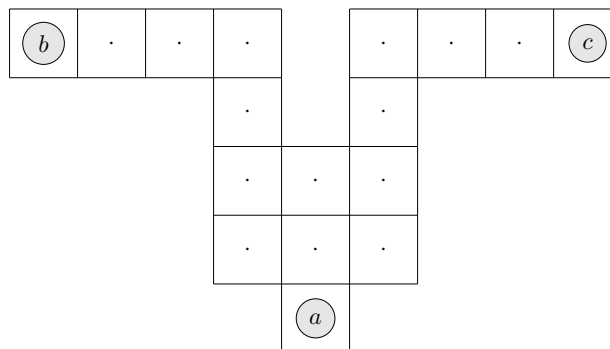
Q35. Afin de transmettre un signal tout en le décalant de deux cases, nous allons utiliser le gadget suivant, sur lequel on souhaite placer deux camions.



Montrer que, si le signal a est faux, pour placer deux camions sur ce gadget, il est nécessaire de rendre le signal b faux pour le gadget suivant. Montrer que, si le signal a est vrai, il est possible de placer ces deux camions de manière à ce que le signal b soit vrai pour le gadget suivant.

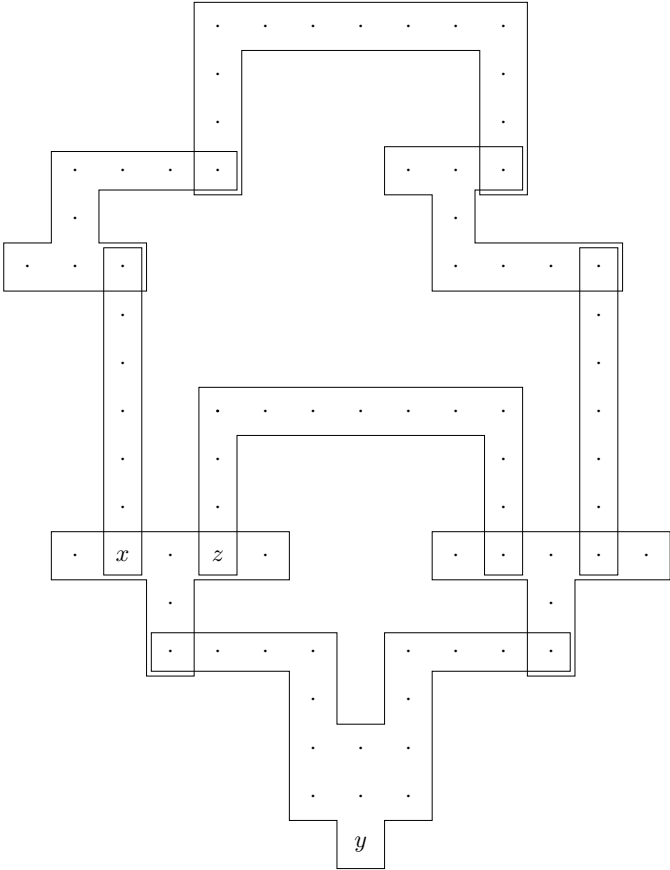
Un signal représentant un littéral x (ou sa négation $\neg x$) peut être utilisé dans plusieurs clauses. C'est pourquoi il est nécessaire de pouvoir le dupliquer.

Q36. Afin de diviser un signal en deux, nous allons utiliser le gadget suivant, sur lequel on souhaite placer cinq camions.



Montrer que, si le signal a est faux, pour placer cinq camions sur ce gadget, il est nécessaire de rendre les signaux b et c faux pour les gadgets suivants. Montrer que, si le signal a est vrai, il est possible de placer cinq camions de manière à ce que les signaux b et c soient vrais pour les gadgets suivants.

Q37. En utilisant les idées développées dans les questions 33 à 36 ainsi que le fait qu'il est possible de satisfaire la formule $\varphi = (x \vee y \vee z) \wedge (\neg x \vee y \vee \neg z)$, déterminer le plus grand nombre de camions qu'il est possible de placer sur le terrain suivant. On justifiera la réponse en donnant une telle disposition ainsi que les valeurs des variables x , y et z associées.



Q38. Montrer que le problème 3CASSE est NP-complet.

◇ Fin ◇

1020 - 6 mars 2025 - 10:48:00

SESSION 2025



MPI5IN

ÉPREUVE MUTUALISÉE AVEC E3A-POLYTECH**ÉPREUVE SPÉCIFIQUE - FILIÈRE MPI****INFORMATIQUE****Durée : 4 heures**

N.B. : le candidat attachera la plus grande importance à la clarté, à la précision et à la concision de la rédaction. Si un candidat est amené à repérer ce qui peut lui sembler être une erreur d'énoncé, il le signalera sur sa copie et devra poursuivre sa composition en expliquant les raisons des initiatives qu'il a été amené à prendre.

RAPPEL DES CONSIGNES

- Utiliser uniquement un stylo noir ou bleu foncé non effaçable pour la rédaction de votre composition ; d'autres couleurs, excepté le vert, bleu clair ou turquoise, peuvent être utilisées, mais exclusivement pour les schémas et la mise en évidence des résultats.
- Ne pas utiliser de correcteur.
- Écrire le mot FIN à la fin de votre composition.

Les calculatrices sont interdites.**Le sujet est composé de trois parties, toutes indépendantes.**

Partie I - Grammaire non contextuelle

Soit la grammaire non contextuelle $G = (\Sigma, V, P, S)$, l'ensemble des règles de production P étant défini par :

$$\begin{aligned} S &\rightarrow SaS \mid A \\ A &\rightarrow AbA \mid B \\ B &\rightarrow BcB \mid \varepsilon \end{aligned}$$

où Σ (respectivement V) est l'ensemble de symboles terminaux (respectivement non terminaux), $S \in V$ est le symbole initial de G et ε dénote le mot vide.

Soit u le mot abc .

Q1. Donner une dérivation à gauche de u . Que peut-on en déduire sur le mot u ?

Q2. Donner deux arbres de dérivation pour le mot aa . En déduire que G est ambiguë.

Une grammaire est dite *récursive gauche directe* si elle possède une règle de la forme $A \rightarrow A\alpha$, où α est une suite de symboles terminaux et non terminaux. A est dite *variable récursive gauche*. Une telle grammaire pose divers problèmes en analyse syntaxique descendante, on cherche alors à éliminer cette récursivité. Ceci est toujours possible pour les langages réguliers.

Q3. Identifier dans G les variables récursives gauches.

Pour éliminer la récursivité gauche, on utilise l'algorithme suivant :

Soit $A \rightarrow A\alpha \mid \beta$ une règle, où α est une suite de symboles terminaux et non terminaux et β est une suite de symboles terminaux et non terminaux ne commençant pas par A .

On remplace cette règle par :

- (i). une règle commençant par $A : A \rightarrow \beta A'$,
- (ii). une règle pour une nouvelle variable $A' : A' \rightarrow \alpha A' \mid \varepsilon$.

Q4. Construire la grammaire non récursive gauche directe G' équivalente à G .

Q5. Prouver que le langage reconnu par G' (ou G) est $\{a, b, c\}^*$.

Partie II - Problème de bin-packing

Cette partie comporte des questions nécessitant un code `OCaml`.

Soient $n, k \in \mathbb{N}^2$. On appelle *rangement* de n objets dans k boîtes une fonction $\mathcal{R} : \llbracket 1, n \rrbracket \rightarrow \llbracket 1, k \rrbracket$. L'ensemble $B_j = \mathcal{R}^{-1}(j) = \{i \in \llbracket 1, n \rrbracket, \mathcal{R}(i) = j\}$ est le contenu de la boîte $j \in \llbracket 1, k \rrbracket$.

Étant données des tailles $a = (a_1 \cdots a_n) \in (\mathbb{Q} \cap]0, 1])^n$, a_i étant la taille de l'objet i , on dit qu'un rangement $\mathcal{R}$ de n objets dans k boîtes est *valide* pour les tailles a si pour tout $j \in \llbracket 1, k \rrbracket$ on a $\sum_{i \in B_j} a_i \leq 1$.

On considère alors le problème de décision BIN-PACKING qui, étant donnés $n, k \in \mathbb{N}^2$ et $a \in (\mathbb{Q} \cap]0, 1])^n$, décide s'il existe un rangement $\mathcal{R}$ de n objets dans k boîtes valide pour les tailles a .

On peut décrire une solution de ce problème par un couple $(k, \mathcal{R})$.

Q6. Montrer que BIN-PACKING est dans NP.

Soit le problème PARTITION suivant :

Pour n entiers $c_1 \cdots c_n$, existe-t-il un sous-ensemble S de $\llbracket 1, n \rrbracket$, tel que $\sum_{i \in S} c_i = \sum_{i \notin S} c_i$?

On admet que le problème PARTITION est NP-complet.

Q7. Montrer par réduction depuis PARTITION que le problème BIN-PACKING est NP-complet.

On s'intéresse par la suite au problème MIN-BIN-PACKING qui demande de déterminer le plus petit k pour lequel il existe un rangement $\mathcal{R}$ de n objets de taille a dans k boîtes. On introduit pour résoudre ce problème une heuristique dite *Premier Casier Décroissant* (**algorithme 2**), se basant sur l'heuristique *Premier Casier* décrite dans l'**algorithme 1**. Dans cet algorithme, le rangement $\mathcal{R}$ est modélisé par une liste de boîtes $\mathcal{L}$.

Algorithme 1 - Algorithme Premier Casier

Entrées : $n \in \mathbb{N}, (a_1 \cdots a_n) \in (\mathbb{Q} \cap]0, 1])^n$

Sorties : Une solution $(k, \mathcal{R})$

début

$\mathcal{L}$: liste de boîtes ouvertes

$\mathcal{L} = \emptyset$

pour $i=1$ à n **faire**

 Rechercher la première boîte de $\mathcal{L}$ dans laquelle l'objet i peut être placé

si une telle boîte existe **alors**

 l'objet i est placé à l'intérieur

sinon

 une nouvelle boîte est ajoutée à $\mathcal{L}$ et l'objet i y est placé

$k := \text{longueur}(\mathcal{L})$

 Dédurre $\mathcal{R}$ de $\mathcal{L}$.

Algorithme 2 - Algorithme Premier Casier Décroissant

Entrées : $n \in \mathbb{N}, (a_1 \cdots a_n) \in (\mathbb{Q} \cap]0, 1])^n$

Sorties : Une solution $(k, \mathcal{R})$

début

σ = permutation telle que $a_{\sigma(1)} \geq a_{\sigma(2)} \cdots \geq a_{\sigma(n)}$

$k, \mathcal{R}_1 := \text{Premier Casier}(n, a_{\sigma(1)}, \cdots, a_{\sigma(n)})$

pour $i = 1$ à n **faire**

$\mathcal{R}(i) := \mathcal{R}_1(\sigma(i))$

Nous allons montrer que l'algorithme *Premier Casier Décroissant* est une $\frac{3}{2}$ -approximation pour le problème MIN-BIN-PACKING.

On note dans la suite k le nombre de boîtes retournées par l'**algorithme 2**, k^* le nombre de boîtes optimal (défini comme le nombre minimal de boîtes permettant de répondre au problème MIN-BIN-PACKING) et $k_1 = \lceil 2k/3 \rceil$, où $\lceil \cdot \rceil$ désigne la partie entière supérieure. B_{k_1} fait référence à la boîte déduite du rangement renvoyé par l'**algorithme 2**.

Q8. Montrer que si B_{k_1} contient un objet i de taille $a_i > \frac{1}{2}$, alors $k^* \geq k_1 \geq \frac{2}{3}k$.

Q9. Montrer que sinon, les boîtes $B_{k_1} \cdots B_k$ contiennent au moins $2(k - k_1) + 1$ objets, aucun d'eux ne pouvant rentrer dans les boîtes $B_1 \cdots B_{k_1-1}$.

Q10. Justifier que $\sum_{i=1}^n a_i > \min(k_1 - 1, 2(k - k_1) + 1)$. En admettant que pour $k \in \mathbb{N}$, $\frac{2}{3}k + \frac{2}{3} \geq \lceil 2k/3 \rceil$,

en déduire que $k^* \geq \left\lceil \frac{2}{3}k \right\rceil \geq \frac{2}{3}k$.

On définit des types record pour représenter les objets et les boîtes en OCaml :

```
type objet = {
  id : int; (*identifiant de la boîte*)
  taille : float; (*taille de la boîte*)
}

type boîte = {
  charge : float; (*somme des tailles des objets de la boîte*)
  objets : objet list; (*liste courante des tailles des objets dans la boîte*)
}
```

Q11. Écrire une constante `boite_vide` : `boite` représentant une boîte vide.

Q12. Écrire une fonction de signature

`ajoute_boite` : `objet` -> `boite` -> `boite`

qui renvoie la boîte obtenue en ajoutant un objet à une boîte donnée.

Q13. Écrire une fonction récursive de signature

`trouve_boite` : `boite list` -> `objet` -> `int`

telle que `trouve_boite bl o` retourne l'indice (à partir de 0) de la première boîte dans `bl` pouvant contenir `o` ou la taille de la liste `bl` si aucune boîte ne peut contenir `o`.

Q14. Écrire une fonction récursive de signature

`transforme` : `'a` -> (`'a` -> `'a`) -> `int` -> `'a list` -> `'a list`

telle que `transforme d f i l` retourne la liste `l` où l'élément `x` à l'indice `i` (à partir de 0) a été remplacé par `f x`. Si `i` est plus grand que la taille de la liste, `f d` est ajouté à la fin.

Q15. Écrire une fonction de signature

`premier_casier` : `objet list` -> `boite list`

qui applique l'**algorithme 1**. On retournera directement la liste de boîtes, on ne cherchera pas à calculer la fonction $\mathcal{R}$. On pourra utiliser la fonction `transforme` précédemment écrite.

Q16. Écrire une fonction de signature

`premier_casier_decroissant` : `objet list` -> `boite list`

qui applique l'**algorithme 2**.

On pourra utiliser `List.sort` : (`'a` -> `'a` -> `int`) -> `'a list` -> `'a list` qui trie la liste passée en argument dans l'ordre croissant suivant la fonction passée en argument (qui renvoie -1, 0 ou 1 pour inférieur, égal ou supérieur respectivement). On pourra utiliser la fonction de comparaison générique `compare` : `'a` -> `'a` -> `int`. Là encore, on retournera directement la liste des boîtes.

Partie III - Algorithmes de couplage

Cette partie comporte des questions nécessitant un code en C.

On s'intéresse ici à un problème d'appariement entre deux groupes d'individus U et V , que l'on suppose de même cardinalité n .

On cherche à appairer les éléments de V aux éléments de U , chaque élément devant être apparié à exactement un élément de l'autre ensemble. Chaque élément de V (respectivement de U) a, de plus, un ordre de préférence total entre tous les éléments de U (respectivement de V).

Définition 1 (Couplage, couplage parfait)

Un ensemble de paires $A \subseteq V \times U$ est un couplage si tout $x \in V$ et tout $y \in U$ apparaissent dans au plus un élément de A . Le couplage est dit parfait si tout $x \in V$ et tout $y \in U$ apparaissent dans un et un seul élément de A .

Notations

Dans toute la suite on notera :

- $|E|$ le cardinal de l'ensemble E ,
- $>^x$ l'ordre associé à l'élément $x \in V \cup U$: si $u_1 \in U$ préfère strictement $v_1 \in V$ à $v_2 \in V$ alors $v_1 >^{u_1} v_2$. Par extension, on définit de même la notion de préférence $\geq^x$.
- $m_A(x)$ le partenaire de $x \in V \cup U$ dans le couplage parfait A .

Définition 2 (Couplage parfait stable)

Un couplage parfait A est dit *stable* si, pour tous couples (v_1, u_1) et (v_2, u_2) de A , il est impossible que l'on ait $u_2 >^{v_1} u_1$ et $v_1 >^{u_2} v_2$.

Définition 3 (Pareto-optimalité)

Soient A et A' deux couplages parfaits stables et $E \subseteq V \cup U$. On dit que A Pareto-domine A' sur E si et seulement si pour tout $x \in E$, $m_A(x) \geq^x m_{A'}(x)$ et il existe $x \in E$ tel que $m_A(x) >^x m_{A'}(x)$. Un couplage est Pareto-optimal s'il est stable et n'est pas Pareto dominé sur $V \cup U$.

En économie, la Pareto-optimalité d'un système exprime le fait que l'on ne peut améliorer la satisfaction d'un individu du système sans diminuer la satisfaction d'un autre.

III.1 - Recherche de couplages stables

L'algorithme de Gale–Shapley (**algorithme 3**) permet de montrer qu'il existe toujours au moins un couplage parfait stable et d'en trouver un de façon efficace. L'idée de l'algorithme est tout d'abord de choisir un des deux ensembles (ici pour illustration V). Les éléments de V font des propositions aux éléments de U . Quand un élément de V fait une proposition à un élément de U , celui-ci peut soit la refuser définitivement, soit l'accepter provisoirement en attendant une éventuelle meilleure offre. Toute nouvelle acceptation vaut rejet définitif des propositions précédemment acceptées.

Algorithme 3 - Algorithme de Gale-Shapley

Entrées : $n = |V| = |U|$, les listes de préférences des éléments de V et U

Sorties : Un couplage parfait stable A

début

$A := \emptyset$

tant que il existe un élément de V non apparié **faire**

Choisir un tel v

$u :=$ élément de U que v préfère parmi ceux à qui il n'a pas encore proposé

si u n'est pas apparié dans A **alors**

$A := A \cup \{(v, u)\}$

sinon

si u est apparié dans A à v' ET $v >^u v'$ **alors**

$A := A \setminus \{(v', u)\}$

$A := A \cup \{(v, u)\}$

retourner A

Soit le jeu de données $\mathcal{D}$ suivant : $V = \{v_1, v_2, v_3\}$, $U = \{u_1, u_2, u_3\}$ et :

V	U
$u_2 >^{v_1} u_1 >^{v_1} u_3$	$v_1 >^{u_1} v_3 >^{u_1} v_2$
$u_1 >^{v_2} u_2 >^{v_2} u_3$	$v_2 >^{u_2} v_1 >^{u_2} v_3$
$u_1 >^{v_3} u_2 >^{v_3} u_3$	$v_2 >^{u_3} v_1 >^{u_3} v_3$

Q17. Donner la trace de l'**algorithme 3** sur ces données. Le choix de v dans l'algorithme se fera par ordre croissant des indices.

Q18. Montrer que l'**algorithme 3** termine.

Q19. Donner, sans le prouver, un invariant de boucle permettant de prouver que l'**algorithme 3** retourne un couplage parfait.

Q20. Montrer que le couplage parfait calculé est stable.

Q21. Montrer que cet algorithme effectue moins de n^2 itérations de boucles.

Pour des préférences données, il peut exister plus d'un couplage parfait stable. Soient A et A' deux tels couplages pour un problème donné. On dira qu'un élément de $V \cup U$ *préfère* A à A' s'il n'a pas le même partenaire dans les deux couplages et s'il préfère celui de A à celui de A' .

Q22. Montrer que si v préfère le couplage A , alors son partenaire u dans A préfère A' .

On peut montrer qu'en même temps le partenaire u' de v dans A' préfère A .

La correction de l'algorithme de Gale-Shapley ne dépend pas de l'élément $v \in V$ qui fait sa proposition à chaque tour. En fait, on montre dans la suite que, quel que soit l'élément $v \in V$ non encore apparié qui fait une proposition à chaque tour, l'algorithme calcule toujours le même couplage parfait stable. Dans la suite, on dira qu'un élément $u \in U$ est *réalisable* pour un élément $v \in V$ s'il existe un couplage parfait stable qui contient (v, u) .

Q23. Montrer qu'au cours de l'**algorithme 3**, chaque $v \in V$ n'est rejeté que par les éléments $u \in U$ qui ne sont pas réalisables pour v .

Soit $M(v) \in U$ l'élément réalisable le mieux classé sur la liste des préférences de v et $P(u) \in V$ l'élément réalisable le moins bien classé sur la liste des préférences de u . On note enfin $A^* = \{(v, M(v)), v \in V\}$.

Q24. Montrer que lors de l'exécution de l'**algorithme 3** :

- 1) Tout $v \in V$, s'il est apparié, l'est avec $u \in U$ supérieur ou égal à $M(v)$ dans sa liste de préférences.
- 2) Tout $v \in V$, s'il n'est pas apparié, n'a fait des propositions qu'à des $u \in U$ strictement supérieurs à $M(v)$ dans sa liste de préférences.

Q25. En déduire que quel que soit l'élément $v \in V$ choisi dans la boucle Tant que de l'**algorithme 3**, l'algorithme termine en produisant l'ensemble A^* .

Ainsi, l'algorithme calcule le meilleur couplage parfait stable possible du point de vue des éléments de V . Il s'avère également que l'algorithme de Gale-Shapley fait correspondre u avec $P(u)$, pour tout $u \in U$. L'algorithme avantage donc de manière claire le groupe ayant l'initiative (ici V) : le couplage stable produit est dit *Pareto-V-optimal* en ce sens qu'il n'est pas Pareto-dominé sur V .

III.2 - Couplages Pareto-optimaux

Dans cette sous-partie, on utilise également le jeu de données $\mathcal{D}$.

L'algorithme de Gale-Shapley ne retourne pas nécessairement un couplage Pareto-optimal. On s'intéresse ici à un algorithme susceptible de produire cette propriété.

Pour construire un couplage Pareto-optimal, on représente les données à l'aide d'un graphe orienté $\mathcal{G} = (S, E)$ où :

- les sommets S sont les éléments de $V \cup U$,
- les arcs E représentent les premiers choix : $(v, u) \in E$ si et seulement si u est le premier choix de v dans sa liste de préférences et $(u, v) \in E$ si et seulement si v est le premier choix de u .

Q26. Montrer que $\mathcal{G}$ contient au moins un circuit, c'est-à-dire un chemin dont les deux sommets extrémités sont identiques.

Q27. Justifier par l'absurde que tout $x \in V \cup U$ est dans au plus un circuit.

On propose alors l'**algorithme 4** pour calculer un couplage, dont on admettra qu'il est Pareto-optimal pour V .

Algorithme 4 - Algorithme de couplage par graphe orienté

Entrées : $n = |V| = |U|$, les listes de préférences des éléments de V et U

Sorties : Un couplage A

début

$A := \emptyset$

tant que *il existe un élément de V non apparié* **faire**

 Construire le graphe $\mathcal{G}$ sur les données courantes

pour chaque *arc (v, u) présent dans un circuit de $\mathcal{G}$* **faire**

$A = A \cup \{(v, u)\}$

 Supprimer du problème tous les $v \in V$ et les $u \in U$ appariés

 Mettre à jour les listes de préférences des individus restants

retourner A

Q28. Donner les graphes et la construction itérative de A produits par l'**algorithme 4** sur $\mathcal{D}$.

Q29. En utilisant les données $\mathcal{D}$, montrer que l'**algorithme 4** ne produit pas toujours un couplage stable.

On s'intéresse alors à une autre forme de stabilité pour assurer à la fois une propriété de Pareto et une stabilité dans les couplages.

Définition 4 (P-stabilité)

Un couplage A est P-stable s'il n'existe aucune paire $(v, v') \in V^2$ telle que v préfère $m_A(v')$ et v' préfère $m_A(v)$.

Q30. Montrer par l'absurde que l'**algorithme 4** produit un couplage P-stable.

Q31. Montrer que l'**algorithme 3** ne produit pas nécessairement un couplage P-stable.

III.3 - Implémentation

Dans la suite, les individus sont numérotés de 0 à $n - 1$. Ainsi $V = \{0, \dots, n - 1\}$ et $U = \{0, \dots, n - 1\}$.

On suppose disposer de deux matrices de taille $n \times n$ L_v et L_u donnant respectivement les listes de préférences des éléments de V et U : ainsi, $L_u[i][j]$ est le j -ème élément de V préféré par l'élément $i \in U$.

On définit ces tableaux en C par l'intermédiaire du code suivant :

```
/* La directive #define est utilisée pour définir une valeur pour la constante n qui
   servira à déclarer des tableaux de taille fixe. */
#define n 4

int main(void) {
    int Lv[n][n], Lu[n][n];
    (...)
}
```

III.3.1 - Algorithme de Gale-Shapley

Pour connaître rapidement le classement des éléments de V dans les listes de préférences des éléments de U , on définit un tableau $Rang$ de taille $n \times n$, tel que $Rang[i][j]$ donne le rang de $j \in V$ dans la liste de préférences de l'élément $i \in U$. Par convention, les rangs commencent à 0.

Q32. Écrire une fonction de prototype

void calcule_rang(int Lu[n][n], int Rang[n][n])

qui construit le tableau $Rang$.

Afin de pouvoir ajouter ou supprimer rapidement des couples (v, u) au couplage parfait stable A , on définit deux tableaux `CoupleV` et `CoupleU` tels que `CoupleV[i]` ou `CoupleU[i]` est le partenaire actuel de i . On définit enfin un dernier tableau `USuiv` de taille n stockant pour chaque élément de V le rang du prochain élément de U à qui il peut faire une proposition.

Q33. Proposer des valeurs d'initialisation pour ces trois tableaux.

Q34. Avec ces structures de données, écrire une fonction de prototype

```
void gale_shapley(int Lv[n][n], int Lu[n][n], int CoupleU[n], int CoupleV[n])
```

qui réalise l'**algorithme 3**. On ne retournera pas le couplage parfait stable, on remplira seulement les variables `CoupleU` et `CoupleV` passées en argument. Pour le choix de v , on prendra les éléments de V par ordre croissant de leur numéro en choisissant à chaque fois le premier libre.

III.3.2 - Algorithme de couplage par graphe orienté

Sans explicitement coder tout l'**algorithme 4** qui requiert de rechercher les circuits dans $\mathcal{G}$, on se propose de coder quelques fonctions utiles à la réalisation de l'algorithme.

Pour pouvoir numéroter les sommets du graphe de manière continue, les individus sont maintenant numérotés de 0 à $2n - 1$ de la manière suivante : $V = \{0, \dots, n - 1\}$ et $U = \{n, \dots, 2n - 1\}$.

On code le graphe orienté $\mathcal{G}$ par listes d'adjacence. On propose les types suivants :

```
struct noeud {
    int individu;           // Element de U union V
    struct noeud* suivant;
};
typedef struct noeud *liste;

struct graphe_s {
    int nb_sommets;
    liste *liste_adjacence;
};
typedef struct graphe_s graphe;
```

Q35. Écrire une fonction de prototype

```
void ajoute_arc(graphe *g, int i, int j)
```

qui ajoute l'arc (i, j) au graphe $\mathcal{G}$.

Q36. Écrire une fonction de prototype

```
void supprime_arc(graphe *g, int i, int j)
```

qui supprime l'arc (i, j) du graphe $\mathcal{G}$.

Q37. Écrire une fonction de prototype

```
void supprime_sommet(graphe *g, int s)
```

qui supprime le sommet s du graphe $\mathcal{G}$. Pour ce faire, on supprimera uniquement tous les arcs dont ce sommet est origine ou destination.

Q38. Écrire une fonction de prototype

```
graphe *construit_graphe(int Lv[n][n], int Lu[n][n])
```

qui construit le graphe $\mathcal{G}$ utilisé dans l'**algorithme 4** et dont la définition est donnée juste avant la **Q26**. On prendra bien garde à la numérotation des éléments de U .

FIN

SESSION 2025



MP7IN

ÉPREUVE MUTUALISÉE AVEC E3A-POLYTECH**ÉPREUVE SPÉCIFIQUE - FILIÈRE MP****INFORMATIQUE****Durée : 4 heures**

N.B. : le candidat attachera la plus grande importance à la clarté, à la précision et à la concision de la rédaction. Si un candidat est amené à repérer ce qui peut lui sembler être une erreur d'énoncé, il le signalera sur sa copie et devra poursuivre sa composition en expliquant les raisons des initiatives qu'il a été amené à prendre.

RAPPEL DES CONSIGNES

- Utiliser uniquement un stylo noir ou bleu foncé non effaçable pour la rédaction de votre composition ; d'autres couleurs, excepté le vert, bleu clair ou turquoise, peuvent être utilisées, mais exclusivement pour les schémas et la mise en évidence des résultats.
- Ne pas utiliser de correcteur.
- Écrire le mot FIN à la fin de votre composition.

Les calculatrices sont interdites.

Le sujet est constitué d'un unique problème et comporte cinq parties qui ne sont pas complètement indépendantes.

Répartition de la gestion d'un réseau minimisant la bande passante

Le Listenbourg (**figure 1**) est une république fictive de la péninsule ibérique, comportant environ 66 millions d'habitants, née sur les réseaux sociaux en octobre 2022. Au Listenbourg deux fournisseurs d'accès, MaxDébit et MinLatence, cherchent à se partager la gestion d'un tout nouveau réseau : l'Ultraneet.

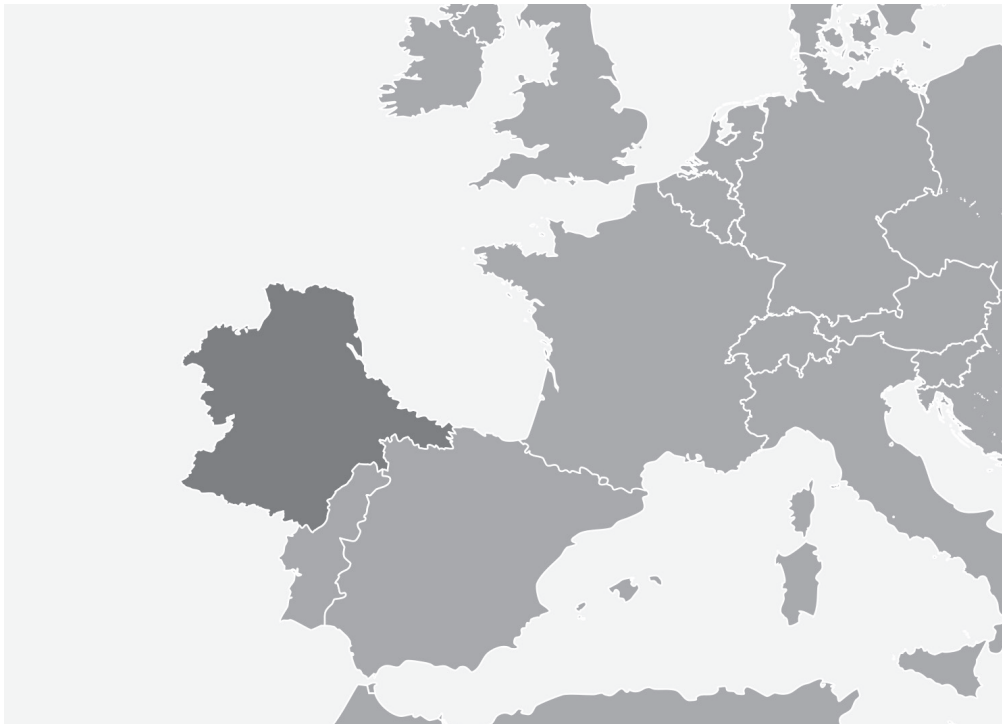


Figure 1 - La république imaginaire du Listenbourg

Le graphe simplifié du réseau Ultraneet de ce pays est représenté à la **figure 2**. Le nom des villes correspondant aux identifiants des sommets est enregistré dans la base de données (**figure 3**). Une arête relie deux villes lorsqu'un câblage physique direct est établi entre elles et la bande passante permise par ce câblage est indiquée comme poids. Il y a au plus une arête entre deux villes. La structure du réseau Ultraneet a été intégralement terminée lors du grand plan « Réseau pour Tous » mené par le premier ministre Petro Sank-Ærixh et il s'agit désormais d'en confier l'exploitation aux deux fournisseurs d'accès concurrents.

Dans ce sujet, on s'intéresse à une procédure de partage de la gestion du réseau entre les deux opérateurs. On suppose qu'une liaison entre deux villes ne peut être exploitée que par l'un ou par l'autre des deux fournisseurs d'accès qui en aura alors l'usage exclusif. Le gouvernement du Listenbourg ne souhaite pas s'embarrasser avec une procédure complexe d'appel d'offre et impose la procédure simple suivante pour répartir l'exploitation exclusive des liaisons physiques du réseau :

- tant qu'il reste des liaisons à attribuer, chaque opérateur, à tour de rôle et en commençant par MaxDébit, choisit parmi les liaisons restantes une liaison qu'il souhaite exploiter exclusivement.

Une fois la procédure de répartition terminée, c'est-à-dire une fois que toutes les liaisons ont été attribuées, chaque fournisseur d'accès dispose de son propre sous-réseau exclusif, dont la gestion lui revient entièrement. L'objectif d'un fournisseur d'accès est d'obtenir un sous-réseau permettant de proposer la meilleure bande passante possible entre deux villes quelconques du pays.

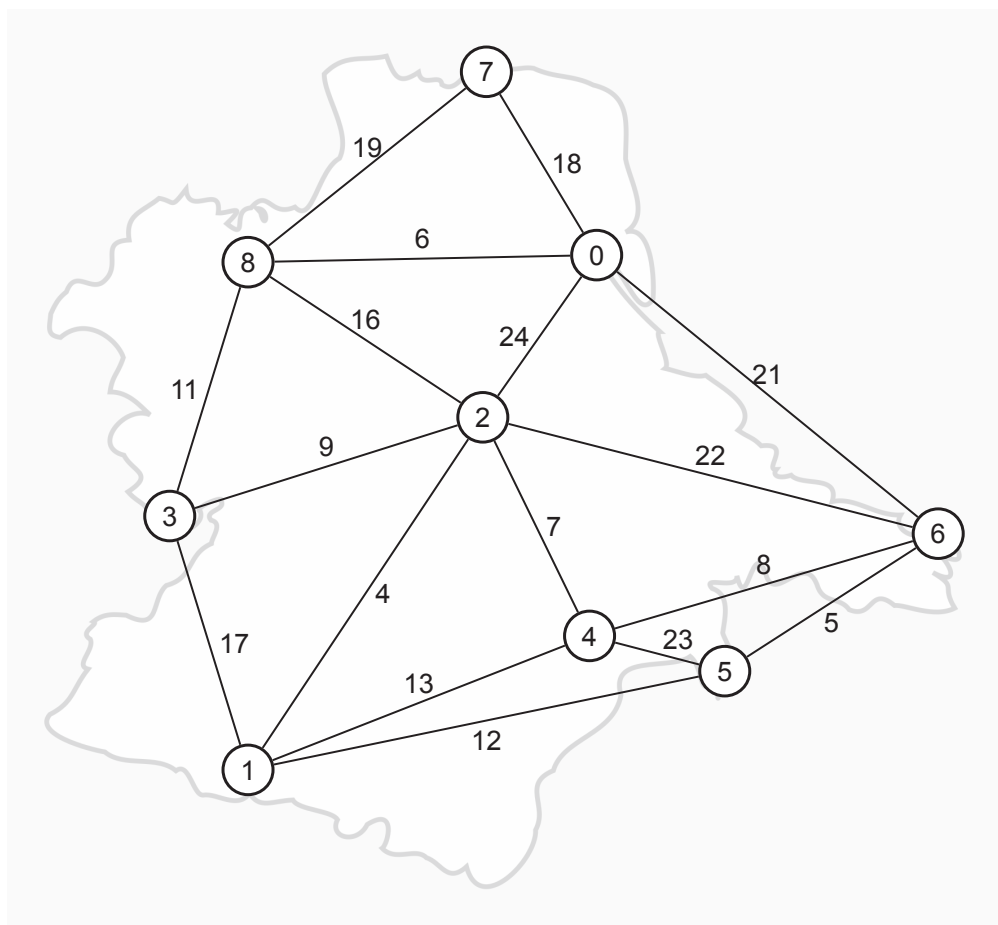


Figure 2 - Réseau simplifié du Listenbourg

La première partie du sujet s'intéresse à l'étude d'une base de données du réseau mise à disposition des fournisseurs d'accès (informatique de tronc commun, langage SQL). La deuxième partie introduit et étudie quelques propriétés de l'arbre couvrant de poids maximal (option informatique). La troisième partie s'intéresse à la recherche de l'arbre couvrant de poids maximal en utilisant l'algorithme de Borůvka (option informatique, langage OCaml). La quatrième partie se propose de montrer que la bande passante limite d'un réseau correspond au poids de l'arête de poids minimal de l'arbre couvrant de poids maximal du graphe (option informatique). La cinquième partie propose de modéliser la procédure de répartition de la gestion du réseau comme un jeu à deux joueurs (informatique de tronc commun, langage Python).

Les cinq parties sont essentiellement indépendantes dans leur traitement mais des notions utiles à la résolution d'une partie peuvent être introduites dans les parties précédentes. La **partie I** est indépendante des quatre autres. La **partie II** introduit des notions utiles à la résolution des **parties III** et **IV**. La **partie IV** introduit des concepts utilisés dans la **partie V**.

Dans tout le sujet, il est toujours admis d'utiliser un résultat ou un programme correspondant à une question précédente, même si cette question n'a pas été résolue.

Partie I - Base de données du réseau

Le ministère des affaires environnementales, des ressources, de l'agriculture et des forêts du Listenbourg met à disposition des deux fournisseurs d'accès une base de données relationnelle. Celle-ci comporte deux tables dont le schéma est le suivant :

- **villes**(id : *entier*, nom : *texte*, pop : *flottant*)
- **liaisons**(id1 : *entier*, id2 : *entier*, bp : *flottant*)

Un enregistrement de la table **villes** comporte l'identifiant unique d'une ville (id), le nom de cette ville (nom) et sa population en millions d'habitants (pop). *A priori*, plusieurs villes du Listenbourg peuvent porter le même nom. Un enregistrement de la table **liaisons** correspond à une liaison physique directe entre deux villes données par leur identifiant (id1, id2) ainsi que la bande passante en $Tb.s^{-1}$ correspondant à ce câblage (bp). On garantit que dans la représentation d'une liaison, l'identifiant de la première ville est toujours strictement inférieur à celui de la deuxième ville. Rappelons qu'il ne peut exister qu'au plus une liaison entre deux villes.

Un exemple simplifié du contenu de cette base de données, correspondant au graphe simplifié du réseau du Listenbourg de la **figure 2**, est donné **figure 3**.

villes			liaisons		
id	nom	pop	id1	id2	bp
0	Lurenberg	12.0	0	2	24.0
1	Veroja	10.0	0	6	21.0
2	Aschlöss	8.0	0	7	18.0
3	Stratord	5.2	0	8	6.0
4	Gossard	5.1	1	2	4.0
5	La Galinera	5.0	1	3	17.0
6	Sainte Marie	5.0	1	4	13.0
7	Atlanitkischer	4.7	1	5	12.0
8	Gasparländ	3.5	2	3	9.0
			2	4	7.0
			2	6	22.0
			2	8	16.0
			3	8	11.0
			4	5	23.0
			4	6	8.0
			5	6	5.0
			7	8	19.0

Figure 3 - Contenu de la base de données correspondant au réseau simplifié de la **figure 2**

- Q1.** Au vu de la situation modélisée, proposer des clés primaires pour les tables **villes** et **liaisons**, en justifiant succinctement. Identifier les clés étrangères présentes dans ce schéma relationnel.
- Q2.** Pour chacune des questions suivantes on demande d'écrire une requête SQL portant sur le schéma relationnel proposé qui fonctionnerait quel que soit le contenu de la base de données et pas uniquement sur l'exemple de contenu proposé à la **figure 3**.
- (a) Écrire une requête SQL qui donne les noms des villes comportant strictement plus de cinq millions d'habitants ;
 - (b) Écrire une requête SQL qui donne la bande passante moyenne des liaisons incidentes à la ville d'identifiant 2 ;
 - (c) Écrire une requête SQL qui donne les noms des deux villes reliées par la liaison de bande passante maximale. *On suppose que toutes les liaisons ont une bande passante différente.*
- Q3.** Déterminer le résultat de la requête SQL suivante sur l'exemple de contenu de la base de données représentée à la **figure 3**.

```
SELECT nom, COUNT(*) AS d
FROM villes
  JOIN (SELECT id1 AS x, id2 FROM liaisons UNION SELECT id2 AS x, id1 FROM liaisons)
      ON id = x
GROUP BY id
HAVING d >= 4
ORDER BY id ASC
```

Partie II - Arbre couvrant de poids maximal

II.1 - Acyclicité, connexité et arbres

Un *graphe* (non orienté) est un couple $G = (S, A)$ formé d'un ensemble S fini non vide de *sommets* et d'un ensemble A d'*arêtes* constitué de parties de S de cardinal 2. On note $|S|$ le nombre de sommets de G et $|A|$ le nombre d'arêtes de G . Les voisins d'un sommet $x \in S$ sont notés $\mathcal{V}(x)$ et leur nombre $d(x) = |\mathcal{V}(x)|$ est le *degré* de x .

Un chemin dans un graphe est une suite $c = x_0 x_1 \dots x_n$ de sommets avec $\forall i \in \llbracket 0, n-1 \rrbracket, \{x_i, x_{i+1}\} \in A$. La longueur $|c|$ d'un tel chemin est $n \in \mathbb{N}$. Un chemin est *élémentaire* si tous les sommets empruntés par ce chemin sont deux à deux distincts. Un chemin est *simple* si toutes les arêtes empruntées par ce chemin sont deux à deux distinctes.

Un *cycle* est un chemin simple $x_0 x_1 \dots x_n$ avec $n \geq 3$ et $x_0 = x_n$. Un graphe est *acyclique* s'il ne comporte pas de cycle.

Un graphe est *connexe* si deux sommets quelconques sont toujours reliés par un chemin. Une *composante connexe* est un ensemble $C \subseteq S$ de sommets dont tous les couples de sommets sont reliés par au moins un chemin.

Un *arbre* est un graphe connexe acyclique.

Si $a = \{x, y\}$ est une partie de S de cardinal 2, on note $G + a$ le graphe $(S, A \cup \{a\})$, c'est-à-dire le graphe G auquel on a ajouté l'arête a et $G - a$ le graphe $(S, A \setminus \{a\})$, c'est-à-dire le graphe G auquel on a retiré l'arête a .

Q4. Montrer que si $G = (S, A)$ est un graphe connexe et que si $a = \{x, y\} \in A$ est une arête de G appartenant à un cycle de G , alors le graphe $G - a$ est encore connexe.

On admet la proposition suivante :

Proposition 1 : relation entre $|S|$ et $|A|$ pour les graphes connexes ou acycliques

Soit $G = (S, A)$ un graphe :

- (a) si G est connexe alors $|A| \geq |S| - 1$;
- (b) si G est acyclique alors $|A| \leq |S| - 1$.

Q5. Montrer que si $G = (S, A)$ est connexe et que $|A| = |S| - 1$ alors G est un arbre.

II.2 - Sous-graphe et arbre couvrant

Dans tout ce sujet un *sous-graphe* d'un graphe G est un graphe $G' = (S, A')$ avec $A' \subseteq A$. Notons qu'un sous-graphe de G comporte exactement les mêmes sommets que G . On identifie un sous-graphe $G' = (S, A')$ avec le sous-ensemble d'arêtes $A' \subseteq A$. Un *arbre couvrant* de G est un sous-graphe de G qui est un arbre.

Q6. Montrer qu'un graphe $G = (S, A)$ est connexe si et seulement s'il admet un arbre couvrant.

Q7. Soit $T = (S, A')$ un arbre couvrant de G avec $A' \subseteq A$ et $a \in A \setminus A'$.

- (a) Justifier que $T + a$ n'est pas acyclique.
- (b) Soit $a' \in A'$ une arête d'un cycle de $T + a$. Montrer que $T + a - a'$ est un arbre couvrant de G .

II.3 - Graphes pondérés et arbre couvrant de poids maximal

Un *graphe pondéré* est un triplet (S, A, p) où (S, A) est un graphe et $p : A \rightarrow \mathbb{R}$ une fonction de pondération des arêtes. Si $\{x, y\} \in A$ est une arête, $p(\{x, y\})$ est appelé le *poids* de cette arête. Le poids d'un graphe pondéré est la somme des poids de ses arêtes :

$$p(G) = \sum_{a \in A} p(a).$$

On étend la notion de sous-graphe aux graphes pondérés de la manière suivante : un sous-graphe d'un graphe pondéré $G = (S, A, p)$ est un graphe pondéré $G' = (S, A', p_{A'})$ où $A' \subseteq A$ et où $p_{A'}$ est la restriction de p à A' . On se permettra de ne pas systématiquement indiquer la fonction de pondération.

Le plan « Réseau pour Tous » du gouvernement a permis d'interconnecter toutes les villes du Listebourg. On considérera donc en général un graphe pondéré $G = (S, A, p)$ non orienté *connexe*, avec la fonction de pondération correspondant à la bande passante associée à chaque liaison. Chaque province ayant opté pour son propre câblage, deux villes ne sont jamais reliées par le même type de câblage et la bande passante de deux liaisons est *toujours* différente. Ceci se traduit donc par une fonction de pondération p injective : dans toute la suite du sujet, les poids des arêtes de G sont toujours deux à deux distincts.

On se propose de démontrer la **proposition 2** suivante :

Proposition 2 : existence et unicité de l'arbre couvrant de poids maximal

Soit $G = (S, A, p)$ un graphe *connexe* muni d'une fonction de pondération *injective*. Alors, il existe un unique arbre couvrant de poids maximal de G . On note $T^* = (S, A^*)$ cet arbre couvrant.

Q8. Montrer l'existence d'un arbre couvrant de poids maximal de G .

- Q9.** Pour montrer l'unicité, on se propose de procéder par l'absurde. Supposons l'existence de deux arbres couvrants de poids maximal $T_1 = (S, A_1)$ et $T_2 = (S, A_2)$ différents. Considérons une arête de poids maximal parmi les arêtes qui appartiennent à un seul des deux arbres, c'est-à-dire une arête de poids maximal dans $(A_1 \setminus A_2) \cup (A_2 \setminus A_1)$, qui est bien non vide puisque $A_1 \neq A_2$. Sans perte de généralité, on peut noter a_2 une telle arête et supposer que $a_2 \in A_2 \setminus A_1$.
- (a) Montrer qu'il existe une arête $a_1 \in A_1 \setminus A_2$ sur un cycle de $T_1 + a_2$.
- (b) Conclure en considérant le sous-graphe $T_1 + a_2 - a_1$.

Partie III - Recherche d'un arbre couvrant de poids maximal

III.1 - Rappels et fonctions élémentaires en OCaml

On rappelle que, dans le langage OCaml :

- on peut représenter des tableaux par le type `'a array` ;
- si `e` est une expression de type `'a`, l'expression `Array.make n e` permet de créer un tableau de type `'a array` de $n \in \mathbb{N}$ cases dont toutes les cases contiennent la valeur de l'expression `e` ;

```
Array.make : int -> 'a -> 'a array
```

- si `a` est un tableau de type `'a array` de taille $n \in \mathbb{N}$, l'expression `Array.length a` permet d'obtenir la valeur `n` ;

```
Array.length : 'a array -> int
```

- si $0 \leq i < n$, l'expression `a.(i)` s'évalue en la valeur contenue à la case `i` du tableau `a` et si `e` est une expression de type `'a`, l'expression `a.(i) <- e` permet de remplacer la valeur contenue à la case `i` du tableau `a` par la valeur de l'expression `e` ;
- la fonction `List.iter` a le comportement suivant : si `f` est une fonction de type `'a -> unit`, alors l'expression `List.iter f [a1; a2; ...; an]` est équivalente à `f a1; f a2; ...; f an`

```
List.iter : ('a -> unit) -> 'a list -> unit
```

- une boucle `for i = a to b do ... done` permet de faire évoluer une variable `i` entre les valeurs entières `a` et `b` **incluses** ;
 - il existe deux fonctions `max` et `min` de type `'a -> 'a -> 'a`.
- Q10.** Écrire une fonction `max_tab : int array -> int` telle que, pour un tableau d'entiers `a` de taille $n \geq 1$, `max_tab a` renvoie la valeur du plus grand entier contenu dans `a`. On attend une complexité linéaire en n , ce que l'on justifiera en une phrase.

III.2 - Représentation des graphes en OCaml

On représente un graphe pondéré $G = (S, A, p)$, avec $S = \llbracket 0, n-1 \rrbracket$ et $n \in \mathbb{N}^*$ par un tableau de listes d'adjacence pondérées.

```
type graphe = (int * float) list array
```

Si g représente un graphe et si $0 \leq i < n$ est un sommet, alors $g.(i)$ contient la liste des couples $\left\{ (j, p(\{i, j\})) \right\}_{j \in V(i)}$, c'est-à-dire la liste des couples de la forme (j, w) avec $w = p(\{i, j\})$ et j voisin de i .

Remarquons que puisque les graphes sont non orientés, si i est dans la liste d'adjacence de j , alors j est dans la liste d'adjacence de i avec le même poids.

Par exemple, si g est le graphe de la **figure 2**, on a $g.(0)$ qui vaut par exemple :

```
[(2, 24.0); (6, 21.0); (7, 18.0); (8, 6.0)]
```

Le nombre de sommets du graphe correspond donc à la taille du tableau de listes d'adjacences :

```
let nb_sommets g = Array.length g
```

Q11. Donner une valeur possible pour $g.(7)$.

On représente une arête par un triplet (w, i, j) où $w = p(\{i, j\})$ et où $i < j$. Remarquons que chaque arête est ainsi représentée exactement une fois et qu'une arête commence par son poids.

Q12. Écrire en OCaml une fonction `ajoute_arete : graphe -> (float * int * int) -> unit` telle que `ajoute_arete g (w, i, j)` permet d'ajouter au graphe g l'arête (w, i, j) . On suppose que cette arête n'était pas déjà présente et la complexité doit être en temps constant.

Q13. Écrire en OCaml une fonction `toutes_les_aretes : graphe -> (float * int * int) list` qui permet d'obtenir la liste des arêtes d'un graphe avec la convention ci-dessus. On attend une complexité linéaire en $|S| + |A|$, ce que l'on justifiera rapidement.

Si a_1 et a_2 sont deux arêtes de type `float * int * int`, l'expression $a_1 < a_2$ compare les arêtes pour l'ordre lexicographique des triplets donc pour la valeur du poids de l'arête en premier. Comme dans ce sujet on suppose toujours que tous les poids sont deux à deux distincts, on peut directement utiliser l'ordre OCaml `<` sur les arêtes pour comparer les poids des arêtes. Ainsi par exemple, `min a1 a2` s'évalue en l'arête de poids minimal entre a_1 et a_2 .

Q14. Écrire une fonction `min_arete : (float * int * int) list -> (float * int * int)` de manière récursive en OCaml qui renvoie l'arête de poids minimal d'une liste d'arêtes supposée non vide. Quelle est sa complexité ?

III.3 - Recherche des composantes connexes

Pour un graphe $G = (S, A, p)$ avec $S = \llbracket 0, n-1 \rrbracket$, on cherche à construire un tableau c de taille n représentant les composantes connexes du graphe. Les composantes connexes sont indexées entre 0 et $p-1$ où p est le nombre de composantes connexes du graphe. Pour $0 \leq i < n$ la case $c.(i)$ comporte l'indice k de la composante connexe du sommet i (ou la valeur -1 si cette composante n'a pas encore été déterminée).

On considère la fonction `explorer : graphe -> int array -> int -> int -> unit` suivante pour un graphe g , un tableau c de taille n , un identifiant de composante connexe $0 \leq k < p$ et un sommet de départ s , vérifiant la convention suivante : « un sommet $0 \leq i < n$ a été visité si et seulement si $c.(i) \geq 0$ ».

```

1 let rec explorer g c k s =
2   if c.(s) < 0 then begin
3     c.(s) <- k;
4     List.iter (fun (v, _) -> explorer g c k v) g.(s)
5   end

```

Q15. Décrire précisément ce que réalise un appel à `explorer g c k s`.

Q16. Écrire en OCaml une fonction `composantes_connexes : graphe -> int array` telle que, pour un graphe g , `composantes_connexes g` s'évalue en le tableau c des indices des composantes connexes. Cette fonction doit avoir une complexité en $O(|S| + |A|)$ que l'on ne demande pas de justifier.

Q17. En déduire une fonction `est_connexe : graphe -> bool` telle que `est_connexe g` s'évalue à `true` si le graphe g est connexe et à `false` sinon.

III.4 - Algorithme de Borůvka

Dans toute cette sous-partie, on considère un graphe $G = (S, A, p)$ connexe avec une fonction de pondération p injective. On note $T^* = (S, A^*)$ l'unique arbre couvrant de poids maximal de G .

Considérons $H = (S, B)$ un sous-graphe acyclique de G , avec donc $B \subseteq A$. On considère $C \subseteq S$ une composante connexe de H . Une arête **sûre** pour C est une arête de A ayant exactement une extrémité dans C dont le poids est maximal parmi les arêtes qui ont exactement une extrémité dans C .

Par exemple, pour le sous-graphe H de la **figure 4** et pour la composante connexe $\{1, 4, 5, 6\}$ l'arête de poids 22 est sûre (maximale parmi les arêtes de poids 4, 7, 17, 21 et 22 qui sont celles ayant exactement une extrémité dans cette composante). Sauf si H est connexe, chaque composante connexe de H comporte exactement une arête sûre. Une même arête peut être sûre pour deux composantes connexes C et C' . Dans l'exemple de la **figure 4**, l'arête de poids 22 est également l'arête sûre pour la composante connexe $\{0, 2\}$.

Q18. Déterminer l'arête sûre pour la composante $\{3\}$ ainsi que celle pour la composante $\{7, 8\}$.

Q19. On suppose dans cette question que $H = (S, B)$ est un sous-graphe acyclique de l'arbre couvrant $T^* = (S, A^*)$ de poids maximal de G , et donc que $B \subseteq A^*$. Montrer que A^* contient toutes les arêtes sûres des composantes connexes de H .

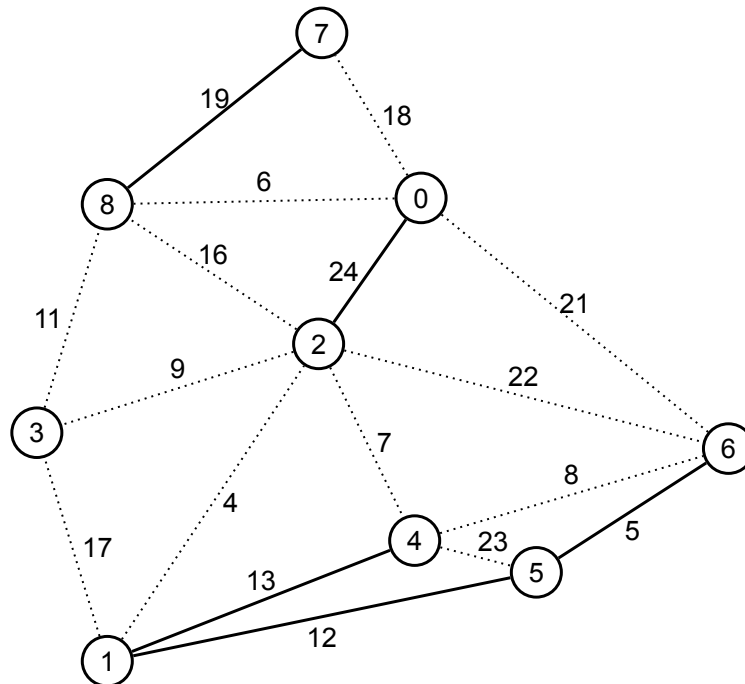


Figure 4 - Un sous-graphe H du graphe de la **figure 2**, comportant quatre composantes connexes. Les arêtes qui n'appartiennent pas au sous-graphe sont représentées en pointillés

Algorithme 1 : algorithme de Borůvka

Entrée : Graphe $G = (S, A, p)$ pondéré injectivement et connexe

Sortie : $H = (S, B)$ l'arbre couvrant de poids maximal de G

```

1  $H = (S, \emptyset)$ ;
2 tant que  $H$  n'est pas connexe faire
3   | ajouter à  $H$  toutes les arêtes sûres pour les composantes connexes de  $H$ ;
4 fin

```

- Q20.** Donner le déroulé de l'**algorithme 1** sur le graphe G de la **figure 2**. Indiquer à chaque étape et pour chaque composante connexe son arête sûre et indiquer l'ensemble des arêtes sûres ajoutées à chaque étape.
- Q21.** Montrer que l'algorithme de Borůvka termine.
- Q22.** Montrer que l'algorithme de Borůvka renvoie l'arbre couvrant de poids maximal $T^* = (S, A^*)$ de G .

On considère la fonction suivante qui prend en entrée un graphe g représentant le graphe $G = (S, A, p)$ connexe pondéré et un graphe h représentant un sous-graphe acyclique $H = (S, B)$ de G supposé non encore connexe.

```

1  let aretes_sures g h =
2    let c = composantes_connexes h in
3    let p = max_tab c + 1 in
4    let aretes = toutes_les_aretes g in
5    let sures = Array.make p (min_arete aretes) in
6    let traite a =
7      let (_, i, j) = a in
8      if c.(i) <> c.(j) then begin
9        sures.(c.(i)) <- max a sures.(c.(i));
10       sures.(c.(j)) <- max a sures.(c.(j));
11      end
12    in
13    List.iter traite aretes;
14    sures

```

Q23. Déterminer le type de cette fonction. Expliquer ce que réalise cette fonction et préciser la structure et le contenu de la valeur renvoyée. Déterminer sa complexité en fonction de $|A|$ en justifiant. *On remarquera que d'après la **proposition 1**, puisque le graphe G est connexe, on a $|S| - 1 \leq |A|$ et donc qu'un $O(|S|)$ est un $O(|A|)$.*

On suppose disposer d'une fonction :

```
ajoute_aretes_sures : graphe -> (float * int * int) array -> unit
```

telle que si h représente un sous-graphe acyclique de G et que $sures$ est la valeur renvoyée par `aretes_sures g h` alors `ajoute_aretes_sures h sures` ajoute à h toutes les arêtes contenues dans $sures$. On garantit que cette fonction est en complexité $O(|A|)$ et qu'elle n'ajoute qu'une seule fois une même arête à h même si cette arête est sûre pour deux composantes connexes différentes.

Q24. Écrire en OCaml une fonction `borukva : graphe -> graphe` telle que `borukva g` sur un graphe g s'évalue en un sous-graphe h de G correspondant à son arbre couvrant de poids maximal.

Q25. On s'intéresse maintenant à la complexité temporelle de cet algorithme.

- (a) Montrer qu'à chaque passage dans la boucle **tant que** à la ligne 2 de l'**algorithme 1**, le nombre de composantes connexes de H est divisé au moins par 2.
- (b) En déduire la complexité temporelle dans le pire cas de l'algorithme de Borůvka.
- (c) Quelle est la complexité temporelle de cet algorithme dans le meilleur cas ?

Partie IV - Chemin de bande passante maximale

Dans cette partie, on considère un graphe pondéré $G = (S, A, p)$, non nécessairement connexe, mais dont la fonction de pondération est injective.

On définit la *bande passante* d'un chemin $c = x_0x_1 \dots x_n$ comme la quantité :

$$\bar{b}(c) = \min_{0 \leq i \leq n-1} p(\{x_i, x_{i+1}\})$$

c'est-à-dire le poids minimal d'une arête de ce chemin. En effet, la bande passante d'un chemin est limitée par la plus petite bande passante des arêtes de ce chemin. La bande passante d'un chemin de longueur nulle est $+\infty$.

Pour deux sommets $x, y \in S$, un *chemin de bande passante maximale* entre x et y est un chemin c qui relie x à y dont la bande passante est maximale parmi tous les chemins entre x et y . Un chemin de bande passante maximale est ainsi un chemin permettant de maximiser la bande passante entre deux villes, ce que cherchent à réaliser les opérateurs MaxDébit et MinLatence. On note :

$$\bar{b}_{\max}(x, y) = \max \{ \bar{b}(c) \mid c \text{ chemin de } x \text{ à } y \}$$

la bande passante d'un chemin de bande passante maximale de x à y . On a $\bar{b}_{\max}(x, y) = -\infty$ si x et y ne sont pas reliés par au moins un chemin. Notons que cette quantité est bien définie, l'ensemble des arêtes étant fini, il y a un nombre fini de valeurs possibles pour la bande passante d'un chemin de x à y .

Q26. Déterminer un chemin de bande passante maximale entre Lurenberg (d'identifiant 0) et Veroja (d'identifiant 1) sur le graphe de la **figure 2**. Quelle est sa bande passante ?

On admet que dans un arbre il existe un unique chemin élémentaire entre tout couple de sommets.

Q27. Soient $x, y \in S$, montrer que l'unique chemin de x à y dans l'arbre couvrant de poids maximal T^* est un chemin de bande passante maximale entre x et y .

Un fournisseur d'accès cherche à pouvoir offrir un chemin avec la bande passante la plus grande possible entre tous les couples de villes possibles. Plus précisément, il s'intéresse à la quantité :

$$\bar{b}_{\lim}(G) = \min \{ \bar{b}_{\max}(x, y) \mid x, y \in S \}$$

appelée la *bande passante limite* d'un graphe G .

Q28. Justifier que

$$\bar{b}_{\lim}(G) = \begin{cases} -\infty & \text{si } G \text{ n'est pas connexe} \\ \min \{ p(a) \mid a \in A^* \} & \text{avec } T^* = (S, A^*) \text{ l'arbre couvrant de poids maximal de } G \text{ sinon.} \end{cases}$$

Partie V - Jeu sur un graphe

Dans cette partie, on suppose que $G = (S, A, p)$ est un graphe pondéré connexe muni d'une fonction de pondération injective.

V.1 - Procédure de partage vue comme un jeu

On peut voir la procédure de partage du réseau décrite dans l'introduction du sujet comme un jeu à deux joueurs. Les deux joueurs sont MaxDébit (joueur max associé à la valeur 1) et MinLatence (joueur min associé à la valeur -1).

Une *configuration* est la donnée de deux sous-graphes G_1 et G_{-1} de G , d'arêtes disjointes, correspondant aux choix des arêtes effectués jusque-là par les deux joueurs, ainsi que la donnée du joueur qui doit jouer (il s'agit du joueur qui *contrôle* cette configuration). Un *coup* possible consiste à choisir une nouvelle arête du graphe non encore attribuée.

Une configuration est finale lorsque toutes les arêtes ont été attribuées et donc lorsque les arêtes de G_1 et G_{-1} partitionnent celles de G . La configuration initiale est contrôlée par MaxDébit et est composée de deux sous-graphes sans arêtes.

Pour tester la viabilité de cette procédure, le gouvernement commence par effectuer un essai sur le réseau de l'île de Kor e rattach e au Listenbourg, repr esent e   la **figure 5**.

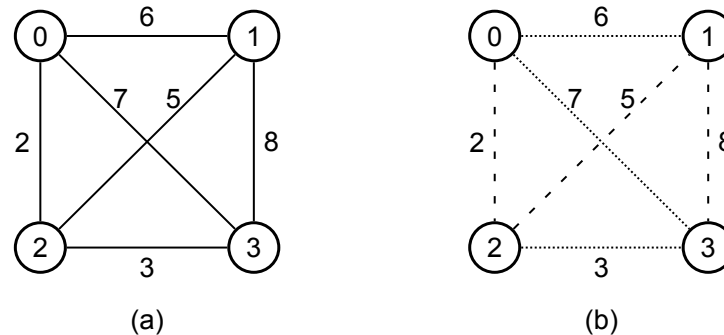


Figure 5 - (a) Le graphe correspondant au r seau de l' le de Kor e. (b) Exemple de configuration finale (contr ol e par MaxD bit) atteinte   l'issue d'une partie. Les ar tes choisies par le joueur MaxD bit sont repr esent es par un trait discontinu et celle par le joueur MinLatence par une ligne en pointill es

Un exemple possible de partie de ce jeu, dont l'issue est repr esent e   la **figure 5**, est la suivante :

- l'ar te $\{1, 3\}$ est choisie par MaxD bit ;
- l'ar te $\{0, 1\}$ est choisie par MinLatence ;
- l'ar te $\{1, 2\}$ est choisie par MaxD bit ;
- l'ar te $\{2, 3\}$ est choisie par MinLatence ;
- l'ar te $\{0, 2\}$ est choisie par MaxD bit ;
- l'ar te $\{0, 3\}$ est choisie par MinLatence.

Ici les deux op rateurs obtiennent la gestion d'un sous-r seau qui est exactement un arbre couvrant, mais ce n'est pas le cas en g n ral, le sous-graphe correspondant au sous-r seau obtenu par un joueur pouvant m me ne pas  tre connexe.

La **partie IV** montre que l'objectif d'un fournisseur d'acc s est de choisir un sous-graphe G' de mani re   maximiser la quantit  $\bar{b}_{\text{lim}}(G')$, c'est- -dire de maximiser la bande passante limite du sous-graphe. D'apr s la **partie IV**, pour cela, il s'agit tout d'abord d'obtenir un sous-graphe G' connexe (sans quoi $\bar{b}_{\text{lim}}(G') = -\infty$) puis, le cas  ch ant, de consid rer l'ar te de poids minimal de l'arbre couvrant de poids maximal de G' , $\bar{b}_{\text{lim}}(G')$  tant alors le poids de cette ar te. Un fournisseur d'acc s estime qu'il est gagnant si la bande passante limite de son sous-graphe $\bar{b}_{\text{lim}}(G')$ est strictement sup rieure   celle de son concurrent. Une configuration finale est donc gagnante pour MaxD bit si $\bar{b}_{\text{lim}}(G_1) > \bar{b}_{\text{lim}}(G_{-1})$ et gagnante pour MinLatence si $\bar{b}_{\text{lim}}(G_1) < \bar{b}_{\text{lim}}(G_{-1})$. Remarquons que si seul l'un des deux joueurs r ussit   obtenir un sous-graphe connexe, celui-ci est n cessairement gagnant. Il peut  galement y avoir match nul si aucun des deux joueurs n'obtient un graphe connexe   l'issue de la proc dure d'attribution.

En reprenant l'exemple propos    la **figure 5**, MaxD bit obtient un sous-graphe dont la plus petite ar te de son arbre couvrant est de poids 2 et MinLatence obtient un sous-graphe dont la plus petite ar te de son arbre couvrant est de poids 3. Le joueur MinLatence remporte donc la partie.

Q29. Justifier que ce jeu termine toujours, c'est- -dire que toute partie est n cessairement finie.

Q30. Justifier que la seule possibilit  de match nul est celle o  aucun des deux joueurs n'obtient un graphe connexe   l'issue de la partie.

Q31. Montrer, sur un exemple, que la strat gie consistant   toujours choisir l'ar te de poids maximal parmi les ar tes restantes n'est pas une strat gie gagnante pour le joueur MaxD bit.

V.2 - Implémentation en Python

On rappelle qu'en Python une boucle `for i in range(a, b)` permet de faire évoluer une variable `i` entre les valeurs `a` **include** et `b` **exclude**.

On modélise en Python le graphe pondéré $G = (S, A, p)$ par un objet `g` de type `Graphe` dont l'implémentation n'est pas précisée ici. Si `g` est un objet de type `Graphe`, `g.nb_sommets` est un entier correspondant à $n = |S|$ et `g.arettes` est une liste de longueur $m = |A|$ contenant les triplets (w, i, j) avec $0 \leq i < j < n$ et $w = p(\{i, j\})$ correspondant aux arêtes pondérées. On note $(a_k)_{k \in \llbracket 0, m-1 \rrbracket}$ les arêtes dans l'ordre de cette énumération.

Par exemple, si `g` représente le graphe de l'île de Korbe représenté à la **figure 5**, on aura par exemple :

```
>>> g.nb_sommets
4
>>> g.arettes
[(6.0, 0, 1), (2.0, 0, 2), (7.0, 0, 3), (5.0, 1, 2), (8.0, 1, 3), (3.0, 2, 3)]
```

On représente une configuration par un couple (t, etat) . La première composante `t` est un entier valant 1 si c'est au joueur MaxDébit de jouer et -1 si c'est au joueur MinLatence de jouer. La deuxième composante `etat` est une liste de longueur m avec, pour $k \in \llbracket 0, m-1 \rrbracket$, `etat[k] == 1` si l'arête a_k a déjà été choisie par le joueur MaxDébit, `etat[k] == -1` si l'arête a_k a déjà été choisie par le joueur MinLatence et `etat[k] == 0` si l'arête a_k n'a pas encore été choisie.

Par exemple, la configuration finale de la **figure 5** est représentée par le couple :

```
(1, [-1, 1, -1, 1, 1, -1])
```

En effet, l'état est contrôlé par le joueur MaxDébit, les arêtes d'indices 1, 3 et 4 ont été choisies par le joueur MaxDébit et les arêtes d'indices 0, 2, 5 par le joueur MinLatence.

La fonction suivante permet de renvoyer la configuration initiale correspondant à un graphe `g` :

```
def configuration_initiale(g):
    etat = [0] * len(g.arettes)
    return (1, etat)
```

La fonction suivante permet de savoir quel joueur contrôle une configuration. Elle renvoie 1 si c'est à MaxDébit de jouer et -1 si c'est à MinLatence de jouer.

```
def tour(c):
    t, etat = c
    return t
```

Q32. Écrire en Python une fonction `finale(c)` qui prend une configuration `c` et qui renvoie `True` si et seulement si la configuration est finale, c'est-à-dire si la partie est terminée.

On rappelle que si `etat` est une liste Python, l'instruction `suiv = etat.copy()` permet de créer une copie de la liste `etat` dans la variable `suiv`.

Q33. Écrire en Python une fonction `configurations_suivantes(c)` qui, étant donné une configuration `c` supposée non finale, renvoie la liste de toutes les configurations accessibles en un coup à partir de cette configuration, c'est-à-dire les configurations obtenues lorsque le joueur qui contrôle cette configuration choisit une nouvelle arête. La configuration `c` ne doit pas être modifiée : il est nécessaire d'en faire des copies.

V.3 - Calcul des attracteurs

On suppose disposer d'une fonction `gagnant(g, c)` qui, étant donné un graphe g et une configuration finale c , renvoie 1, 0 ou -1 suivant que la configuration finale est gagnante pour MaxDébit, un match nul ou gagnante pour MinLatence. Pour cela, cette fonction va calculer les sous-graphes G_1 et G_{-1} obtenus par les deux joueurs, vérifier si ceux-ci sont connexes, le cas échéant en calculer les arbres couvrants de poids maximal, puis le poids de l'arête de poids minimal de ces arbres couvrants, en déduire $\bar{b}_{\text{lim}}(G_1)$ et $\bar{b}_{\text{lim}}(G_{-1})$, les comparer et enfin renvoyer le gagnant (ou 0 pour match nul).

La fonction suivante permet d'attribuer un score dans $\{-1, 0, 1\}$ à une configuration non nécessairement finale.

```

1 def score(g, c):
2     if est_finale(c):
3         return gagnant(g, c)
4     t, etat = c
5     score_fils = [score(g, suiv) for suiv in configurations_suivantes(c)]
6     if t == 1:
7         return max(score_fils)
8     else:
9         return min(score_fils)

```

Q34. Indiquer à quoi correspondent les configurations (non nécessairement finales) de score -1 , de score 0 et de score 1.

Q35. Considérons une configuration non finale de score 1. Expliquer comment construire une stratégie gagnante pour MaxDébit à partir de cette configuration. On ne demande pas d'implémenter cette fonction en Python ni de montrer que cette stratégie est effectivement gagnante, mais uniquement d'expliquer comment l'obtenir.

Les trois sous-parties suivantes sont complètement indépendantes.

V.4 - Mémoïsation

L'implémentation précédente de la fonction `score` va conduire à recalculer un grand nombre de fois le score pour une même configuration. On se propose d'adopter une technique de mémoïsation. On se donne un dictionnaire `cache` qui sera, pour simplifier, une variable globale. Ce dictionnaire permet de mémoriser le score des configurations pour lesquelles celui-ci a déjà été calculé. Comme les listes ne peuvent pas être utilisées comme clés pour les dictionnaires, on utilisera la fonction `fige` ci-après pour convertir une configuration avec des listes en configuration avec des n -uplets avant de l'utiliser comme clé du dictionnaire `cache`.

```

def fige(c):
    t, etat = c
    return (t, tuple(etat))

```

Par exemple, en considérant la configuration obtenue après les trois premiers coups lors de la partie donnée en exemple dans la **sous-partie V.1**, on obtient :

```

>>> c_ex = (-1, [-1, 0, 0, 1, 1, 0])
>>> fige(c_ex)
(-1, (-1, 0, 0, 1, 1, 0))

```

- Q36.** Compléter le squelette de la fonction `score_memo` ci-dessous pour que celle-ci se comporte exactement comme la fonction `score` de la **sous-partie V.3**, mais en utilisant le dictionnaire `cache` pour ne pas recalculer plusieurs fois le score d'une même configuration.

```
cache = {}

def score_memo(g, c):
    # On peut utiliser dans cette fonction la variable globale `cache`
    ...
```

V.5 - Algorithme MinMax avec heuristique et exploration jusqu'à une profondeur p

Même en utilisant une fonction mémorisée, le coût du calcul du score de toutes les configurations avec l'approche précédente peut vite se révéler prohibitif. On se propose dans cette sous-partie d'utiliser l'algorithme MinMax avec une heuristique en limitant l'exploration jusqu'à une profondeur $p \in \mathbb{N}$. On suppose disposer d'une heuristique, c'est-à-dire une évaluation approximative de la qualité d'une configuration non finale sous la forme d'un score dans $[-1, 1]$, avec un score positif si la configuration semble favorable pour MaxDébit et un score négatif si la configuration semble favorable pour MinLatence. On suppose donc disposer d'une fonction heuristique telle que `heuristique(g, c)` pour un graphe g et une configuration non finale c renvoie un score dans $[-1, 1]$.

- Q37.** Écrire en Python une fonction `score_minmax(g, c, p)` sur le modèle de la fonction `score` de la **sous-partie V.3** (sans la mémorisation de la **sous-partie V.4**) telle que `score_minmax(g, c, p)` calcule un score pour une configuration c , mais en limitant l'exploration à une profondeur $p \in \mathbb{N}$, et en utilisant l'heuristique comme substitut lorsque l'on atteint la limite de profondeur.

V.6 - Vol de stratégie

- Q38.** Montrer que, quel que soit le graphe connexe à pondération injective, il n'existe pas de stratégie gagnante pour MinLatence. *On pourra remarquer qu'avoir un coup arbitraire en plus ne peut qu'être bénéfique et procéder par l'absurde en supposant que le deuxième joueur dispose d'une stratégie gagnante pour construire une stratégie gagnante pour le premier joueur.*

Ainsi, il existe pour MaxDébit une stratégie lui assurant de ne pas perdre : elle lui assure soit de gagner soit de forcer un match nul. Le gouvernement du Listenbourg semble avoir privilégié un des deux opérateurs.

FIN